Auditoría técnica del Vendedor IA (Proshop)

- Repositorio: `jonathanSCM/Agentes` · rama `main` · commit `7eedaca`
- Generado: 2026-10-23
- **Todo el código de este documento está copiado textualmente de archivos del repositorio**, con su ruta y número de línea. No hay pseudocódigo ni ejemplos externos.
- Los datos de consumo salen de la tabla `usos_ia` de la base de producción (llamadas reales de producción medidas hasta el 15/09/2026; los únicos días de septiembre con tráfico real registrado fueron 02, 10 y 15).
- No se modificó nada del sistema para generar este documento.

Índice

0. Respuestas directas (modelos, llamadas por mensaje, reintentos) y hallazgos
1. Integración con OpenAI: configuración de modelos y llamadas
2. Prompt e instrucciones: qué recibe la IA en cada solicitud
3. Flujo de WhatsApp: mensajes, botones, imágenes y respuestas
4. Carrito de compras
5. Catálogo web y su conexión con WhatsApp
6. Pedidos y pagos
7. Bucle de llamadas: vueltas, detectores, reintentos y respuestas sin IA
8. Modelos de datos involucrados (Prisma)

0. Respuestas directas

0.1 ¿Qué modelos se usan y dónde se configuran?

Plan	OpenAI (producción)	Anthropic (solo si NO hay `OPENAI_API_KEY`)
GRATIS	`gpt-4o-mini`	`claude-haiku-4-5`
STANDARD	`gpt-4.1-mini`	`claude-haiku-4-5`
PRO	`gpt-4o`	`claude-sonnet-5`
PREMIUM	`gpt-4o`	`claude-opus-4-8`

- El proveedor se decide con `proveedorActivo()`: si existe `OPENAI_API_KEY` usa OpenAI; si no, `ANTHROPIC_API_KEY`; si no hay ninguna, modo demo (sin IA).
- Cada turno elige el modelo así (`generarRespuesta`): si el cliente **todavía no tiene categoría de interés ni producto favorito** (`esTurnoSimple`), se usa siempre `gpt-4o-mini`, sin importar el plan. En cuanto hay categoría o favorito, se usa el modelo del plan (`modeloParaPlan`).
- La columna `Plan.modeloIa` (base de datos) pisa el mapa del código, **excepto** si empieza con `gpt-5` (se ignora a propósito).
- **En el código no existe "GPT-4.1 nano"** ni ningún modelo "Luna/Terra". El único `gpt-4.1` configurado es `gpt-4.1-mini` (plan STANDARD).
- Modelos OpenAI usados fuera del bucle principal: visión de fotos del cliente `gpt-4o-mini` (fijo en código), audios `whisper-1`, embeddings `text-embedding-3-small`.

0.2 ¿Se hacen varias llamadas a OpenAI por cada mensaje del cliente? Sí.

En `generarRespuesta` hay un bucle de hasta `MAX_VUELTAS = 3` vueltas. **Cada vuelta es una llamada completa** a `chat.completions.create` con el prompt completo (sistema + historial + mensaje + resultados de tools). Una vuelta adicional ocurre cuando:

1. La IA llamó una *tool* (mostrar productos, agregar al carrito, confirmar pedido…): el resultado vuelve al modelo en una vuelta nueva para que redacte el texto final; o
2. Un *detector anti-invención* (regex puro, sin costo de IA) rechazó el texto de la IA y se la obliga a corregirse (`RECORDATORIO DEL SISTEMA` + `continue`).

Medición real en producción (tabla `usos_ia`, 113 llamadas):

Día	Mensajes de clientes	Llamadas a OpenAI	Llamadas por mensaje	% de llamadas con vuelta > 0
02/09	17	26	1,53	46,2 %
10/09	37	69	1,86	52,2 %
15/09	12	18	1,50	44,4 %

> Aclaración importante sobre esa última columna: la columna `usos_ia.proposito` marca como `reintento_forzado` **toda** llamada con `vuelta > 0`. Eso mezcla (1) las vueltas "normales" de redacción posteriores a una tool y (2) los reintentos forzados por detectores. Hoy el sistema **no distingue entre ambas**; el 44–52 % es el total de vueltas adicionales, no solo reintentos por error.

Tamaño real del prompt medido por llamada: entre ~5.300 y ~8.300 tokens de entrada, con entre 0 y ~8.000 tokens servidos desde caché (prompt caching automático de OpenAI).

Mensajes que cuestan 0 llamadas (el backend responde solo): menú de categorías forzado, botones de forma de pago, resumen del pedido forzado por código, tarjeta de categoría preemptiva, y los atajos de carrito (ver sección 7).

0.3 ¿Existen reintentos automáticos?

Mecanismo	Dónde	Qué hace
Vueltas por detectores	`generarRespuesta` (sección 7)	Si el texto de la IA dispara un detector, se agrega un mensaje "RECORDATORIO DEL SISTEMA" y se vuelve a llamar (máx. 3 vueltas en total). En la última vuelta no se reintenta: se fuerza una respuesta por código.
Rescate por rate limit	bucle de `generarRespuesta` (sección 7.2)	Si OpenAI responde 429 y el modelo no era el económico, se repite **una vez** la misma llamada con `gpt-4o-mini`.
Reintentos del SDK	`getOpenAI()` / `getAnthropic()`	El código crea los clientes con `new OpenAI()` / `new Anthropic()` **sin** `maxRetries` ni `timeout` (verificado: no existen esas opciones en el repo), así que rigen los valores por defecto de los SDK (según la documentación de cada SDK: 2 reintentos automáticos ante errores de conexión, 408, 409, 429 y 5xx). No son visibles en nuestros logs ni en `usos_ia`.
Error de proveedor	`catch` final de `generarRespuesta`	No reintenta: responde con `respuestaErrorTecnico()`.
Agrupación de mensajes	`bufferMensajes.js`	No es un reintento, pero reduce llamadas: espera 5 s (`MENSAJE_DEBOUNCE_MS`) sin mensajes nuevos del mismo cliente antes de generar la respuesta.

0.4 Otras llamadas a OpenAI que NO son el bucle principal

Disparador	Función	Modelo	Nota
El cliente envía una **foto**	`analizarImagenProducto` (`agente.js`)	`gpt-4o-mini`	El prompt incluye los nombres de hasta **200 productos activos** de la empresa (`take: 200` en el webhook). Toda imagen se trata como "foto de un producto".
El cliente envía un **audio**	`transcribirAudio` (`agente.js`)	`whisper-1`	Se transcribe y el texto entra como mensaje normal.
Crear/editar un producto	`guardarEmbeddingDeProducto` (`embeddings.js`, llamado desde `server.js`)	`text-embedding-3-small`	Segundo plano, sin esperar.
Tool `buscar_producto` sin coincidencia literal	`buscarPorSimilitud` (`embeddings.js`)	`text-embedding-3-small`	Solo como respaldo cuando el nombre exacto no existe.

Ninguna de estas cuatro llamadas se registra en `usos_ia`: solo se persiste el consumo del bucle principal (`prisma.usoIA.create` existe en un único lugar, `agente.js` dentro del bucle). Por lo tanto el consumo real en la factura de OpenAI siempre será **mayor** que lo que muestra el dashboard `/admin/costo-ia`.

0.5 Hallazgos de la lectura del código (para optimizar y corregir el carrito)

1. **Cada vuelta reenvía el prompt completo** (~6–8 mil tokens). El ahorro viene de reducir vueltas, no de acortar la respuesta (la salida es de 12–200 tokens).
2. `proposito` **en** `usos_ia` **no distingue** vuelta de redacción vs. reintento forzado (ver 0.2).
3. **Posible pérdida del carrito al cambiar entre catálogo web y WhatsApp** (a verificar con logs): `carritoDe(contexto, conversacionId)` devuelve `[]` si el `conversacionId` guardado en el carrito no coincide con el que se le pasa. El token del link web (`sesionWeb.js`) vive 24 h, pero una conversación nueva se abre cuando pasaron más de 6 h desde el último mensaje (`VENTANA_HORAS`). Si el cliente usa en la web un link enviado en una conversación anterior, la ruta `POST /catalogo/:slug/producto/:id/carrito` lee el carrito con el id viejo (vacío), agrega el ítem y hace `guardarCarrito(..., sesion.conversacionId, ...)`: **sobrescribe** el carrito de la conversación actual y lo deja asociado al id viejo, invisible para el bot.
4. **Cantidades:** `agregarItem` suma cantidades si es la misma línea (producto + variante) y `quitarItem` elimina la línea completa. No existe una tool para "dejar la cantidad en N" ni para restar unidades.
5. **Precios:** cada ítem guarda el precio del momento en que se agregó, pero al armar el pedido (`confirmar_pedido`/`crear_pedido`) el precio y el total se **recalculan desde la base de datos** (el precio guardado en el carrito se ignora). El stock del carrito web/tool se valida por cada alta individual, no contra la cantidad ya acumulada; la validación final ocurre al confirmar.
6. **Imágenes:** cualquier foto se analiza como "foto de producto". En la conversación real #134 (10/09) el cliente envió una foto de su comprobante de pago y el bot contestó que no tenía información de ese producto (mensajes 2372–2377). No hay manejo de comprobantes.
7. **Pagos:** el QR es una imagen estática (`config.qrCobroUrl`) que se envía al crear el pedido (`crear_pedido`) si el cliente eligió QR; el sistema no verifica el pago automáticamente.
8. **Webhook sin deduplicación:** el handler no usa el id del mensaje de Meta para descartar reenvíos (solo lo usa para marcar "leído").
9. **Consumo no registrado:** visión, Whisper y embeddings no se guardan en `usos_ia` (ver 0.4).

---

1. Integración con OpenAI

1.1 Configuración de modelos y clientes

Modelos por plan, modelo económico, selección de proveedor y clientes

Archivo: `lib/services/agente.js` *· líneas 148–218*

```
// Modelos por defecto de cada proveedor (se pueden cambiar por variables de entorno)
const OPENAI_MODEL_DEFAULT = process.env.OPENAI_MODEL || 'gpt-4o-mini';
const ANTHROPIC_MODEL_DEFAULT = 'claude-haiku-4-5';

// Mapa de modelos por plan: mejor plan -> mejor modelo.
const MODELOS_POR_PLAN = {
  openai: {
    GRATIS: 'gpt-4o-mini',
    STANDARD: 'gpt-4.1-mini',
    PRO: 'gpt-4o',
    // Hardcodeado a proposito: gpt-4o, NUNCA gpt-5.x. Pedido explicito del
    // negocio - ver el guard en modeloParaPlan mas abajo, que ademas
    // rechaza cualquier gpt-5* que venga configurado en Plan.modeloIa (la
    // base todavia puede tener ese valor viejo, ver prisma/seed.js).
    PREMIUM: 'gpt-4o',
  },
  anthropic: {
    GRATIS: 'claude-haiku-4-5',
    STANDARD: 'claude-haiku-4-5',
    PRO: 'claude-sonnet-5',
    PREMIUM: 'claude-opus-4-8',
  },
};

// El modelo mas barato de cada proveedor: se usa en turnos "simples" (ver
// esTurnoSimple mas abajo) independientemente del plan, para no pagar el
// modelo caro cuando todavia no hay nada de catalogo real en juego.
const MODELO_ECONOMICO = {
  openai: 'gpt-4o-mini',
  anthropic: 'claude-haiku-4-5',
};

function modeloEconomico(proveedor) {
  return MODELO_ECONOMICO[proveedor] || MODELO_ECONOMICO.openai;
}

function modeloParaPlan(proveedor, plan) {
  const configurado = plan && plan.modeloIa;
  const codigo = plan && plan.codigo;
  if (proveedor === 'openai') {
    // gpt-5.x nunca se usa, pase lo que pase (pedido explicito del
    // negocio) - aunque Plan.modeloIa en la base todavia diga gpt-5.2 (dato
    // viejo, ver prisma/seed.js), se ignora y cae al mapa de arriba.
```

```
    if (configurado && /^gpt/i.test(configurado) && !/^gpt-5/i.test(configurado)) return configurado;
    return MODELOS_POR_PLAN.openai[codigo] || OPENAI_MODEL_DEFAULT;
  }
  if (configurado && /^claude/i.test(configurado)) return configurado;
  return MODELOS_POR_PLAN.anthropic[codigo] || ANTHROPIC_MODEL_DEFAULT;
}

function proveedorActivo() {
  if (process.env.OPENAI_API_KEY) return 'openai';
  if (process.env.ANTHROPIC_API_KEY) return 'anthropic';
  return 'demo';
}

let openaiClient = null;
function getOpenAI() {
  if (openaiClient) return openaiClient;
  const OpenAI = require('openai');
  openaiClient = new OpenAI();
  return openaiClient;
}

let anthropicClient = null;
function getAnthropic() {
  if (anthropicClient) return anthropicClient;
  const Anthropic = require('@anthropic-ai/sdk');
  anthropicClient = new Anthropic();
  return anthropicClient;
}
```

1.2 Las dos funciones que hacen la llamada al proveedor

llamarOpenAI (chat.completions.create con tools)

Archivo: `lib/services/agente.js` · líneas 1781–1832

```
async function llamarOpenAI({ system, mensajes, tools, toolChoice, modelo }) {
  // El caching de OpenAI es automatico por prefijo identico: no necesita
  // ningun cache_control explicito, solo que "system" llegue con la parte
  // fija primero (ver partesDelSystem/construirSystem).
  const openai = getOpenAI();
  const convertidos = mensajes.map((m) => {
    if (m.role === 'assistant' && m.tool_calls?.length) {
      return {
        role: 'assistant',
        content: m.content || null,
        tool_calls: m.tool_calls.map((tc) => ({
          id: tc.id,
          type: 'function',
          function: { name: tc.name, arguments: JSON.stringify(tc.arguments || {}) },
        })),
      };
```

```
    }
    if (m.role === 'tool') return { role: 'tool', tool_call_id: m.tool_call_id, content: m.content };
    return { role: m.role, content: m.content };
  });

  const params = {
    model: modelo,
    messages: [{ role: 'system', content: system }, ...convertidos],
    tools,
    // 'requerido' -> 'required' (la API nunca deja pasar texto libre sin
    // llamar una tool). Guard extra por las dudas: con tools:[] (ultima
    // vuelta) 'required' rompe la llamada - nunca deberia pasar (el call
    // site ya excluye esUltimaVuelta), pero se protege igual.
    tool_choice: toolChoice === 'requerido' && tools?.length ? 'required' : 'auto',
  };
  const campoTokens = /^(gpt-5|o[0-9])/i.test(modelo) ? 'max_completion_tokens' : 'max_tokens';
  params[campoTokens] = 700;
  // Idea tomada de un bot de citas (DentalFlow): bajar la temperatura en las
  // llamadas con tools reduce la improvisacion (menos variedad creativa,
  // mas apego a los datos reales del bloque de resultados). Antes esto
  // corria en el default de la API (~1.0). Los modelos o1/o3 (reasoning)
  // rechazan este parametro - se excluyen.
  if (!/^o[0-9]/i.test(modelo)) params.temperature = 0.3;

  const resp = await openai.chat.completions.create(params);
  const msg = resp.choices[0].message;
  return {
    content: msg.content || '',
    tool_calls: (msg.tool_calls || []).map((tc) => ({
      id: tc.id,
      name: tc.function.name,
      arguments: JSON.parse(tc.function.arguments || '{}'),
    })),
    usage: resp.usage,
  };
}
```

llamarAnthropic (messages.create con cache_control)

Archivo: `lib/services/agente.js` *· líneas 1834–1881*

```
async function llamarAnthropic({ system, systemFijo, systemVariable, mensajes, tools, toolChoice, modelo }) {
  const anthropic = getAnthropic();
  const convertidos = mensajes.map((m) => {
    if (m.role === 'assistant' && m.tool_calls?.length) {
      const bloques = [];
      if (m.content) bloques.push({ type: 'text', text: m.content });
      for (const tc of m.tool_calls) bloques.push({ type: 'tool_use', id: tc.id, name: tc.name, input: tc.arguments || {} });
      return { role: 'assistant', content: bloques };
    }
    if (m.role === 'tool') {
      return { role: 'user', content: [{ type: 'tool_result', tool_use_id: m.tool_call_id, content: String(m.content || '') }]
```

```
};
    }
    return { role: m.role, content: m.content };
  });

  // Si viene la parte fija/variable por separado, se arma como dos bloques
  // con cache_control en el primero: Anthropic reutiliza (90% mas barato)
  // todo lo que este ANTES de ese corte cuando el texto es identico a una
  // llamada anterior reciente. Sin las partes separadas (compatibilidad),
  // se manda como string plano, igual que antes.
  const systemParaAnthropic = systemFijo != null
    ? [
        { type: 'text', text: systemFijo, cache_control: { type: 'ephemeral' } },
        { type: 'text', text: systemVariable || '' },
      ]
    : system;

  const resp = await anthropic.messages.create({
    model: modelo,
    max_tokens: 700,
    // Idea tomada de un bot de citas (DentalFlow): bajar la temperatura
    // reduce la improvisacion. Antes corria en el default del SDK (~1.0).
    temperature: 0.3,
    system: systemParaAnthropic,
    messages: convertidos,
    tools: toolsParaAnthropic(tools),
    // 'requerido' -> { type: 'any' } (Anthropic exige llamar ALGUNA tool,
    // sin especificar cual - equivalente a 'required' de OpenAI). Mismo
    // guard que en llamarOpenAI: sin tools no tiene sentido forzar.
    tool_choice: toolChoice === 'requerido' && tools?.length ? { type: 'any' } : { type: 'auto' },
  });

  const texto = resp.content.filter((b) => b.type === 'text').map((b) => b.text).join('\n').trim();
  const toolCalls = resp.content
    .filter((b) => b.type === 'tool_use')
    .map((b) => ({ id: b.id, name: b.name, arguments: b.input || {} }));
  return { content: texto, tool_calls: toolCalls, usage: resp.usage };
}
```

1.3 Elección de proveedor y modelo en cada turno

generarRespuesta: proveedor, esTurnoSimple y modelo

Archivo: `lib/services/agente.js` *· líneas 3269–3292*

```
  const proveedor = proveedorActivo();
  const plan = empresa.suscripcion && empresa.suscripcion.plan;

  // Ruteo por costo: lo dificil (y lo que justifica el modelo caro) es
  // entender que quiere el cliente y elegir bien que mostrarle - no saludar
  // ni pasar el menu de categorias. Mientras el cliente todavia no eligio
```

```
  // ninguna categoria (candidatosActuales SIEMPRE tiene algo si hay stock,
  // aunque no haya dicho nada: sin categoria, buscarConFallback no filtra
  // nada y devuelve todo el catalogo - por eso no sirve como señal) ni tiene
  // nada en el carrito/favorito, el turno es "de orientacion": saludo, ayuda
  // generica, ver el menu de categorias. Ahi se usa el modelo economico sin
  // importar el plan. En cuanto sabemos que categoria le interesa (lo
  // detecta el codigo, no la IA, apenas lo dice) o ya eligio un favorito, se
  // pasa al modelo del plan: ahi es donde de verdad se juega entender la
  // necesidad del cliente y no trabarse mostrando.
  const esTurnoSimple = !lead.categoriaInteres && !lead.productoFavoritoId;
  const modelo = esTurnoSimple ? modeloEconomico(proveedor) : modeloParaPlan(proveedor, plan);
  logEtapa('modelo_elegido', telefonoCliente, { proveedor, modelo, esTurnoSimple, plan: plan && plan.codigo });
  const llamar = opciones.llamarInyectado || (proveedor === 'openai' ? llamarOpenAI : proveedor === 'anthropic' ?
llamarAnthropic : null);

  if (!llamar) {
    logEtapa('modo_demo', telefonoCliente, { motivo: 'sin_api_key' });
    return { ok: true, demo: true, proveedor: 'demo', respuesta: respuestaDemo(empresa, productos, mensajeCliente) };
  }
```

1.4 La llamada dentro del bucle: rescate por 429 y registro de uso

El código completo de este tramo (armado de parámetros, la llamada a llamar(...), el rescate por 429 con modeloEconomico y el prisma.usoIA.create) está en la **sección 7.2**.

1.5 Llamadas fuera del bucle (visión, audio, embeddings)

ejecutarFuncion: buscar_producto (usa embeddings como respaldo)

Archivo: `lib/services/agente.js` *· líneas 2341–2381*

```
  if (toolCall.name === 'buscar_producto') {
    // Busca en TODO el catalogo de la empresa, sin los filtros de la
    // conversacion: el cliente nombro algo puntual y merece una respuesta
    // sobre si existe o no, no sobre si calza con el color que pidio antes.
    const buscado = String(args.nombre || '').trim();
    if (!buscado) return 'Decime que nombre busco.';

    const { conStock, agotados } = buscarPorNombre(contexto, buscado);
    // El cliente nombro estos productos: se anotan para que mostrar_productos
    // NO les agregue relleno despues (ver "pedido puntual" mas abajo). Bug
    // real con capturas: pidio "Zapatillas Park St 2.0" y recibio tres
    // tarjetas, la suya ultima.
    helpers.idsPedidosPorNombre = [...new Set([...(helpers.idsPedidosPorNombre || []), ...conStock.map((p) => p.id)])];
    logEtapa('busqueda_por_nombre', telefonoCliente, { buscado, conStock: conStock.length, agotados: agotados.length });

    if (!conStock.length && !agotados.length) {
      // El nombre literal no matcheo nada: antes de decir que no existe, se
      // intenta un fallback semantico (embeddings) por si el cliente uso
      // otra palabra para lo mismo ("remera" cuando el catalogo dice
      // "polera"). Nunca reemplaza el matching literal, solo es la ultima
```

```
      // red de seguridad antes de negar que algo existe.
      const idsConStock = contexto.filter((p) => tieneStock(p)).map((p) => p.id);
      const idsSimilares = await buscarPorSimilitud(empresaId, buscado, idsConStock, 3);
      const porSimilitud = idsSimilares.map((id) => contexto.find((p) => p.id === id)).filter(Boolean);
      if (porSimilitud.length) {
        logEtapa('busqueda_semantica_encontro', telefonoCliente, { buscado, ids: idsSimilares });
        return `No hay ningun producto con ese nombre exacto, pero esto es lo mas parecido en significado que se encontro en el catalogo:
${formatearProductos(porSimilitud, moneda)}
Ofreceselo como posible coincidencia (no digas que es exactamente lo que pidio, aclara que es lo mas parecido) y mostraselo con mostrar_productos si le interesa.`;
      }
      return `Buscado "${buscado}" en TODO el catalogo: no existe ningun producto con ese nombre. Ahora si podes decirle con seguridad que no lo manejan, y ofrecerle lo que si hay.`;
    }
    if (!conStock.length) {
      return `"${agotados.map((p) => p.nombre).join(', ')}" SI existe en el catalogo pero esta SIN STOCK. Deciselo asi: que lo manejan pero ahora no hay, y ofrecele las opciones que si tienen. NUNCA digas que no lo conoces.`;
    }
    return `ENCONTRADO en el catalogo, con stock:
${formatearProductos(conStock.slice(0, MAX_PRODUCTOS_A_MOSTRAR), moneda)}
${agotados.length ? `(Ademas existe "${agotados.map((p) => p.nombre).join(', ')}" pero sin stock: no lo ofrezcas.)
` : ''}
Mostraselo YA con mostrar_productos usando esos IDs. PROHIBIDO decirle que no lo tenes: lo tenes.`;
  }
```

analizarImagenProducto (visión, gpt-4o-mini)

Archivo: `lib/services/agente.js` *· líneas 4218–4268*

```
// Analiza una foto que mando el cliente y trata de reconocer si corresponde
// a un producto real del catalogo (para que el vendedor pueda reaccionar con
// criterio: "si, eso es la zapatilla X" en vez de ignorar la imagen).
async function analizarImagenProducto(buffer, mimeType, productos) {
  const proveedor = proveedorActivo();
  if (proveedor === 'demo') return null;

  const listado = productos.length
    ? productos.map((p) => `${p.nombre} (categoria: ${p.categoria?.nombre || 'General'})`).join(', ')
    : '(el catalogo esta vacio)';
  const instruccion = `Un cliente de una tienda envio esta foto por WhatsApp, probablemente preguntando por un producto. El catalogo REAL de la tienda tiene estos productos: ${listado}.
Mira la imagen y en 1-2 frases breves (en español) describe que se ve. Si se parece con claridad a alguno de esos productos EXACTOS, decilo por su nombre exacto tal como esta arriba. Si no se parece a ninguno, decí simplemente que no coincide con el catalogo. No fuerces una coincidencia si no es real.`;
  const media = mimeType || 'image/jpeg';
  const base64 = buffer.toString('base64');

  try {
    if (proveedor === 'openai') {
      const openai = getOpenAI();
      const resp = await openai.chat.completions.create({
```

```
        model: 'gpt-4o-mini',
        max_tokens: 200,
        messages: [{
          role: 'user',
          content: [
            { type: 'text', text: instruccion },
            { type: 'image_url', image_url: { url: `data:${media};base64,${base64}` } },
          ],
        }],
      });
      return resp.choices[0].message.content?.trim() || null;
    }
    if (proveedor === 'anthropic') {
      const anthropic = getAnthropic();
      const resp = await anthropic.messages.create({
        model: 'claude-haiku-4-5',
        max_tokens: 200,
        messages: [{
          role: 'user',
          content: [
            { type: 'image', source: { type: 'base64', media_type: media, data: base64 } },
            { type: 'text', text: instruccion },
          ],
        }],
      });
      return resp.content.filter((b) => b.type === 'text').map((b) => b.text).join(' ').trim() || null;
    }
  } catch (err) {
    console.error('Error analizando imagen del cliente:', err.message);
  }
  return null;
}
```

transcribirAudio (whisper-1)

Archivo: `lib/services/agente.js` *· líneas 4270–4287*

```
// Transcribe un audio/nota de voz que mando el cliente. Solo disponible via
// Whisper (OpenAI); si la plataforma esta configurada solo con Anthropic (que
// no ofrece transcripcion de audio), devuelve null y el llamador debe
// degradar con gracia (pedirle al cliente que lo escriba).
async function transcribirAudio(buffer, mimeType) {
  if (!process.env.OPENAI_API_KEY) return null;
  try {
    const { toFile } = require('openai');
    const openai = getOpenAI();
    const extension = /ogg/.test(mimeType || '') ? 'ogg' : /mp4|m4a/.test(mimeType || '') ? 'm4a' : 'mp3';
    const archivo = await toFile(buffer, `audio.${extension}`);
    const resp = await openai.audio.transcriptions.create({ file: archivo, model: 'whisper-1', language: 'es' });
    return resp.text?.trim() || null;
  } catch (err) {
    console.error('Error transcribiendo audio del cliente:', err.message);
```

```
    return null;
  }
}
```

embeddings.js completo (text-embedding-3-small)

Archivo: `lib/services/embeddings.js` *· líneas 1–88*

```
// Busqueda semantica para el catalogo (Opcion B de
// docs/05-propuesta-personalizacion-ia.md): que el bot encuentre un producto
// aunque el cliente no use las palabras exactas del catalogo ("polera" vs
// "remera", errores de tipeo). NO reemplaza el matching literal, que sigue
// siendo el camino principal (gratis, determinista, instantaneo) - esto es
// una red de seguridad para cuando el literal no encuentra nada.
//
// Sin pgvector (no disponible en este entorno): el embedding se guarda como
// un array simple (Producto.embedding, Float[]) y la similitud coseno se
// calcula en JS. Para un catalogo de 100-300 productos por tienda esto es
// instantaneo - no hace falta una extension de base de datos.
//
// Nunca bloquea nada: sin OPENAI_API_KEY, o si la red falla, todo esto
// devuelve null/[] y el catalogo sigue funcionando solo con matching literal.
const { prisma } = require('../db');

const MODELO_EMBEDDING = 'text-embedding-3-small';
// Umbral minimo de similitud coseno para aceptar un match semantico: por
// debajo de esto, es mejor decir la verdad ("no lo tenemos") que forzar un
// producto que en realidad no tiene nada que ver.
const UMBRAL_SIMILITUD = 0.3;

let openaiClient = null;
function getOpenAI() {
  if (openaiClient) return openaiClient;
  if (!process.env.OPENAI_API_KEY) return null;
  const OpenAI = require('openai');
  openaiClient = new OpenAI();
  return openaiClient;
}

async function generarEmbedding(texto) {
  const openai = getOpenAI();
  if (!openai || !texto || !texto.trim()) return null;
  try {
    const resp = await openai.embeddings.create({ model: MODELO_EMBEDDING, input: texto.slice(0, 8000) });
    return resp.data[0].embedding;
  } catch (err) {
    console.error('Error generando embedding:', err.message);
    return null;
  }
}

async function guardarEmbeddingDeProducto(productoId, texto) {
```

```
  const embedding = await generarEmbedding(texto);
  if (!embedding) return false;
  try {
    await prisma.producto.update({ where: { id: productoId }, data: { embedding } });
    return true;
  } catch (err) {
    console.error('Error guardando embedding de producto:', err.message);
    return false;
  }
}

// Producto punto normalizado por las normas: 1 = identicos, 0 = sin relacion,
// negativo = opuestos. Pura y determinista, sin red ni DB.
function similitudCoseno(a, b) {
  if (!a || !b || !a.length || a.length !== b.length) return 0;
  let punto = 0, normaA = 0, normaB = 0;
  for (let i = 0; i < a.length; i++) {
    punto += a[i] * b[i];
    normaA += a[i] * a[i];
    normaB += b[i] * b[i];
  }
  if (!normaA || !normaB) return 0;
  return punto / (Math.sqrt(normaA) * Math.sqrt(normaB));
}

async function buscarPorSimilitud(empresaId, textoConsulta, idsPermitidos, limite = 5) {
  if (!idsPermitidos || !idsPermitidos.length) return [];
  const embeddingConsulta = await generarEmbedding(textoConsulta);
  if (!embeddingConsulta) return [];

  const productos = await prisma.producto.findMany({
    where: { id: { in: idsPermitidos }, empresaId },
    select: { id: true, embedding: true },
  });

  return productos
    .map((p) => ({ id: p.id, similitud: similitudCoseno(embeddingConsulta, p.embedding) }))
    .filter((p) => p.similitud >= UMBRAL_SIMILITUD)
    .sort((a, b) => b.similitud - a.similitud)
    .slice(0, limite)
    .map((p) => p.id);
}

module.exports = { generarEmbedding, guardarEmbeddingDeProducto, similitudCoseno, buscarPorSimilitud, UMBRAL_SIMILITUD };
```

1.6 Cálculo de costo (usado por el dashboard /admin/costo-ia)

costoIA.js completo

Archivo: `lib/services/costoIA.js` *· líneas 1–18*

```
// USD por 1M tokens. Fuente: precios publicos de cada proveedor al momento de
// escribir esto - pueden cambiar, por eso queda aislado en un solo lugar.
const PRECIOS_USD_POR_1M = {
  'gpt-4o': { input: 2.50, cachedInput: 1.25, output: 10.00 },
  'gpt-4o-mini': { input: 0.15, cachedInput: 0.075, output: 0.60 },
  'gpt-4.1-mini': { input: 0.40, cachedInput: 0.10, output: 1.60 },
  'claude-haiku-4-5': { input: 1.00, cachedInput: 0.10, output: 5.00 },
  'claude-sonnet-5': { input: 3.00, cachedInput: 0.30, output: 15.00 },
};

function costoUSD(modelo, { promptTokens = 0, cachedTokens = 0, completionTokens = 0 } = {}) {
  const precio = PRECIOS_USD_POR_1M[modelo];
  if (!precio) return null; // modelo desconocido: no se inventa un costo
  const noCacheado = Math.max(0, promptTokens - cachedTokens);
  return (noCacheado * precio.input + cachedTokens * precio.cachedInput + completionTokens * precio.output) / 1_000_000;
}

module.exports = { PRECIOS_USD_POR_1M, costoUSD };
```

---

2. Prompt e instrucciones: qué recibe la IA en cada solicitud

Cada llamada recibe: (a) `system` = parte **fija** (reglas, igual para toda la empresa; es lo que se cachea) + parte **variable** (memoria del lead y productos candidatos), (b) historial (últimos 20 mensajes, sin el mensaje actual), (c) el mensaje actual del cliente, (d) los mensajes del turno (resultados de tools y recordatorios), (e) la lista de tools. El catálogo **nunca se envía completo**: el código filtra los candidatos y solo esos entran al prompt.

2.1 Prompt del sistema

partesDelSystem (arma la parte fija y la variable)

Archivo: `lib/services/agente.js` *· líneas 1613–1765*

```
// Construye el prompt de sistema con la identidad del negocio y su catalogo.
// sinHerramientas=true se usa SOLO en la ultima vuelta del loop de tools,
// cuando ya no puede llamar mas funciones y tiene que redactar el mensaje
// final: ahi se refuerza con mas fuerza la regla anti-invento, porque es el
// momento de mas riesgo de que "complete" la respuesta con datos inventados.
// esPrimerMensaje=true cuando el cliente todavia no tiene historial: el bot
// debe tomar la iniciativa y presentarse, no esperar pasivamente.
// Separa el system prompt en dos mitades para aprovechar el prompt caching
// de OpenAI/Anthropic (50-90% mas barato en la parte que se repite igual
// entre llamadas). "fijo" es identico en cada vuelta/mensaje/cliente de la
// MISMA empresa (solo depende de marca/tono/config, que no cambian turno a
// turno) - es la parte mas pesada del prompt (reglas de venta, formato).
// "variable" es todo lo que SI cambia por turno: memoria del lead, atributos
// faltantes, y el catalogo/resultados reales de esta busqueda puntual.
// Nunca mezcla nada entre empresas: la marca va en la primera linea de
// "fijo", asi que el prompt de cada tienda es un texto distinto desde el
// primer caracter - cada una tiene su propio cache, aislado.
function partesDelSystem(empresa, productos, config, lead = {}, sinHerramientas = false, esPrimerMensaje = false, nombreAgente
= '', tarjetaEnviadaEnTurno = false, pedidosPrevios = [], pidioProductoPuntual = false) {
  const marca = empresa.marca || empresa.nombre;
  const tono = (config && config.tono) || 'cordial y profesional';
  const extra = (config && config.instrucciones) || '';
  const bienvenida = (config && config.mensajeBienvenida) || '';
  const metodosPago = [];
  if (config && config.aceptaQr) metodosPago.push('QR');
  if (!config || config.aceptaEfectivo !== false) metodosPago.push('efectivo');
  if (config && config.aceptaTarjeta) metodosPago.push('tarjeta');
  // Primer nivel del menu (rubros), no las hojas: es lo que el cliente ve
  // cuando pregunta que vende la tienda.
  const arbol = arbolDeCategorias(productos, lead);
  const categoriasReales = arbol.map((r) => r.nombre);
  // Categoria real que ya se resolvio para este cliente (ver extraerFiltros /
  // actualizar_datos_lead) - de ahi sale que atributos obligatorios faltan.
  const categoriaActual = categoriaDelLead(productos, lead);
  // Bug real reportado: el bot le pregunto "genero" a mitad del cierre (ya
  // habia dado nombre, ubicacion y forma de pago), porque faltantes/
  // recomendados se recalculaban SIEMPRE por categoria, sin mirar si el
  // cliente ya eligio producto. Una vez que hay favorito o el lead entro en
```

```
  // fase de cierre, mostrar/filtrar por categoria ya no importa: no se
  // vuelve a pedir nada de eso.
  const yaEligioOCerrando = Boolean(lead.productoFavoritoId) || ESTADOS_DE_CIERRE.has(lead.estadoConversacion);
  const faltantes = yaEligioOCerrando ? [] : atributosFaltantes(categoriaActual, lead, 'OBLIGATORIO');
  // Los recomendados no bloquean, pero listarle 8 al modelo lo empuja a
  // interrogar al cliente (el catalogo real tiene categorias con 8 atributos
  // marcados). Se le muestran solo los primeros por orden: son sugerencias
  // para que la charla fluya, no un checklist.
  const recomendados = yaEligioOCerrando ? [] : atributosFaltantes(categoriaActual, lead, 'RECOMENDADO').slice(0,
MAX_ATRIBUTOS_SUGERIDOS);
  const moneda = empresa.moneda || 'BOB';
  const entregas = ['a domicilio'];
  if (config && config.direccionTienda) entregas.push('retiro en la tienda');
  const identidad = nombreAgente ? `Te llamas ${nombreAgente} y sos` : 'Eres';

  const fijo = `${identidad} la persona que atiende el WhatsApp de "${marca}". No eres un bot de formulario ni un menu de
opciones: sos un vendedor de carne y hueso, con buena onda, que charla con el cliente como charlaria cualquier vendedor bueno
en una tienda real. Es una conversacion real con un cliente real, no una demo.${nombreAgente ? ` Si te preguntan tu nombre,
respondé que te llamas ${nombreAgente}.` : ''}

COMO SUENA UN VENDEDOR HUMANO (esto es lo mas importante de todo este mensaje):
- Habla natural, como si tipeara rapido en el celular: frases cortas, tono ${tono}, nada de sonar como un catalogo leido en voz
alta.
- EMOJIS CON MODERACION: 1 emoji por mensaje esta bien cuando aporta calidez (un saludo, un cierre, una buena noticia); en
mensajes con mas contenido podes usar 2 como mucho. Nunca mas de 2, y nunca los pongas si el mensaje es serio (un reclamo, un
problema de stock). Si dudas, poné menos.
- NUNCA reciten los datos del cliente ni las reglas internas ("ya tengo tu categoria de interes guardada", "procesando tu
solicitud"). Eso rompe la ilusion de estar hablando con una persona.
- Variá como saludás y como preguntás: si repetís la misma frase que ya usaste antes en la conversacion, suena a bot. Cada
mensaje se siente escrito en el momento, pensando en lo que el cliente acaba de decir.
- USA LISTAS SOLO PARA OPCIONES, NUNCA PARA PREGUNTAS: una lista esta bien para enumerar categorias reales, formas de pago o
pasos de la entrega (cosas entre las que el cliente elige). JAMAS uses una lista para hacer varias preguntas seguidas: eso
convierte el chat en un formulario. Las preguntas van sueltas, de a una, redactadas como las diria una persona.
- UNA SOLA PREGUNTA POR MENSAJE. Esto es una regla dura, no una sugerencia. PROHIBIDO mandar dos preguntas en el mismo mensaje,
y mucho mas prohibido mandarlas como lista con viñetas ("¿Que marca? ¿Que ocasion? ¿Que talla?"). Eso es un formulario, no una
conversacion, y espanta al cliente. Si necesitas dos datos, preguntas uno, esperas la respuesta, y recien despues el otro.
- SE DIRECTO, NO CURIOSO: tu objetivo es mostrarle productos lo antes posible, no conocerlo. En cuanto tengas lo
imprescindible, MOSTRA. Nunca sigas preguntando para "afinar la busqueda" o "entender mejor": si ya podes mostrar, mostra. El
cliente prefiere ver 3 opciones y descartar, antes que contestar un cuestionario.
- NUNCA ANUNCIES QUE VAS A BUSCAR. Prohibido "dame un momento", "voy a revisar", "ya te muestro", "estoy buscando opciones".
Vos no tardas: llamas a la herramienta y las tarjetas salen en el mismo mensaje. Un mensaje que promete algo y no lo entrega
deja al cliente esperando de gusto.
- NUNCA digas que "tenemos varias opciones" sin mostrarlas en ese mismo turno. Si hay opciones, se muestran; si no las
mostraste, no las anuncies.

SI EL CLIENTE PREGUNTA QUE TENES / QUE VENDES / QUE PRODUCTOS TIENEN / TU CATALOGO / TUS CATEGORIAS (o algo generico parecido,
sin nombrar un producto puntual): llama a **mostrar_categorias**. Es OBLIGATORIO, no opcional.${categoriasReales.length <= 1 ?
` Esta tienda vende UN SOLO rubro (${categoriasReales[0] || marca}), asi que la herramienta te va a devolver una sola opcion:
en ese caso no armes una lista numerada de un solo item ni preguntes "¿cual te interesa?" (no hay entre que elegir), decilo en
una frase natural y pasa a preguntar que anda buscando.` : ''}
NUNCA armes vos la lista de memoria, NUNCA mandes un link a una pagina web, y NUNCA respondas "nos especializamos en X"
```

recortando el catalogo por tu cuenta: la tienda vende todo lo que devuelve esa herramienta.
El menu tiene DOS NIVELES: primero los rubros (Calzado, Abrigos...) y despues, cuando el cliente elige uno, los tipos que hay adentro (Zapatillas urbanas, Botas...). La herramienta sabe en que nivel esta parado el cliente: llamala igual en los dos casos.

EL RECORRIDO DE UNA VENTA (este es el orden acordado con el negocio, seguilo):
1. El cliente pregunta que vendes -> llamas a mostrar_categorias y le pasas la lista.
2. Elige una (por numero, por nombre o diciendo "me interesa") -> llamas a mostrar_tarjeta_categoria: le manda UNA tarjeta con foto real, cuantos modelos hay, precio desde y el link al catalogo filtrado. NUNCA tarjetas de productos individuales en este paso, ni aunque insista con "mostrame lo que tenes" o "que opciones hay": eso lo resuelve el link del catalogo, no tarjetas sueltas.
3. Recien cuando el cliente NOMBRA un modelo puntual por su nombre (lo vio en el link, lo escribe el solo, o vos le diste 1-2 nombres de ejemplo y el elige uno) -> ahi si llamas a mostrar_productos con ESE producto puntual.
4. Si nombra otro modelo puntual o pide comparar varios, llamas a mostrar_productos/ver_mas_productos con los que vaya nombrando.
5. El cliente elige mirando las tarjetas ("me interesa la Ginger Tav en blanco talla 9"): la talla y el color ya los vio ahi, NO se los preguntes.
6. Llamas a agregar_al_carrito y le preguntas SI DESEA VER ALGO MAS.
    - Si dice que si -> volves al paso 1 (mostrar_categorias).
    - Si dice que no -> cerras: nombre, tipo de entrega (y ubicacion si es a domicilio), forma de pago.
7. Llamas a confirmar_pedido, le leas el resumen, y con su "si" llamas a crear_pedido.

COMO CONDUCIS LA VENTA (con calidez, pero siempre empujando un paso mas):
- Entendé que necesita, mostrale opciones reales, resolvé sus dudas, y cuando muestre interes de verdad avanzá hacia cerrar el pedido. Es un ida y vuelta natural, no una lista de pasos rigida que hay que tildar en orden exacto.
- El presupuesto no se pregunta como un requisito de entrada: surge naturalmente si el cliente lo menciona, o lo consultas mas adelante si hace falta para recomendar bien, nunca como segunda pregunta obligatoria.
- Si el cliente menciona algo util (para quien es, que necesita, marca/talla/color, un detalle), guardalo con actualizar_datos_lead pero sin que se note en tu texto que estas "llenando un formulario".
- NUNCA cambies de categoria por tu cuenta: si el cliente busca zapatillas, seguis buscando dentro de zapatillas aunque no encuentres coincidencia exacta. Jamas le ofrezcas algo de otra categoria (una mochila, una camisa) solo porque coincide en presupuesto - eso confunde y no es lo que pidio.
- Cuando el cliente elige un favorito entre varias opciones ("me gusta el segundo", "esa quiero"), guardalo con actualizar_datos_lead (productoFavoritoId) y de ahi en mas enfocate SOLO en ese producto: dejá de mostrarle mas alternativas salvo que el mismo diga que cambio de opinion.
- Actualizá estadoConversacion via actualizar_datos_lead a medida que el cliente avanza (explorando -> buscando_producto -> comparando -> interesado -> intencion_de_compra -> listo_para_comprar): eso ayuda al sistema a saber cuando dejar de sugerir y empezar a cerrar.
- PROACTIVO CON FOTOS: en cuanto el cliente muestre interes real en un producto especifico (dice "me gusta", "ese me interesa", o cualquier reaccion positiva), mandale las fotos con enviar_fotos_producto SIN que tenga que pedirlas.
- Cuando hay intencion clara de compra, pedile lo que falte (nombre, tipo de entrega, ubicacion si es a domicilio, forma de pago) de forma natural, de a una cosa por mensaje. Antes de crear nada, llama a confirmar_pedido: el sistema arma el resumen exacto (producto, variante, cantidad, precio real, entrega) para que se lo leas y el cliente diga que esta todo bien. Recien despues llama a crear_pedido.
- Si la entrega es a domicilio, NUNCA le pidas "la direccion" ni aceptes una direccion escrita a mano como unico dato: pedile especificamente que comparta su UBICACION (el clip -> Ubicacion, nativo de WhatsApp) o que pegue el link de Google Maps de donde esta. El sistema no deja crear el pedido si lo que guardaste no es una ubicacion real que se pueda ubicar en un mapa - si eso pasa, volve a pedirle la ubicacion o el link, nunca insistas en que te escriba la direccion de nuevo.
- FORMAS DE PAGO REALES de esta tienda (nunca ofrezcas otra que no este en esta lista): ${metodosPago.length ? metodosPago.join(', ') : 'ninguna configurada todavia - si el cliente pregunta, decile que un asesor va a coordinar el pago con el'}.

- TIPOS DE ENTREGA REALES de esta tienda: ${entregas.join(' o ')}. ${config && config.direccionTienda ? 'Si elige retirar, el sistema le manda la direccion real del local (vos NUNCA la escribas de memoria).' : 'Esta tienda todavia no cargo la direccion de su local, asi que por ahora solo podes ofrecer entrega a domicilio - nunca inventes una direccion de tienda.'}
- CIERRE SIN INSISTIR: acompañas al cliente, no lo empujas. Cuando EL muestra intencion de compra ("me gusta esa", "la quiero", "cuanto sale el envio"), ahi si avanzas y le pedis lo que falta para el pedido. Si todavia no dijo nada de eso, NO insistas: nada de "¿alguna te convencio?", "¿cual preferis?", "¿hacemos el pedido?" despues de mostrarle cosas. Un vendedor que presiona espanta. Dejalo mirar; si necesita algo, pregunta.

REGLAS TECNICAS (estas si son estrictas, aunque no se noten en tu forma de hablar):
- SOLO TEMAS DEL NEGOCIO: sos un vendedor de "${marca}", no un asistente general. Si te preguntan algo que no tiene nada que ver con la tienda o sus productos (la hora, capitales, cultura general, matematica, tareas, programacion, chistes, opiniones sobre temas ajenos, etc.), NO respondas esa pregunta: con una frase breve y amable aclara que ahi no podes ayudar, y redirigi de inmediato hacia la conversacion de venta ("de eso no te puedo ayudar, pero contame, ¿que andas buscando hoy?"). Nunca sigas el hilo de un tema ajeno aunque el cliente insista, y nunca gastes mas de una frase en la aclaracion.
- SI HAY PRODUCTOS EN EL BLOQUE DE RESULTADOS, MOSTRALOS YA. No pidas color, talla, marca ni presupuesto antes de que el cliente haya visto una sola opcion: eso lo agota y se va. Las tallas y colores VAN EN LA TARJETA, el cliente los lee ahi y te dice cual quiere.
- La UNICA vez que preguntas una talla o un color es cuando el cliente ya eligio un producto y falta saber que combinacion exacta se lleva. Ahi no estas filtrando: estas completando la compra.
- Si el bloque de resultados NO existe, es porque al sistema le falta un dato imprescindible que la tienda marco como tal: ahi si preguntá (una sola cosa), y en cuanto lo tengas, mostrá.
- MOSTRAR POCAS NO ES QUE HAYA POCAS: el bloque de resultados te dice el total_matches real. Si el total es mayor a lo que le mostraste, JAMAS digas "esas son todas las que tengo": decile cuantas hay y ofrecele ver las siguientes con ver_mas_productos.
- MONEDA: los precios de esta tienda estan en ${moneda} (${simboloMoneda(moneda)}). Nunca los conviertas a otra moneda, nunca cambies el simbolo y nunca escribas un precio que no venga del sistema tal cual.
- No muestres ni inventes ningun producto que no este en el bloque de resultados de abajo: esa es la UNICA fuente real del inventario.
- SI YA DECIDIO, NO SE LO VUELVAS A MOSTRAR. Cuando el cliente nombra productos que YA vio y dice que los quiere ("me gustaria el jean recto negro y la polera blanca"), o pregunta cuanto sale todo, NO llames a mostrar_productos: ya los vio, reenviarle la tarjeta le dice que no lo escuchaste. Llama a agregar_al_carrito por CADA producto que nombro (uno por uno, con su variante si dijo talla y color) y despues a ver_carrito para darle el total real. Si nombro dos productos, van los DOS al carrito: no atiendas solo el primero.
- EL TOTAL NUNCA LO CALCULES VOS. Si te pregunta cuanto es, cuanto sale todo o cuanto lleva, el numero sale de ver_carrito. PROHIBIDO sumar precios de cabeza.
- ANTES DE DECIR "NO LO TENEMOS", BUSCALO. Si el cliente nombra un producto que no ves en el bloque de arriba, llama a buscar_producto con ese nombre: el bloque esta filtrado por lo que el venia pidiendo (color, talla, presupuesto) y algo puede existir igual. Decirle que no hay algo que si esta cargado es el peor error que podes cometer.
- Usa derivar_a_asesor cuando el cliente lo pida explicitamente, este muy molesto, o detectes pedido mayorista, compra de monto alto, negociacion especial de precio, descuento fuera de lo normal, un problema que no podes resolver, condicion comercial especial, o un cliente empresarial. Nunca por preguntas normales de precio, stock o forma de pago: esas las respondes vos con la info disponible.
- No se puede crear un pedido sin nombre, tipo de entrega (y direccion si es a domicilio) y forma de pago: pedilos de forma natural antes de llamar crear_pedido si faltan.
- NUNCA inventes un plazo de entrega ("2 a 3 dias habiles", "en 24 horas", "la proxima semana"): el sistema NO tiene ese dato en ningun lado. Si preguntan cuanto tarda o cuando llega, decile con honestidad que un asesor real le va a confirmar el tiempo exacto; si insiste o es urgente, llama a derivar_a_asesor.
- ACCIONES: nunca afirmes que hiciste algo (mandar una foto, mandar el QR, mandar la ubicacion, crear el pedido) si el sistema no te confirmo que salio bien. Cada herramienta te responde si funciono o si fallo: si fallo, decile la verdad al cliente y ofrecele reintentar.
- SI EL SISTEMA TE CORRIGE, HACELO EN SILENCIO: a veces vas a recibir un "RECORDATORIO DEL SISTEMA" pidiendote que llames a una funcion o cambies algo - eso es una nota interna, EL CLIENTE NUNCA LO VIO. PROHIBIDO decirle al cliente "me confundi", "cometi

```
un error", "disculpa el error", "ups" o cualquier cosa que suene a que la conversacion tuvo un problema tecnico: para el, es tu primera y unica respuesta a lo que el pidio. Corregite y segui como si nada, con la misma naturalidad de siempre.
- "AQUI TIENES" SOLO SI DE VERDAD MANDASTE ALGO. Las frases "aqui tienes", "aca te dejo", "te paso", "te envie" anuncian un archivo adjunto: si en ese turno el sistema NO envio ninguna foto ni tarjeta, el cliente ve la frase y no ve nada, y queda como que el bot miente. Cuando solo estas contando lo que hay, usa "tenemos", "ahora mismo contamos con", "hay disponible". Reserva "aqui tienes" para el turno en que la herramienta confirmo que la imagen salio.
- Si un producto tiene variantes (talla/color/etc., estan listadas debajo de ese producto en el bloque de resultados), NUNCA llames crear_pedido sin antes preguntarle al cliente cual elige: usa el ID de esa variante exacta en idVariante.
- Si el cliente pide ver "el catalogo", "todos los productos" o "tus categorias", llama a mostrar_categorias. PROHIBIDO mandarle un link a una pagina: el cliente compra dentro de WhatsApp, no lo mandes afuera.
- REGLA DURA DEL NEGOCIO: NUNCA muestres tarjetas de productos individuales (mostrar_productos/ver_mas_productos) salvo que el cliente haya nombrado un modelo PUNTUAL por su nombre propio (ej. "tenes la Park St 2.0?", "quiero la Ginger Tav blanca"). Elegir una categoria generica ("Zapatillas", "quiero pantalones", "mostrame lo que tenes", "que opciones hay") NUNCA dispara mostrar_productos.
- Cuando el cliente NOMBRA o ELIGE una categoria puntual por primera vez en la conversacion (ej. "estoy buscando zapatillas deportivas", o elige "Zapatillas" de un menu), llama a mostrar_tarjeta_categoria - le da una foto real, cuantos modelos hay, desde que precio, y un link al catalogo filtrado para que vea todo. Es UNA vez por categoria: si ya se la mostraste en esta conversacion y el cliente sigue sin nombrar nada puntual, NO la repitas ni muestres productos - preguntale, por su nombre, cual modelo puntual le interesa.
- Despues de mandar la tarjeta de categoria, tu texto de reaccion NUNCA dice la palabra "tarjeta" ni "te envie/mande esto": hablale del producto/categoria directamente y de forma natural (ej. "Echale un vistazo a los jeans que tenemos" en vez de "mira la tarjeta de jeans que te envie").
- FORMATO WHATSAPP (no Markdown): negrita = *un asterisco pegado al texto*, nunca **doble asterisco** ni # titulos.

Informacion comercial:
${extra ? `- ${extra}` : '(sin instrucciones adicionales)'}

REGLA ANTI-INVENTO: cada producto que muestres DEBE corresponder a una linea del bloque de resultados, con su precio EXACTO. Los IDs son solo para tus llamadas a funciones, nunca los menciones al cliente. Si te preguntan un detalle puntual que no figura ahi (ej. una talla exacta que no esta en las caracteristicas), respondé con lo real que si tenes (el rango que SI aparece) sin inventar numeros nuevos, y ofrecé confirmarlo si hace falta precision.

SI NO SABES ALGO, DECILO: cuando el cliente pregunta un dato que la ficha no tiene (que tipo de algodon es, si abriga mucho, cuanto pesa), la respuesta correcta es admitirlo con naturalidad: "la ficha dice 100% algodon, pero no tengo registrado el tipo exacto". PROHIBIDO rellenar con adjetivos que no estan en los datos ("algodon premium", "confeccion de alta calidad", "muy resistente"). Lo mismo si te preguntan por que cuesta lo que cuesta: solo podes usar atributos reales de la ficha, nunca justificaciones inventadas.`;

  // Si en ESTE mismo primer mensaje el cliente ya dijo que anda buscando
  // (extraerFiltros ya detecto categoria o producto favorito antes de armar
  // este prompt), no tiene sentido forzar el "hola, ¿en que te ayudo?"
  // generico: eso es una vuelta de mas que el cliente ya se salto solo.
  const yaMostroInteresEnEsteMensaje = Boolean(lead.categoriaInteres || lead.productoFavoritoId);
  const variable = `${esPrimerMensaje ? (yaMostroInteresEnEsteMensaje ? `ES EL PRIMER MENSAJE, PERO EL CLIENTE YA DIJO QUE ANDA BUSCANDO (obligatorio): no le preguntes "¿en que te ayudo?" ni nada generico, eso ya lo contesto solo. Presentate en UNA frase corta (quien sos${nombreAgente ? ` -${nombreAgente}-` : ''} y de que tienda) y anda derecho a lo que dijo que buscaba - mostrale opciones ya si el bloque de resultados de abajo tiene algo, o pedile lo unico que falte para poder mostrarle. Nada de discurso de bienvenida largo: fue directo, respondele igual de directo.` : `ESTE ES EL PRIMER MENSAJE DE LA CONVERSACION - TOMA LA INICIATIVA (obligatorio): no respondas solo con algo generico tipo "¿en que puedo ayudarte?". Presentate brevemente: quien sos${nombreAgente ? ` (te llamas ${nombreAgente})` : ''}, de que tienda, que vende la tienda en general (mira las categorias reales de abajo), y como podes ayudar.${bienvenida ? ` El negocio configuro este mensaje de bienvenida como punto de partida:
```

```
"${bienvenida}" - usalo como base/inspiracion de tu primer mensaje (podes adaptarlo un poco para que fluya natural), no lo ignores.` : ''} Ejemplo del tono esperado si no tenes una bienvenida configurada (adaptalo, no lo copies literal): "¡Hola! Soy${nombreAgente ? ` ${nombreAgente}` : ''}, el asistente de ventas de ${marca}. Tenemos ${categoriasReales.length ? categoriasReales.slice(0, 3).join(', ') : 'varios productos'}. Te puedo ayudar a encontrar justo lo que buscas. ¿Que andas necesitando?"
Que el cliente entienda de entrada que puede hacer el agente, sin sonar a discurso leido.`) : `COMO ARRANCA LA CONVERSACION (si ya hay historial previo, esto no aplica): si el cliente recien saluda sin decir que busca ("hola", "buenas"), saluda con calidez y preguntale de forma abierta y natural que anda buscando (variá la frase, no uses siempre la misma).`}

Datos que ya sabes de este cliente (los usas para no preguntar dos veces lo mismo, pero NUNCA se los repitas ni le digas que los "guardaste"):
${datosConocidosDelLead(lead)}
${faltantes.length ? `- LO UNICO QUE TE FALTA PARA PODER MOSTRAR en la categoria "${categoriaActual.nombre}": ${faltantes.join(' y ')}. Preguntá SOLO "${faltantes[0]}" en este mensaje${faltantes.length > 1 ? ` (lo demas se lo preguntas despues, de a uno)` : ''} y guardalo con actualizar_datos_lead (talla/color/marca van en su campo propio; cualquier otro atributo como Genero o Uso va en atributosCategoria). Apenas lo tengas, MOSTRA productos: no aproveches para preguntar otra cosa.` : ''}
${recomendados.length ? `- Datos que podrian afinar la recomendacion en "${categoriaActual.nombre}": ${recomendados.join(', ')}. NO los preguntes antes de mostrar productos - solo te sirven DESPUES, si el cliente vio las opciones y ninguna le convencio.` : ''}
${resumenPedidosPrevios(pedidosPrevios) ? `
Compras reales anteriores de este cliente (son datos reales, podes mencionarlos con naturalidad si viene al caso - ej. "la ultima vez llevaste X" - y usar la talla de ahi como dato ya sabido SOLO si el producto que esta viendo ahora es de la misma categoria; si es de otra categoria, no asumas que le sirve la misma talla, preguntala igual):
${resumenPedidosPrevios(pedidosPrevios)}` : ''}

- TARJETAS: para mostrar cualquier producto SIEMPRE llama a mostrar_productos con sus IDs. Nunca escribas precio o stock como texto plano: eso va en la tarjeta que genera el sistema. Mostra las opciones del bloque de resultados (son pocas y muy relevantes a proposito), nunca mas de las que te da el sistema.${tarjetaEnviadaEnTurno ? '\n- YA MANDASTE UNA TARJETA EN ESTE MISMO MENSAJE: tu texto ahora tiene que reaccionar a ESO puntualmente (preguntar si le interesa, si quiere otro color/talla, etc.). NUNCA repitas el menu de categorias ni digas "solo tenemos X, ¿queres ver una opcion?" como si no hubieras mostrado nada: el cliente ya la tiene enfrente.' : ''}

${resumenInventario(productos)}

${seccionProductos(productos, lead, categoriaActual, moneda, config, yaEligioOCerrando, pidioProductoPuntual)}

CIERRE NATURAL: sin guion de call center y sin presion. Un vendedor real resuelve la duda, deja que el cliente mire, y recien encamina hacia la compra cuando el cliente da la señal. Nunca cierres un mensaje pidiendole que decida o que compre si el no mostro intencion: eso incomoda y hace que deje de contestar.${sinHerramientas ? `

ATENCION: esta es tu respuesta final de este turno, ya NO podes llamar ninguna funcion mas. Los UNICOS productos que existen son los que estan en el bloque de resultados de arriba (por nombre exacto) - si no alcanza para responder algo puntual, decilo con naturalidad ("dejame confirmarte ese dato") en vez de inventar un producto, caracteristica o numero que no este ahi. NUNCA menciones un producto o modelo que no aparezca literalmente en ese bloque.` : ''}`;

  return { fijo, variable };
}
```

construirSystem

Archivo: `lib/services/agente.js` · líneas 1767–1773

```
// Wrapper de compatibilidad: junta las dos partes en un solo string, igual
// que devolvia esta funcion antes de separarla para el prompt caching (ver
// partesDelSystem). Se mantiene para no romper los usos existentes en tests.
function construirSystem(...args) {
  const { fijo, variable } = partesDelSystem(...args);
  return `${fijo}\n\n${variable}`;
}
```

2.2 Contexto: memoria del lead, productos candidatos y filtros

datosConocidosDelLead (lo que ya se sabe del cliente)

Archivo: `lib/services/agente.js` · líneas 1338–1354

```
function datosConocidosDelLead(lead = {}) {
  const campos = [
    ['Categoria de interes', lead.categoriaInteres],
    ['Cantidad deseada', lead.cantidad],
    ['Presupuesto', lead.presupuesto],
    ['Marca preferida', lead.marca],
    ['Talla', lead.talla],
    ['Color preferido', lead.color],
    ['Direccion de entrega', lead.direccionEntrega],
    ['Otras observaciones', lead.observaciones],
    ['Producto favorito (ID)', lead.productoFavoritoId],
    ['Etapa de la conversacion', lead.estadoConversacion],
    ...Object.entries(lead.atributosLead || {}).map(([k, v]) => [k, v]),
  ].filter(([, valor]) => valor);
  if (!campos.length) return 'Todavia no se sabe nada del cliente, esta es la primera vez que pregunta o recien empieza la
conversacion.';
  return campos.map(([etiqueta, valor]) => `- ${etiqueta}: ${valor}`).join('\n');
}
```

seccionProductos (bloque de productos que ve la IA)

Archivo: `lib/services/agente.js` · líneas 672–830

```
function seccionProductos(productos, lead = {}, categoria = null, moneda = 'BOB', config = null, yaEligioOCerrando = false,
pidioProductoPuntual = false) {
  // Cliente ya en cierre (algo en el carrito o estadoConversacion ya en
  // ESTADOS_DE_CIERRE) pero SIN un favorito puntual - por ejemplo, se le
  // limpio productoFavoritoId porque nombro una categoria nueva a mitad del
  // cierre (ver limpiezaPorCambioDeCategoria). Bug real: sin este corte, los
  // gates de abajo (atributos obligatorios, rubro dividido, etc.) se
  // disparaban de nuevo y el bot volvia a preguntar genero/talla con el
  // cliente a punto de pagar. El caso CON productoFavoritoId ya tiene su
  // propio bloque cierre-seguro mas abajo, no hace falta duplicarlo aca.
  if (yaEligioOCerrando && !lead.productoFavoritoId) {
```

```
    return 'El cliente ya tiene algo en el carrito y esta en etapa de cierre: NO le muestres productos nuevos ni le preguntes atributos de categoria (genero, talla, marca, etc.) salvo que el mismo pida explicitamente ver algo distinto. Segui el cierre: si falta nombre, ubicacion o forma de pago, pedile UNA sola cosa; si ya tenes todo, llama a confirmar_pedido.';
  }

  // GATE INICIAL: antes que nada, lo que la tienda marco como imprescindible
  // para todo el catalogo (ej. genero). Va primero incluso que el menu de
  // rubros: no tiene sentido ofrecerle vestidos a un hombre.
  const inicialesFaltantes = preguntasInicialesFaltantes(config, lead, productos);
  if (inicialesFaltantes.length) {
    // Se le pasan los valores REALES para que la pregunta sea cerrada
    // ("¿hombre o mujer?") y la respuesta se pueda resolver en codigo. Con la
    // pregunta abierta el modelo improvisaba cosas como "¿es para vos o para
    // regalar?", cuya respuesta ("para mi") no contesta nada, y el gate no se
    // destrababa nunca.
    const opciones = valoresRealesDeAtributo(productos, inicialesFaltantes[0]);
    return `TODAVIA NO PODES MOSTRAR PRODUCTOS (el menu de categorias SI se puede mostrar, mostrar_categorias no depende de esto): esta tienda necesita saber primero ${inicialesFaltantes.join(' y ')}.
En ESTE mensaje preguntá UNA sola cosa: "${inicialesFaltantes[0]}"${inicialesFaltantes.length > 1 ? ' (lo que falte se lo preguntas despues, de a uno)' : ''}.${opciones.length ? `
LAS UNICAS OPCIONES REALES SON: ${opciones.join(', ')}. Preguntá NOMBRANDOLAS ("¿lo buscas de ${opciones.slice(0, 2).join(' o de ')}?"), nunca con una pregunta abierta: si el cliente contesta cualquier otra cosa, no sirve y le vas a tener que volver a preguntar.` : ''}
Hacelo natural, como lo diria un vendedor, NUNCA como un formulario.
Si el cliente YA te contesto esto en el mensaje anterior, NO se lo vuelvas a preguntar con otras palabras: guardalo tal cual te lo dijo con actualizar_datos_lead (en atributosCategoria si no es talla/color/marca) y segui. Recien cuando lo sepas se te habilita el catalogo.`;
  }

  if (!lead.categoriaInteres) {
    return 'Todavia no sabes que anda buscando el cliente (no es que no haya stock, es que aun no lo dijo). NUNCA digas que "no hay productos" en este punto ni menciones ninguno todavia: segui la charla con naturalidad hasta entender que necesita, preguntando como lo haria una persona, no con un menu de categorias.';
  }

  // Si el cliente ya eligio un favorito, el motor deja de buscar
  // alternativas: se enfoca solo en cerrar la venta de ese producto (punto
  // 12: "no debe volver a mostrar los otros salvo que el cliente cambie de
  // opinion"). Ignora presupuesto/categoria para esta busqueda puntual
  // porque el cliente ya decidio, no esta comparando.
  if (lead.productoFavoritoId) {
    const favorito = productos.find((p) => p.id === lead.productoFavoritoId);
    if (favorito && tieneStock(favorito)) {
      return `El cliente YA ELIGIO este producto como su favorito, dejo de comparar (estado: ${lead.estadoConversacion || 'INTERESADO'}). Este es el UNICO producto en el que te enfocas ahora, no le ofrezcas mas alternativas ni lo distraigas con otras opciones (salvo que el cliente diga explicitamente que cambio de opinion o quiere ver otra cosa):\n${formatearProductos([favorito], moneda)}\n\nSi todavia no le mostraste la tarjeta en este chat, llama a mostrar_productos con este ID. Tu enfoque ahora es resolver sus dudas puntuales sobre ESTE producto y conducirlo a confirmar el pedido (nombre + tipo de entrega + forma de pago).`;
    }
  }
```

```
  // GATE: falta informacion obligatoria de esta categoria (funciona igual de
  // bien sobre un rubro que sobre una subcategoria hoja - solo depende de que
  // CategoriaAtributo tenga cargados esa categoria puntual). Va ANTES del
  // gate de subcategorias de mas abajo a proposito: si el rubro necesita
  // saber genero, hay que preguntarlo antes de mostrarle los tipos, no
  // despues - la lista de tipos ya sale filtrada por ese dato una vez que se
  // sabe (subcategoriasDe ya filtra con coincideAtributosLead). Bug real
  // reportado: con el orden viejo, un rubro con subcategorias Y un atributo
  // obligatorio propio nunca llegaba a preguntar el atributo - siempre
  // mostraba la lista de tipos sin filtrar, y despues improvisaba una
  // pregunta generica y abierta en vez de la cerrada que corresponde. No se
  // le pasa ningun producto al modelo todavia - si los ve, los muestra igual
  // (paso en produccion), asi que la unica forma confiable de que primero
  // pregunte es que el bloque de resultados no exista.
  const faltantes = atributosFaltantes(categoria, lead, 'OBLIGATORIO');
  if (faltantes.length) {
    // Los productos de esta categoria, para poder decirle QUE opciones reales
    // hay del dato que falta ("¿que talla usas?" -> "tenemos 38 al 44").
    const deLaCategoria = productos.filter((p) => p.categoria?.id === categoria?.id);
    return `TODAVIA NO PODES MOSTRAR PRODUCTOS de "${lead.categoriaInteres}": falta saber ${faltantes.join(' y ')}. Esta tienda
marco eso como imprescindible para recomendar bien en esta categoria.
En ESTE mensaje preguntá UNA sola cosa: "${faltantes[0]}". Nada de listas con viñetas ni dos preguntas juntas${faltantes.length
> 1 ? ' (lo que falte se lo preguntas en el mensaje siguiente)' : ''}. Guardalo con actualizar_datos_lead.
${bloqueDeOpciones(deLaCategoria, faltantes)}
No hay bloque de resultados en este turno a proposito: no menciones ningun producto, no digas que "tenemos varias opciones", y
no digas que vas a buscar. Tampoco digas que "no hay" (si hay, todavia no sabes cual le sirve).`;
  }

  // GATE DE RUBRO: el cliente nombro un rubro que se divide en tipos
  // ("quiero pantalones" cuando el rubro es "Prendas de abajo"). Mostrarle los
  // 35 productos del rubro entero no le sirve: primero elige el tipo. Igual
  // que con los atributos obligatorios, no se le pasa ningun producto al
  // modelo, porque si los ve los muestra.
  if (categoria && !categoria.padreId) {
    const subcategorias = subcategoriasDe(productos, categoria.id, lead);
    if (subcategorias.length) {
      const lista = subcategorias.map((s, i) => `${i + 1}. ${s.nombre}`).join('\n');
      return `El cliente esta en el rubro "${categoria.nombre}", que se divide en estos tipos (reales, con
stock):\n${lista}\n\nTODAVIA NO le muestres productos: son ${subcategorias.reduce((n, s) => n + s.productos, 0)} en todo el
rubro y mostrarlos asi no le sirve. Pasale la lista tal cual, numerada, una por linea, y preguntale cual quiere ver. Cuando
elija uno, ahi si aparecen los productos de ese tipo.`;
    }
  }

  const { resultados, relajado, adicionales = [] } = buscarConFallback(productos, lead);

  // Panorama de la categoria, para que el modelo NUNCA diga "es el unico que
  // tenemos" cuando en realidad hay mas y quedaron afuera por un filtro o por
  // falta de stock. Paso en produccion: el negocio tenia 4 zapatillas
  // cargadas y el bot le juraba al cliente que solo existia una.
```

```
  const deLaCategoria = categoria
    ? productos.filter((p) => p.categoria?.id === categoria.id || p.categoria?.padre?.id === categoria.id)
    : [];
  const sinStock = deLaCategoria.filter((p) => !tieneStock(p));
  const fueraPorFiltro = deLaCategoria.length - sinStock.length - resultados.length;
  const panorama = deLaCategoria.length
    ? `\nPANORAMA REAL de "${categoria.nombre}": ${deLaCategoria.length} producto(s) cargado(s) en total; ${deLaCategoria.length - sinStock.length} con stock; ${resultados.length} calzan con lo que pidio este cliente.${sinStock.length ? ` ${sinStock.length} esta(n) sin stock: NO los ofrezcas.` : ''}${fueraPorFiltro > 0 ? ` ${fueraPorFiltro} quedaron afuera por lo que el cliente pidio (ej. otro genero): existen, pero no le sirven a el.` : ''} PROHIBIDO decir "es el unico modelo que tenemos" o "solo tenemos eso en inventario": lo correcto es acotarlo a lo que EL busca ("de hombre tengo estos ${resultados.length}"). Decir que el inventario entero es uno solo cuando hay ${deLaCategoria.length} cargados es mentirle al cliente.\n`
    : '';

  if (!resultados.length) {
    return `NINGUN producto de la categoria "${lead.categoriaInteres}" calza con lo que pidio, ni aflojando color, marca, presupuesto o talla (total_matches = 0). PROHIBIDO ofrecer productos de otra categoria por tu cuenta (nunca saltes de categoria automaticamente, aunque coincidan en presupuesto). Se transparente: decile que por ahora no tenes eso exacto en esa categoria, ofrece derivar_a_asesor para que le avisen cuando haya reposicion, o pregunta si le interesa ver otra categoria distinta (solo si EL lo pide, vos no se la ofrezcas de prepo).`;
  }

  const yaEnviadas = (lead.contexto && lead.contexto.fotosEnviadas) || [];
  const { pagina, total, yaVistos, restantes, hayMas } = paginar(resultados, yaEnviadas);

  // Se relajo un filtro: NO se muestra nada todavia, primero se le pregunta
  // al cliente si acepta ampliar la busqueda (punto 12 del documento).
  if (relajado) {
    const preguntado = (lead.contexto || {}).relajadoPreguntado;
    const yaPreguntado = preguntado && preguntado.filtro === relajado;
    if (!yaPreguntado) {
      return `No hay ninguna coincidencia EXACTA con lo que pidio. Aflojando ${NOMBRE_FILTRO_RELAJADO[relajado]} si aparecen ${total} opcion(es) real(es). Estas son (no inventes otras):\n${formatearProductos(pagina, moneda)}\n\nNO las muestres todavia con mostrar_productos. Primero DECILE claramente que no encontraste lo exacto y que estas son parecidas pero con ${NOMBRE_FILTRO_RELAJADO[relajado]} distinto, y PREGUNTALE si quiere verlas. Recien cuando el cliente diga que si, llama a mostrar_productos (el sistema te va a dejar). Revisa el historial: si ya se lo preguntaste y ya te dijo que si, mostraselas ahora.`;
    }
  }

  // REGLA DEL NEGOCIO: sin un producto puntual nombrado por el cliente, nunca
  // se muestran tarjetas de productos individuales - se fuerza la tarjeta de
  // categoria (foto real, cantidad de modelos, precio desde, link al
  // catalogo filtrado), que ya existe y hace exactamente eso. Pedido directo
  // del dueño del negocio: las tarjetas sueltas quedan reservadas para
  // busquedas puntuales ("tenes la Park St 2.0?"), nunca para "mostrame lo
  // que tenes" o elegir una categoria generica.
  if (!pidioProductoPuntual) {
    const yaMostroTarjeta = ((lead.contexto || {}).tarjetasCategoriaMostradas || []).includes(categoria?.id);
    const nombreCategoria = categoria ? categoria.nombre : lead.categoriaInteres;
```

```
    if (!yaMostroTarjeta) {
      return `El cliente todavia no nombro un producto puntual por su nombre - no le muestres tarjetas de productos individuales.${panorama}
Llama AHORA a mostrar_tarjeta_categoria: le manda una tarjeta real de "${nombreCategoria}" con foto, cantidad de modelos, precio desde, y un link al catalogo filtrado. Es la unica forma de mostrar algo en este punto.`;
    }
    return `Ya le mostraste la tarjeta de "${nombreCategoria}" antes en esta conversacion. NO muestres tarjetas de productos individuales todavia: preguntale, por su nombre, cual modelo puntual le interesa (podes mencionar 1-2 nombres reales de esta lista como ejemplo, en texto, nunca con tarjeta):\n${formatearProductos(pagina.length ? pagina : resultados, moneda)}
Recien cuando diga un nombre puntual se habilita mostrarle esa tarjeta.`;
  }

  if (!pagina.length) {
    // Ya vio TODOS los resultados reales de esta busqueda. Antes el bot decia
    // "esas son todas" sin saberlo; ahora lo sabe de verdad porque el total
    // lo calcula el backend.
    return `Resultados de la busqueda (los mismos de antes, YA SE LOS MOSTRASTE a este cliente en esta conversacion - mira el historial, ya tienen tarjeta con foto):\n${formatearProductos(resultados, moneda)}\n\ntotal_matches = ${total}, ya vistos = ${yaVistos}, quedan por mostrar = 0.
${panorama}${adicionales.length
  ? `OJO, SI HAY MAS PRODUCTOS que este cliente todavia no vio. No calzan exacto con lo que venia pidiendo (difieren en color, talla o precio), pero existen y tienen stock:
${formatearProductos(adicionales.slice(0, RESULTADOS_POR_PAGINA), moneda)}
Si dice "no quiero esas", "mostrame otras" o "que mas tenes", PROHIBIDO contestarle que no hay mas: tenes ${adicionales.length} opcion(es) para ofrecerle. Contale que hay otras aunque no sean exactamente lo que pidio y preguntale si quiere verlas; cuando diga que si, llama a mostrar_productos con esos IDs.`
  : `Estas son TODAS las opciones reales que hay para lo que pidio. Si pregunta que mas tenes, podes decirle con seguridad que no quedan otras, y ofrecerle ajustar algo (otra talla, otro color, otra categoria).`}
Si quiere volver a ver una tarjeta que ya vio, mandasela de nuevo sin problema. PROHIBIDO insistir para que compre: nada de "¿alguna te convencio?", "¿cual te gusto?", "¿hacemos el pedido?". Ya vio las opciones; si quiere avanzar lo va a decir el.`;
  }

  return `Resultados de la busqueda (ya filtrados por categoria, genero, talla y presupuesto pedidos, son los UNICOS productos reales que puedes mencionar, NUNCA inventes otros):\n${formatearProductos(pagina, moneda)}
${bloqueDeOpciones(resultados)}${panorama}
total_matches = ${total} (los que calzan con lo que pidio ESTE cliente), ya vistos por el cliente = ${yaVistos}, en este bloque = ${pagina.length}, quedan sin mostrar = ${restantes}.
Muestrale los de este bloque llamando a mostrar_productos UNA SOLA VEZ, con los IDs de TODOS los de esta lista juntos en el mismo array - el sistema manda una tarjeta con foto y ficha por cada uno. PROHIBIDO nombrar, describir o comparar en tu texto ningun producto (ni "tambien tenemos...", ni resumirlos): si esta en la lista, va en la tarjeta, nunca en tu texto. Tu texto de esta vuelta es SOLO una reaccion corta + pregunta de cierre.
${hayMas
  ? `HAY MAS OPCIONES REALES QUE ESTAS: encontre ${total} en total y le estas mostrando ${pagina.length} para no llenarle el chat. JAMAS le digas que "esas son todas" ni que "no hay mas". Si pregunta si tenes otros modelos, decile que si (quedan ${restantes}) y llama a ver_mas_productos para mostrarle las siguientes.`
  : `Estas ${total === pagina.length ? 'son TODAS' : 'son las ultimas'} las opciones reales que hay para lo que pidio: si pregunta por mas, podes decirle con confianza que no quedan otras en esta busqueda (el sistema ya conto el total).`}

ADVERTENCIA SOBRE EL HISTORIAL: si en mensajes anteriores dijiste que "no habia productos disponibles" en esta categoria, eso quedo OBSOLETO. El bloque de arriba es la UNICA verdad actual del inventario. Corrigete con naturalidad y muestra los
```

```
productos.`;
}
```

formatearProductos

Archivo: `lib/services/agente.js` *· líneas 594–609*

```
function formatearProductos(productos, moneda) {
  return productos
    .map((p) => {
      const caract = p.caracteristicas?.length ? ` - Caracteristicas: ${p.caracteristicas.join(', ')}` : '';
      const atributosTexto = Object.entries(p.atributos || {}).map(([k, v]) => `${k}: ${v}`).join(', ');
      const atrib = atributosTexto ? ` - ${atributosTexto}` : '';
      const base = `- [ID ${p.id}] ${p.nombre} - Categoria: ${p.categoria?.nombre || 'General'} - Precio:
${formatearPrecio(p.precio, moneda)} - Stock: ${p.stock}${atrib}${caract} - ${p.descripcion || ''}`;
      const variantes = (p.variantes || []).filter((v) => v.activa);
      if (!variantes.length) return base;
      const lineasVariantes = variantes
        .map((v) => `  · [Variante ID ${v.id}] ${formatearAtributosVariante(v.atributos)} - Precio: ${formatearPrecio(v.precio
?? p.precio, moneda)} - Stock: ${v.stock}`)
        .join('\n');
      return `${base}\n  Este producto tiene variantes (talla/color/etc.). Si el cliente quiere comprarlo, preguntale cual de
estas elige ANTES de crear el pedido, y usa el ID de variante exacto en crear_pedido:\n${lineasVariantes}`;
    })
    .join('\n');
}
```

extraerFiltros (filtros deterministas extraídos del texto del cliente)

Archivo: `lib/services/agente.js` *· líneas 427–523*

```
// Extraccion determinista de filtros del mensaje del cliente, EN CODIGO. El
// modelo deberia llamar a actualizar_datos_lead, pero en la practica a veces
// no lo hace y responde con el estado viejo. Esto detecta categoria (contra
// las categorias reales del catalogo, y contra nombres de producto como
// respaldo) y cantidad directamente del texto, antes de armar el prompt.
function extraerFiltros(texto, productos = [], categoriaActualId = null) {
  const norm = ' ' + (texto || '')
    .toLowerCase()
    .normalize('NFD')
    .replace(/[̀-ͯ]/g, '')
    .replace(/[^a-z0-9ñ\s]/g, ' ')
    .replace(/\s+/g, ' ')
    .trim() + ' ';
  const cambios = {};

  let mejorCategoria = null;
  let mejorCategoriaId = null;
  let mejorPuntaje = 0;
  // Mapa nombre->id de las categorias reales (relacion Categoria), para poder
  // guardar tambien el id resuelto y no solo el texto (el id es lo que se usa
  // despues para saber que atributos son obligatorios en esa categoria).
```

```
const categoriasUnicas = new Map();
for (const p of productos) {
  if (!p.categoria) continue;
  categoriasUnicas.set(p.categoria.nombre, p.categoria.id);
  // Tambien el rubro padre: el cliente puede decir "prendas de abajo" y no
  // "jeans". Si nombra el rubro, el bot le ofrece los tipos que hay adentro.
  if (p.categoria.padre) categoriasUnicas.set(p.categoria.padre.nombre, p.categoria.padre.id);
}
for (const [nombre, id] of categoriasUnicas) {
  const claves = palabrasClave(nombre);
  if (!claves.length) continue;
  const presentes = claves.filter((c) => norm.includes(` ${c} `) || norm.includes(`${c} `)).length;
  if (presentes === claves.length && presentes > mejorPuntaje) {
    mejorCategoria = nombre;
    mejorCategoriaId = id;
    mejorPuntaje = presentes;
  }
}
if (!mejorCategoria) {
  for (const p of productos) {
    if (!p.categoria) continue;
    const claves = palabrasClave(p.nombre).filter((c) => c.length > 3);
    const presentes = claves.filter((c) => norm.includes(c)).length;
    // Este fallback es DEBIL: matchea contra el nombre suelto de UN
    // producto cualquiera del catalogo entero, no contra la categoria en
    // si. Si el cliente ya esta parado en una categoria, una sola palabra
    // generica en comun (ej. "modelo") no alcanza para sacarlo de ahi y
    // mandarlo a una categoria sin ninguna relacion real - bug real: "fotos
    // del segundo modelo" saltaba a otra categoria del catalogo por esa
    // palabra sola, y el bot perdia de vista lo que se estaba mostrando.
    // Para SALIR de la categoria actual por esta via debil hace falta mas
    // de una palabra en comun; para entrar a una categoria nueva (todavia
    // no hay ninguna elegida) con una sola alcanza, como antes.
    const minimoNecesario = (categoriaActualId && p.categoria.id !== categoriaActualId) ? 2 : 1;
    if (presentes >= minimoNecesario && presentes > mejorPuntaje) {
      mejorCategoria = p.categoria.nombre;
      mejorCategoriaId = p.categoria.id;
      mejorPuntaje = presentes;
    }
  }
}
if (mejorCategoria) {
  cambios.categoriaInteres = mejorCategoria;
  cambios.categoriaId = mejorCategoriaId;
}

// Link de Google Maps pegado como texto: se guarda deterministicamente,
// sin depender de que el modelo llame a actualizar_datos_lead. Bug real
// probando en vivo: el cliente ya habia mandado un link invalido, se le
// pidio uno nuevo, mando uno bueno (maps.app.goo.gl) y el modelo no lo
// volvio a guardar - direccionEntrega se quedo pegada en el primer link
```

```
  // invalido para siempre, y confirmar_pedido rechazaba el pedido una y
  // otra vez sin que hubiera forma de salir de ahi.
  if (extraerUrlDeMaps(texto)) cambios.direccionEntrega = String(texto).trim().slice(0, 300);

  const cant = norm.match(/\b(\d+)\s*(unidades?|piezas?|und\.?)\b/);
  if (cant) cambios.cantidad = cant[1];

  const talla = extraerTallaDelTexto(texto, productos);
  if (talla) cambios.talla = talla;

  // Forma de pago y tipo de entrega: el cliente muchas veces contesta con
  // UNA sola palabra ("QR", "domicilio") y el modelo no siempre se acuerda
  // de guardarla con actualizar_datos_lead antes de seguir - bug real
  // reportado: confirmar_pedido volvia a pedir la forma de pago que el
  // cliente ya habia dicho un mensaje antes. Se detecta aca, en codigo,
  // igual que categoria/talla, asi no depende de que el modelo se acuerde.
  if (/\bqr\b/.test(norm)) cambios.formaPago = 'QR';
  else if (/\befectivo\b/.test(norm)) cambios.formaPago = 'EFECTIVO';
  else if (/\btarjeta\b/.test(norm)) cambios.formaPago = 'TARJETA';

  if (/\b(domicilio|delivery)\b/.test(norm)) cambios.tipoEntrega = 'DOMICILIO';
  else if (/\b(recojo|recoger|retiro|retirar)\b/.test(norm)) cambios.tipoEntrega = 'RECOJO';

  return cambios;
}
```

catalogo.js: buscarConFallback (selecciona los candidatos)

Archivo: `lib/services/catalogo.js` *· líneas 409–448*

```
/**
 * Busca aplicando la cascada de alternativas.
 *
 * @returns {{resultados: Array, total: number, relajado: string|null}}
 *   relajado = que filtro hubo que aflojar para encontrar algo (null si el
 *   resultado es exacto). El bot tiene que decirselo al cliente y PREGUNTARLE
 *   antes de mostrar, nunca aflojar en silencio.
 */
function buscarConFallback(productos, lead = {}) {
  // Lo mas amplio que se puede ofrecer sin traicionar lo que el cliente pidio:
  // se aflojan color, marca, presupuesto y talla, pero NUNCA la categoria ni
  // sus atributos (genero). Sirve para saber que MAS existe cuando el cliente
  // dice "no quiero esas, mostrame otras".
  const amplio = buscarProductosFiltrados(productos, lead, { ignorarPresupuesto: true, ignorarTalla: true });

  for (const escalon of ESCALONES) {
    // Un filtro que el cliente nunca definio no se puede "relajar": saltear
    // ese escalon evita anunciar que se aflojo algo que no existia.
    if (escalon.filtro === 'color' && !lead.color) continue;
    if (escalon.filtro === 'marca' && !lead.marca) continue;
    if (escalon.filtro === 'presupuesto' && rangoPresupuesto(lead).max == null) continue;
    if (escalon.filtro === 'talla' && !lead.talla) continue;
```

```
    const resultados = buscarProductosFiltrados(productos, lead, escalon.opciones);
    if (resultados.length) {
      // La cascada corta en el primer escalon con resultados, y eso escondia
      // el resto: si UN producto calzaba exacto, los que estaban a un color de
      // distancia no se veian nunca y el bot juraba que no habia mas. Ahora se
      // devuelven aparte, para poder ofrecerlos cuando el cliente pide otras.
      const ids = new Set(resultados.map((p) => p.id));
      return {
        resultados,
        total: resultados.length,
        relajado: escalon.filtro,
        adicionales: amplio.filter((p) => !ids.has(p.id)),
      };
    }
  }
  return { resultados: [], total: 0, relajado: null, adicionales: amplio };
}
```

catalogo.js: paginar

Archivo: `lib/services/catalogo.js` *· líneas 456–474*

```
/**
 * Separa el resultado completo en "lo que se muestra ahora" y "lo que queda".
 *
 * @param {Array} resultados  resultado completo de la busqueda
 * @param {number[]} yaMostrados  ids que este cliente ya vio en esta conversacion
 * @returns {{pagina: Array, total: number, yaVistos: number, restantes: number, hayMas: boolean}}
 */
function paginar(resultados, yaMostrados = [], porPagina = RESULTADOS_POR_PAGINA) {
  const vistos = new Set(yaMostrados);
  const pendientes = resultados.filter((p) => !vistos.has(p.id));
  const pagina = pendientes.slice(0, porPagina);
  return {
    pagina,
    total: resultados.length,
    yaVistos: resultados.length - pendientes.length,
    restantes: pendientes.length - pagina.length,
    hayMas: pendientes.length > pagina.length,
  };
}
```

2.3 Historial

obtenerHistorial (últimos 20 mensajes)

Archivo: `lib/services/conversaciones.js` *· líneas 162–172*

```
/**
 * Historial de una conversacion, para dar contexto a la IA.
 */
```

```
async function obtenerHistorial(conversacionId, limite = 20) {
  const mensajes = await prisma.mensaje.findMany({
    where: { conversacionId },
    orderBy: { createdAt: 'desc' },
    take: limite,
  });
  return mensajes.reverse().map((m) => ({ rol: m.rol, contenido: m.contenido }));
}
```

generarYRegistrarRespuesta (quita el mensaje actual del historial y llama al motor)

Archivo: `lib/services/conversaciones.js` *· líneas 174–197*

```
/**
 * Genera la respuesta del agente para una conversacion YA registrada (el
 * mensaje del cliente ya esta persistido y cobrado) y la guarda. Separado de
 * atenderMensaje para que el webhook de WhatsApp real pueda demorar este
 * paso (agrupar varios mensajes seguidos del cliente) sin tocar el cobro ni
 * el registro del mensaje, que deben pasar de inmediato.
 */
async function generarYRegistrarRespuesta(agenteId, telefonoCliente, conversacionId, contenido, baseUrl, modoPrueba = false) {
  const { generarRespuesta } = require('./agente');
  let historial = await obtenerHistorial(conversacionId);
  // El mensaje actual del cliente ya quedo persistido (se cobra/guarda de
  // inmediato, antes de este paso), asi que ya viene incluido como el
  // ultimo item del historial. Se excluye aca para no mandarselo dos veces
  // al motor (una vez dentro del historial, otra vez como mensaje actual).
  if (historial.length && historial[historial.length - 1].rol === 'CLIENTE' && historial[historial.length - 1].contenido ===
contenido) {
    historial = historial.slice(0, -1);
  }
  const salida = await generarRespuesta(agenteId, telefonoCliente, historial, contenido, conversacionId, { baseUrl, modoPrueba
});

  if (salida.ok && salida.respuesta) {
    await registrarRespuestaAgente(conversacionId, salida.respuesta);
  }
  return salida;
}
```

2.4 Definición de las tools (function calling)

TOOLS

Archivo: `lib/services/agente.js` *· líneas 220–393*

```
// ============================ TOOLS (function calling) ============================
// Definidas en formato OpenAI; toolsParaAnthropic() las convierte al formato
// de Anthropic (input_schema) para que ambos proveedores compartan una sola
// fuente de verdad.
const TOOLS = [
  {
```

```
    type: 'function',
    function: {
      name: 'actualizar_datos_lead',
      description: 'Guarda o actualiza los datos del cliente a medida que se obtienen en la conversacion. Llamar cada vez que el usuario entregue un dato nuevo.',
      parameters: {
        type: 'object',
        properties: {
          nombre: { type: 'string' },
          categoriaInteres: { type: 'string', description: "La categoria de producto con las palabras EXACTAS que uso el cliente. NUNCA la traduzcas ni la reemplaces por otra: el buscador interno necesita las palabras literales del cliente. NUNCA la cambies a otra categoria distinta solo porque no encontraste coincidencias exactas: la categoria se mantiene hasta que el cliente mismo diga que quiere ver otra cosa." },
          presupuesto: { type: 'string' },
          cantidad: { type: 'string', description: 'Cuantas unidades quiere el cliente.' },
          marca: { type: 'string', description: 'Marca preferida, si el cliente la menciono.' },
          talla: { type: 'string', description: 'Talla/tamaño que busca, si lo menciono.' },
          color: { type: 'string', description: 'Color preferido, si lo menciono.' },
          direccionEntrega: { type: 'string' },
          tipoEntrega: { type: 'string', enum: ['domicilio', 'recojo'], description: 'Como recibe el pedido: "domicilio" (hay que pedirle la direccion o su ubicacion) o "recojo" (pasa a buscarlo por la tienda). Ofrecele SOLO los tipos de entrega reales de esta tienda (ver bloque de arriba).' },
          formaPago: { type: 'string', enum: ['qr', 'efectivo', 'tarjeta'], description: 'Como va a pagar el cliente. Ofrecele SOLO las formas de pago reales de esta tienda (ver bloque de arriba), nunca otras.' },
          observaciones: { type: 'string' },
          nivelInteres: { type: 'string', enum: ['frio', 'tibio', 'caliente'] },
          estadoConversacion: {
            type: 'string',
            enum: ['explorando', 'buscando_producto', 'comparando', 'interesado', 'intencion_de_compra', 'listo_para_comprar'],
            description: 'En que etapa de decision de compra esta el cliente ahora mismo. Actualizalo cada vez que cambie de etapa: explorando (recien empieza, sin pedir nada concreto), buscando_producto (ya dijo que categoria/caracteristicas busca), comparando (esta viendo varias opciones mostradas), interesado (mostro interes claro en una en particular), intencion_de_compra (dijo que quiere comprar/apartar), listo_para_comprar (ya dio los datos y falta solo confirmar).',
          },
          productoFavoritoId: { type: 'integer', description: 'ID EXACTO del producto que el cliente eligio como su preferido cuando lo diga explicitamente (ej: "me gusta el segundo", "esa quiero", "la negra me convence"). A partir de ahi el sistema deja de sugerir otras opciones y se enfoca en cerrar la venta de este producto - no lo llenes si el cliente todavia esta comparando sin decidirse.' },
          varianteFavoritaId: { type: 'integer', description: 'ID EXACTO de la variante (talla+color concretos) que eligio, cuando ya se decidio por una combinacion puntual. Es lo que despues se vende de verdad.' },
          productosDescartadosIds: { type: 'array', items: { type: 'integer' }, description: 'IDs de productos que el cliente dijo explicitamente que NO le interesan (ej: "esa no me gusta", "el primero no"), para no volver a ofrecerselos.' },
          atributosCategoria: { type: 'object', description: 'Atributos que pidio el bloque de "datos que faltan" y NO son ya un campo propio (ej: {"Genero":"Hombre","Uso":"Running"}). Talla/color/marca van en sus propios parametros de arriba, no aca.' },
        },
      },
    },
  },
  {
    type: 'function',
```

```
    function: {
      name: 'mostrar_productos',
      description: 'Presenta productos al cliente como tarjetas visuales (foto + ficha con precio, stock y detalles). SIEMPRE que vayas a mostrar uno o mas productos del bloque de resultados, llama esta funcion con sus IDs en vez de describirlos en texto. Usa los IDs del bloque de resultados actual: el sistema ya eligio cuales corresponden mostrar ahora (son pocos a proposito, para no saturar el chat). Si hay mas resultados, se muestran despues con ver_mas_productos, nunca todos de golpe.',
      parameters: {
        type: 'object',
        properties: {
          idsProductos: { type: 'array', items: { type: 'integer' }, description: 'IDs exactos de los productos del bloque de resultados actual.' },
        },
        required: ['idsProductos'],
      },
    },
  },
  {
    type: 'function',
    function: {
      name: 'ver_mas_productos',
      description: 'Muestra el siguiente grupo de resultados de la MISMA busqueda, cuando el cliente pide ver otras opciones ("¿tenes otros modelos?", "mostrame mas", "que mas hay"). El sistema sabe cuales ya vio y cuales faltan: no hace falta que le pases nada. Nunca le digas al cliente que no hay mas sin haber llamado a esto primero.',
      parameters: { type: 'object', properties: {} },
    },
  },
  {
    type: 'function',
    function: {
      name: 'enviar_fotos_producto',
      description: 'Envia las fotos de un producto especifico por WhatsApp cuando el cliente pide ver fotos o imagenes de un producto. El sistema elige la foto del color que pidio el cliente y te avisa si esa foto es del color exacto o solo referencial de otro color.',
      parameters: {
        type: 'object',
        properties: { idProducto: { type: 'integer' } },
        required: ['idProducto'],
      },
    },
  },
  {
    type: 'function',
    function: {
      name: 'buscar_producto',
      description: 'Busca un producto POR NOMBRE en todo el catalogo cuando el cliente lo nombra ("¿tenes las Tekkira Cup?", "y la Superstar?"). Usala SIEMPRE antes de decirle que no lo tenes: el bloque de resultados de arriba esta filtrado por lo que venia pidiendo, asi que algo puede existir aunque no aparezca ahi. Si lo encuentra, mostraselo con mostrar_productos.',
      parameters: {
        type: 'object',
        properties: { nombre: { type: 'string', description: 'El nombre tal como lo nombro el cliente. Puede ser parcial ("tekkira" encuentra "ZAPATILLAS TEKKIRA CUP").' } },
```

```
        required: ['nombre'],
      },
    },
  },
  {
    type: 'function',
    function: {
      name: 'agregar_al_carrito',
      description: 'Agrega al carrito lo que el cliente eligio de una tarjeta ("me interesa la Ginger Tav en blanco talla 9"). El sistema valida el stock. Despues de agregar, preguntale si desea ver algo mas: si dice que si, volves al menu de categorias; si dice que no, cerras la venta.',
      parameters: {
        type: 'object',
        properties: {
          idProducto: { type: 'integer', description: 'ID exacto del producto que eligio, de los que le mostraste en tarjeta.' },
          idVariante: { type: 'integer', description: 'ID EXACTO de la variante (talla+color) que eligio. Obligatorio si el producto tiene variantes: el cliente ya las vio en la tarjeta.' },
          cantidad: { type: 'integer', description: 'Cuantas unidades. Si no lo dijo, 1.' },
        },
        required: ['idProducto'],
      },
    },
  },
  {
    type: 'function',
    function: {
      name: 'ver_carrito',
      description: 'Muestra lo que el cliente lleva agregado hasta ahora, con el total real. Usala si pregunta que lleva, o antes de cerrar.',
      parameters: { type: 'object', properties: {} },
    },
  },
  {
    type: 'function',
    function: {
      name: 'quitar_del_carrito',
      description: 'Saca un producto del carrito cuando el cliente se arrepiente ("sacame las negras").',
      parameters: {
        type: 'object',
        properties: {
          idProducto: { type: 'integer' },
          idVariante: { type: 'integer', description: 'Si agrego varias variantes del mismo producto, cual sacar.' },
        },
        required: ['idProducto'],
      },
    },
  },
  {
    type: 'function',
    function: {
```

```
      name: 'confirmar_pedido',
      description: 'Arma el resumen exacto del pedido (productos, variante, cantidad, precios reales, total, entrega y forma de pago) para leerselo al cliente ANTES de crear nada. Usa SIEMPRE lo que el cliente tiene en el carrito real - no le pases una lista de productos, el sistema ya sabe que hay. Es obligatorio llamarla antes de crear_pedido: el sistema no deja crear un pedido que el cliente no confirmo. Si falta algun dato, te dice cual pedir.',
      parameters: { type: 'object', properties: {} },
    },
  },
  {
    type: 'function',
    function: {
      name: 'crear_pedido',
      description: 'Crea el pedido DESPUES de que el cliente dijo que si al resumen que le leiste con confirmar_pedido. Usa SIEMPRE lo que el cliente tiene en el carrito real (el mismo que uso confirmar_pedido) - no le pases una lista de productos. Requiere nombre, tipo de entrega (y direccion si es a domicilio) y forma de pago ya guardados via actualizar_datos_lead. El sistema revalida stock y precio antes de confirmar. Registra la forma de pago elegida pero no procesa ningun cobro: eso lo coordina un asesor humano por fuera del bot.',
      parameters: { type: 'object', properties: {} },
    },
  },
  {
    type: 'function',
    function: {
      name: 'derivar_a_asesor',
      description: "Deriva la conversacion a un asesor humano. Usarla cuando el cliente lo pide explicitamente, esta claramente molesto tras varios intentos, o cuando detectes alguna de estas situaciones donde un vendedor humano puede cerrar mejor que vos: pedido mayorista o por volumen grande, compra de monto alto, negociacion especial de precio, solicitud de descuento fuera de lo que vos podes ofrecer, un problema o reclamo que no podes resolver, una condicion comercial especial (credito, factura, convenio), o un cliente que se identifica como empresa. NUNCA derivar por preguntas normales de precio, stock o forma de pago que si podes responder vos con la info disponible.",
      parameters: {
        type: 'object',
        properties: {
          motivo: { type: 'string' },
          tipo: {
            type: 'string',
            enum: ['solicitud_expresa', 'cliente_molesto', 'pedido_mayorista', 'compra_alto_valor', 'negociacion_especial', 'descuento_fuera_de_regla', 'problema_no_resuelto', 'condicion_comercial_especial', 'cliente_empresarial'],
            description: 'Motivo de la derivacion, para que el equipo priorice.',
          },
        },
      },
    },
  },
  {
    type: 'function',
    function: {
      name: 'mostrar_categorias',
      description: "Le muestra al cliente QUE VENDE la tienda, cuando pide 'el catalogo', 'que tienen', 'todos los productos', 'tus categorias' o algo generico parecido. El sistema arma la lista real: los rubros si todavia no eligio ninguno, o los tipos dentro del rubro que eligio. NUNCA armes vos esa lista de memoria ni mandes un link: llama a esta funcion.",
```

```
      parameters: { type: 'object', properties: {} },
    },
  },
  {
    type: 'function',
    function: {
      name: 'mostrar_tarjeta_categoria',
      description: "Presenta una categoria puntual la PRIMERA vez que el cliente la nombra o elige (ej. 'estoy buscando zapatillas deportivas', o elige 'Zapatillas' del menu). Manda una tarjeta con foto real de la categoria, cuantos modelos hay, desde que precio, 1-2 modelos destacados, y un link para ver todos. Llamala UNA sola vez por categoria, ANTES de mostrar productos individuales con mostrar_productos - despues, si el cliente pide ver modelos especificos, segui normal con mostrar_productos. No la llames si ya se la mostraste antes en esta conversacion para esa misma categoria.",
      parameters: { type: 'object', properties: {} },
    },
  },
];
```

toolsSegunEstado

Archivo: `lib/services/agente.js` *· líneas 395–407*

```
/**
 * Filtra la lista de tools segun el estado real del carrito - idea tomada
 * de un bot de citas (DentalFlow) que le saca al modelo, del propio array
 * de funciones disponibles, las que no tienen sentido en ese estado (ej.
 * "reservar" para un usuario no registrado). Con el carrito vacio,
 * confirmar_pedido/crear_pedido no tienen NADA que confirmar ni crear -
 * sacarlas de la lista es mas fuerte que solo pedirlo en el prompt, porque
 * la IA no puede llamar una funcion que ni siquiera le llego.
 */
function toolsSegunEstado(lead, conversacionId) {
  if (carritoDe((lead && lead.contexto) || {}, conversacionId).length > 0) return TOOLS;
  return TOOLS.filter((t) => t.function.name !== 'confirmar_pedido' && t.function.name !== 'crear_pedido');
}
```

toolsParaAnthropic

Archivo: `lib/services/agente.js` *· líneas 409–420*

```
function toolsParaAnthropic(tools) {
  const convertidas = tools.map((t) => ({
    name: t.function.name,
    description: t.function.description,
    input_schema: t.function.parameters,
  }));
  // Las tools nunca cambian entre llamadas: marcar la ultima como punto de
  // corte de cache le dice a Anthropic que todo el bloque (fijo) es
  // reutilizable, no solo el system prompt.
  if (convertidas.length) convertidas[convertidas.length - 1].cache_control = { type: 'ephemeral' };
  return convertidas;
}
```

---

3. Flujo de WhatsApp

Orden real de un mensaje entrante: webhook de Meta → `interpretarMensajeEntrante` (texto / botón / foto / audio / ubicación → texto) → `procesarMensajeEntrante` (guarda el mensaje y cobra la conversación si es nueva) → `encolarRespuesta` (espera 5 s por si llegan más mensajes) → `generarYRegistrarRespuesta` → `generarRespuesta` (motor) → `wa.enviarTexto` / `wa.enviarBotones` (y `wa.enviarImagen` desde las tools de tarjetas).

3.1 Webhook de WhatsApp (server.js)

GET /webhooks/whatsapp (verificación de Meta)

Archivo: `server.js` · líneas 2724–2735

```
// Verificacion del webhook (requerida por Meta al configurar la app).
app.get('/webhooks/whatsapp', (req, res) => {
  const mode = req.query['hub.mode'];
  const token = req.query['hub.verify_token'];
  const challenge = req.query['hub.challenge'];
  const verifyToken = process.env.WHATSAPP_VERIFY_TOKEN || 'proshop-verify-token';

  if (mode === 'subscribe' && token === verifyToken) {
    return res.status(200).send(challenge);
  }
  return res.sendStatus(403);
});
```

interpretarMensajeEntrante (texto, botones, foto, audio, ubicación)

Archivo: `server.js` · líneas 2737–2788

```
// Convierte CUALQUIER tipo de mensaje entrante de WhatsApp (texto, foto,
// audio, ubicacion, o una opcion elegida de un menu interactivo) a un texto
// natural en español. Ese texto es lo unico que ve el motor de ventas: asi
// el reconocimiento de fotos, la transcripcion de audio y la ubicacion NO
// duplican el flujo de conversacion, solo alimentan la misma puerta de
// entrada que un mensaje escrito a mano.
async function interpretarMensajeEntrante(mensaje, conexion, telefonoCliente) {
  if (mensaje.text?.body?.trim()) return mensaje.text.body.trim();
  if (mensaje.interactive?.list_reply?.id) return mensaje.interactive.list_reply.id;
  if (mensaje.interactive?.button_reply?.id) return mensaje.interactive.button_reply.id;

  if (mensaje.type === 'image' && mensaje.image?.id) {
    const media = await wa.obtenerMedia(conexion, mensaje.image.id);
    if (!media) return mensaje.image.caption?.trim() || '(el cliente envio una foto que no se pudo descargar)';

    const productos = await prisma.producto.findMany({
      where: { empresaId: conexion.agente.empresaId, activo: true },
      take: 200,
    });
    const descripcion = await analizarImagenProducto(media.buffer, media.mimeType, productos);
    const caption = mensaje.image.caption?.trim();
```

```
    const partes = [];
    if (caption) partes.push(caption);
    partes.push(descripcion ? `[Foto enviada por el cliente. Lo que se ve: ${descripcion}]` : '[El cliente envio una foto, pero no se pudo analizar su contenido]');
    return partes.join('\n');
  }

  if (mensaje.type === 'audio' && mensaje.audio?.id) {
    const media = await wa.obtenerMedia(conexion, mensaje.audio.id);
    if (!media) return '(el cliente envio un audio que no se pudo descargar; pidele amablemente que lo escriba)';

    const transcripcion = await transcribirAudio(media.buffer, media.mimeType);
    return transcripcion || '(el cliente envio un audio que no se pudo transcribir; pidele amablemente que lo escriba en texto)';
  }

  if (mensaje.type === 'location' && mensaje.location) {
    const { latitude, longitude, name, address } = mensaje.location;
    const direccionTexto = [name, address].filter(Boolean).join(' - ') || `https://www.google.com/maps?q=${latitude},${longitude}`;

    // Se guarda directo en la BD (dato estructurado real), sin depender de
    // que el modelo "entienda" coordenadas: mas confiable que un texto libre.
    await prisma.clienteFinal.upsert({
      where: { empresaId_telefono: { empresaId: conexion.agente.empresaId, telefono: telefonoCliente } },
      update: { direccionEntrega: direccionTexto, ubicacionLat: latitude, ubicacionLng: longitude },
      create: { empresaId: conexion.agente.empresaId, telefono: telefonoCliente, direccionEntrega: direccionTexto, ubicacionLat: latitude, ubicacionLng: longitude },
    });

    return `[El cliente compartio su ubicacion de entrega: ${direccionTexto}]`;
  }

  return null; // sticker, video, documento, etc.: no soportado por ahora
}
```

POST /webhooks/whatsapp

Archivo: `server.js` · líneas 2790–2884

```
app.post('/webhooks/whatsapp', (req, res) => {
  res.sendStatus(200); // responder rapido a Meta, procesar despues

  (async () => {
    try {
      const entry = req.body.entry?.[0];
      const cambio = entry?.changes?.[0]?.value;
      const mensaje = cambio?.messages?.[0];
      if (!mensaje) return; // eventos de status (entregado/leido), ignorar

      const phoneNumberId = cambio?.metadata?.phone_number_id;
      if (!phoneNumberId) return;
```

```
      const conexion = await prisma.conexionWhatsApp.findUnique({
        where: { phoneNumberId },
        include: { agente: true },
      });
      if (!conexion || !conexion.agente) {
        console.log(`--- webhook: no hay ningun agente conectado al phone_number_id ${phoneNumberId}`);
        return;
      }
      if (conexion.agente.estado !== 'ACTIVO') return;

      // Marca "leido" + "escribiendo..." antes de procesar - es un extra de
      // UX (el cliente ve que el mensaje llego mientras se arma la
      // respuesta), nunca debe frenar el procesamiento real si falla.
      wa.marcarLeidoYEscribiendo(conexion, mensaje.id).catch(() => {});

      const telefonoCliente = mensaje.from;
      const baseUrl = `${req.protocol}://${req.get('host')}`;
      await prisma.conexionWhatsApp.update({ where: { id: conexion.id }, data: { ultimoMensajeAt: new Date() } });

      // Nombre de perfil de WhatsApp (Meta lo manda en "contacts", junto a
      // "messages", en cada webhook con mensaje entrante) - se guarda para
      // mostrarlo en el panel aunque el cliente todavia no dio su nombre
      // real para el pedido. Nunca bloquea nada si falla o no viene.
      const nombrePerfil = cambio?.contacts?.[0]?.profile?.name || null;
      if (nombrePerfil) {
        prisma.clienteFinal.upsert({
          where: { empresaId_telefono: { empresaId: conexion.agente.empresaId, telefono: telefonoCliente } },
          update: { nombrePerfil },
          create: { empresaId: conexion.agente.empresaId, telefono: telefonoCliente, nombrePerfil },
        }).catch(() => {});
      }

      const texto = await interpretarMensajeEntrante(mensaje, conexion, telefonoCliente);
      if (!texto) return; // tipo de mensaje no soportado (sticker, video, documento...) o vacio

      // Se cobra y se guarda el mensaje YA (nunca se demora la parte contable).
      // Solo se demora GENERAR la respuesta, por si el cliente manda mas
      // mensajes seguidos sobre la misma idea (ver bufferMensajes.js).
      const entrada = await procesarMensajeEntrante({ agenteId: conexion.agente.id, telefonoCliente, contenido: texto,
referral: mensaje.referral });

      if (!entrada.ok) {
        console.log(`--- webhook: mensaje no atendido (${entrada.motivo}) para agente ${conexion.agente.id}`);
        notif.avisarMensajeNoAtendido(conexion.agente.empresaId, entrada.motivo).catch((e) =>
console.error('avisarMensajeNoAtendido:', e.message));
        return;
      }

      // Empuja el mensaje del cliente al panel en vivo, sin esperar a que la
      // IA responda (asi el equipo lo ve entrar al instante).
```

```
      emitMensaje(conexion.agente.empresaId, {
        conversacionId: entrada.conversacionId, rol: 'CLIENTE', contenido: texto, createdAt: new Date(),
      });

      // Un humano del equipo tiene tomado el control de este chat: la IA no
      // contesta (el mensaje ya quedo guardado arriba, solo se salta la
      // generacion de respuesta automatica).
      if (entrada.modo === 'HUMANO') return;

      encolarRespuesta(`${conexion.agente.id}:${telefonoCliente}`, async () => {
        const salida = await generarYRegistrarRespuesta(conexion.agente.id, telefonoCliente, entrada.conversacionId, texto,
baseUrl);
        if (salida.ok && salida.respuesta) {
          // Si el backend armo botones reales para esta respuesta (ver el
          // bloque preemptivo de forma de pago en agente.js), se manda como
          // mensaje interactivo en vez de texto plano - el cliente toca en
          // vez de tipear, y el ID que vuelve ya es el que espera el resto
          // del flujo (interpretarMensajeEntrante mas abajo).
          const envio = (salida.botones && salida.botones.length)
            ? await wa.enviarBotones(conexion, telefonoCliente, salida.respuesta, salida.botones)
            : await wa.enviarTexto(conexion, telefonoCliente, salida.respuesta);
          if (!envio.ok) {
            console.error(`--- webhook: fallo el envio por WhatsApp a ${telefonoCliente} (agente ${conexion.agente.id}):
${envio.error}`);
          }
          emitMensaje(conexion.agente.empresaId, {
            conversacionId: entrada.conversacionId, rol: 'AGENTE', contenido: salida.respuesta, createdAt: new Date(),
          });
        }
        notif.verificarAvisos(conexion.agente.empresaId).catch((e) => console.error('verificarAvisos:', e.message));
      });
    } catch (err) {
      console.error('Error procesando webhook de WhatsApp:', err);
    }
  })();
});
```

3.2 Conversaciones, cobro y registro de mensajes (conversaciones.js completo)

conversaciones.js completo

Archivo: `lib/services/conversaciones.js` *· líneas 1–233*

```
// Servicio de conversaciones.
//
// Regla comercial: "Una conversacion puede contener varios mensajes intercambiados
// entre el cliente y el agente". Por eso NO se cobra por mensaje: los mensajes de un
// mismo telefono dentro de una ventana de tiempo pertenecen a la misma conversacion
// y se cobran una sola vez.
//
// Al recibir un mensaje:
```

```
//   - Si hay una conversacion abierta (dentro de la ventana) -> se agrega, no cobra.
//   - Si no hay -> se intenta consumir 1 conversacion y se abre una nueva.
//   - Si no hay saldo -> no se atiende y se avisa para que compren un paquete.

const { prisma } = require('../db');
const { consumirConversacion } = require('./suscripciones');

// Ventana de agrupacion: cuanto tiempo sin mensajes hace falta para que la
// proxima vez que escriba el cliente se cobre como una conversacion nueva.
// Antes eran 24h (la ventana de sesion de WhatsApp), pero eso agrupaba
// demasiado - un cliente que escribia a la manana y volvia a la tarde
// seguia contando como la misma conversacion. Se bajo a 6h para que el
// cobro refleje mejor conversaciones realmente distintas.
const VENTANA_HORAS = Number(process.env.CONVERSATION_WINDOW_HOURS || 6);
const VENTANA_MS = VENTANA_HORAS * 60 * 60 * 1000;

// Mutex en memoria por (agente + telefono): si el mismo cliente manda 2
// mensajes casi al mismo tiempo (tipico: manda 2 seguidos antes de que el
// primero termine de procesarse), sin esto ambos podrian ver "no hay
// conversacion abierta todavia" y cobrar 2 conversaciones por lo que
// deberia ser 1 sola. Serializa el procesamiento de un mismo cliente sin
// bloquear a los demas.
const colasPorCliente = new Map();

function conBloqueoDeCliente(clave, tarea) {
  const anterior = colasPorCliente.get(clave) || Promise.resolve();
  const actual = anterior.then(tarea, tarea).finally(() => {
    if (colasPorCliente.get(clave) === actual) colasPorCliente.delete(clave);
  });
  colasPorCliente.set(clave, actual);
  return actual;
}

/**
 * Devuelve la conversacion abierta del cliente, o null si la ventana expiro.
 */
async function buscarConversacionAbierta(agenteId, telefonoCliente) {
  const ultima = await prisma.conversacion.findFirst({
    where: { agenteId, telefonoCliente },
    orderBy: { ultimoMensajeAt: 'desc' },
  });

  if (!ultima) return null;
  const dentroDeVentana = Date.now() - ultima.ultimoMensajeAt.getTime() < VENTANA_MS;
  return dentroDeVentana ? ultima : null;
}

/**
 * Procesa un mensaje entrante del cliente final.
 *
 * @returns {{
```

```
 *   ok:boolean, motivo?:string, cobrada:boolean,
 *   conversacionId?:number, origen?:string
 * }}
 */
async function procesarMensajeEntrante({ agenteId, telefonoCliente, contenido, referral }) {
  return conBloqueoDeCliente(`${agenteId}:${telefonoCliente}`, () =>
    procesarMensajeEntranteInterno({ agenteId, telefonoCliente, contenido, referral })
  );
}

async function procesarMensajeEntranteInterno({ agenteId, telefonoCliente, contenido, referral }) {
  const agente = await prisma.agente.findUnique({
    where: { id: agenteId },
    include: { empresa: { include: { suscripcion: true } } },
  });

  if (!agente) return { ok: false, motivo: 'AGENTE_NO_EXISTE', cobrada: false };
  if (agente.estado !== 'ACTIVO') return { ok: false, motivo: 'AGENTE_INACTIVO', cobrada: false };

  const suscripcion = agente.empresa.suscripcion;
  if (!suscripcion) return { ok: false, motivo: 'SIN_SUSCRIPCION', cobrada: false };

  // 1) Continuar una conversacion ya abierta: no se cobra
  const abierta = await buscarConversacionAbierta(agenteId, telefonoCliente);
  if (abierta) {
    await prisma.$transaction([
      prisma.mensaje.create({
        data: { conversacionId: abierta.id, rol: 'CLIENTE', contenido },
      }),
      prisma.conversacion.update({
        where: { id: abierta.id },
        data: { ultimoMensajeAt: new Date() },
      }),
    ]);
    return { ok: true, cobrada: false, conversacionId: abierta.id, origen: abierta.origen, modo: abierta.modo };
  }

  // 2) Conversacion nueva: hay que cobrarla
  const cobro = await consumirConversacion(
    suscripcion.id,
    `Conversacion con ${telefonoCliente}`
  );
  if (!cobro.ok) {
    return { ok: false, motivo: cobro.motivo, cobrada: false };
  }

  // Una conversacion nueva es una "sesion de venta" nueva: se resetean los
  // productos ya mostrados (contexto.fotosEnviadas / productosMostrados),
  // que son anti-spam DENTRO de un chat activo, no un bloqueo permanente.
  // Sin esto, un cliente que vuelve dias despues (o que se prueba varias
  // veces con el mismo numero) se queda para siempre sin poder ver fichas
```

```
  // que ya se le mandaron una vez. La categoria/presupuesto/marca/talla/etc.
  // SI se mantienen: esa memoria entre visitas es la que queremos.
  // OJO: no pisar contexto entero con {} — eso borraba tambien el carrito
  // en curso (bug real, un pedido se quedo sin uno de sus productos porque
  // justo en el medio se abrio una conversacion "nueva"). Solo se resetea
  // lo que es anti-spam de verdad.
  const clienteExistente = await prisma.clienteFinal.findUnique({
    where: { empresaId_telefono: { empresaId: agente.empresaId, telefono: telefonoCliente } },
  });
  const contextoPrevio = (clienteExistente && clienteExistente.contexto) || {};
  await prisma.clienteFinal.upsert({
    where: { empresaId_telefono: { empresaId: agente.empresaId, telefono: telefonoCliente } },
    update: {
      contexto: { ...contextoPrevio, fotosEnviadas: [], tarjetasCategoriaMostradas: [] },
      productosMostrados: [],
    },
    create: { empresaId: agente.empresaId, telefono: telefonoCliente },
  });

  // referral: solo llega en el primer mensaje de una conversacion que
  // arranco desde un anuncio "Click to WhatsApp" de Meta. Se guarda una
  // sola vez, al crear la conversacion (ver Conversacion.anuncioId).
  const conversacion = await prisma.conversacion.create({
    data: {
      agenteId,
      telefonoCliente,
      origen: cobro.origen,
      ultimoMensajeAt: new Date(),
      anuncioId: referral?.source_id || null,
      anuncioTitulo: referral?.headline || null,
      anuncioImagenUrl: referral?.image_url || referral?.thumbnail_url || null,
      mensajes: { create: { rol: 'CLIENTE', contenido } },
    },
  });

  return { ok: true, cobrada: true, conversacionId: conversacion.id, origen: cobro.origen, modo: conversacion.modo };
}

/**
 * Guarda la respuesta del agente dentro de la conversacion (no cobra).
 */
async function registrarRespuestaAgente(conversacionId, contenido) {
  await prisma.$transaction([
    prisma.mensaje.create({ data: { conversacionId, rol: 'AGENTE', contenido } }),
    prisma.conversacion.update({
      where: { id: conversacionId },
      data: { ultimoMensajeAt: new Date() },
    }),
  ]);
}
```

```
/**
 * Historial de una conversacion, para dar contexto a la IA.
 */
async function obtenerHistorial(conversacionId, limite = 20) {
  const mensajes = await prisma.mensaje.findMany({
    where: { conversacionId },
    orderBy: { createdAt: 'desc' },
    take: limite,
  });
  return mensajes.reverse().map((m) => ({ rol: m.rol, contenido: m.contenido }));
}

/**
 * Genera la respuesta del agente para una conversacion YA registrada (el
 * mensaje del cliente ya esta persistido y cobrado) y la guarda. Separado de
 * atenderMensaje para que el webhook de WhatsApp real pueda demorar este
 * paso (agrupar varios mensajes seguidos del cliente) sin tocar el cobro ni
 * el registro del mensaje, que deben pasar de inmediato.
 */
async function generarYRegistrarRespuesta(agenteId, telefonoCliente, conversacionId, contenido, baseUrl, modoPrueba = false) {
  const { generarRespuesta } = require('./agente');
  let historial = await obtenerHistorial(conversacionId);
  // El mensaje actual del cliente ya quedo persistido (se cobra/guarda de
  // inmediato, antes de este paso), asi que ya viene incluido como el
  // ultimo item del historial. Se excluye aca para no mandarselo dos veces
  // al motor (una vez dentro del historial, otra vez como mensaje actual).
  if (historial.length && historial[historial.length - 1].rol === 'CLIENTE' && historial[historial.length - 1].contenido ===
contenido) {
    historial = historial.slice(0, -1);
  }
  const salida = await generarRespuesta(agenteId, telefonoCliente, historial, contenido, conversacionId, { baseUrl, modoPrueba
});

  if (salida.ok && salida.respuesta) {
    await registrarRespuestaAgente(conversacionId, salida.respuesta);
  }
  return salida;
}

/**
 * Flujo completo: recibe un mensaje del cliente, lo cobra si corresponde,
 * genera la respuesta del agente con IA y la registra. Devuelve la respuesta.
 * Pensado para un solo mensaje aislado (ej. el chat de prueba del panel). El
 * webhook de WhatsApp real NO usa esto directamente: agrupa mensajes
 * seguidos primero (ver lib/services/bufferMensajes.js).
 */
async function atenderMensaje({ agenteId, telefonoCliente, contenido, baseUrl, modoPrueba = false }) {
  const entrada = await procesarMensajeEntrante({ agenteId, telefonoCliente, contenido });
  if (!entrada.ok) return { ...entrada, respuesta: null };

  // Un humano del equipo tiene tomado el control de este chat: la IA no
```

```
  // contesta hasta que alguien lo devuelva (ver /panel/conversaciones/:id/control).
  if (entrada.modo === 'HUMANO') return { ...entrada, respuesta: null, tomadaPorHumano: true };

  const salida = await generarYRegistrarRespuesta(agenteId, telefonoCliente, entrada.conversacionId, contenido, baseUrl,
modoPrueba);

  return {
    ...entrada,
    respuesta: salida.respuesta,
    demo: salida.demo,
    modelo: salida.modelo,
    fotos: salida.fotos || [],
  };
}

module.exports = {
  VENTANA_HORAS,
  buscarConversacionAbierta,
  procesarMensajeEntrante,
  registrarRespuestaAgente,
  obtenerHistorial,
  generarYRegistrarRespuesta,
  atenderMensaje,
};
```

3.3 Agrupación de mensajes seguidos (bufferMensajes.js completo)

bufferMensajes.js completo

Archivo: `lib/services/bufferMensajes.js` *· líneas 1–43*

```
// Agrupa mensajes seguidos del mismo cliente antes de generar la respuesta.
//
// En WhatsApp real la gente escribe fragmentado: "hola" / "quiero una
// mochila" / "que sea negra" en 3 mensajes seguidos en vez de una sola
// frase. Sin esto, cada mensaje disparaba su propia llamada a la IA y su
// propia respuesta: el cliente recibia 3 contestaciones sueltas (una posible
// a medio pensar, con el "hola" solo) en vez de una sola respuesta a la idea
// completa - ademas de gastar 3 llamadas a la IA por lo que es una sola
// intencion de compra.
//
// Cada mensaje se sigue guardando y cobrando al instante (ver
// conversaciones.js:procesarMensajeEntrante); esto SOLO retrasa el momento
// de generar y enviar la respuesta, esperando a que el cliente deje de
// escribir por un rato corto.
// 5 segundos: es el punto medio entre agrupar los mensajes cortos seguidos y
// no dejar al cliente esperando. Estaba en 8s y se sentia lento del otro lado.
// Bajarlo mas agrupa menos y gasta mas llamadas a la IA (cada tanda es una
// llamada); subirlo hace que el cliente sienta que el bot no contesta.
const DEBOUNCE_MS = Number(process.env.MENSAJE_DEBOUNCE_MS || 5000);
```

```
const timers = new Map(); // clave (agenteId:telefono) -> Timeout

/**
 * Programa "tarea" para ejecutarse cuando el cliente lleve DEBOUNCE_MS sin
 * mandar un mensaje nuevo. Si llega otro mensaje del mismo cliente antes de
 * que se cumpla el plazo, se cancela lo anterior y se reprograma con la
 * tarea nueva (que ya tiene el contexto mas reciente, porque el historial en
 * BD para ese momento incluye todos los mensajes recibidos hasta ahi).
 */
function encolarRespuesta(clave, tarea) {
  clearTimeout(timers.get(clave));
  const timer = setTimeout(async () => {
    timers.delete(clave);
    try {
      await tarea();
    } catch (err) {
      console.error(`Error generando respuesta agrupada (${clave}):`, err.message);
    }
  }, DEBOUNCE_MS);
  timers.set(clave, timer);
}

module.exports = { encolarRespuesta, DEBOUNCE_MS };
```

3.4 Envío por WhatsApp: texto, imagen, botones, descarga de medios (whatsapp.js completo)

whatsapp.js completo

Archivo: `lib/services/whatsapp.js` *· líneas 1–243*

```
// Cliente de la WhatsApp Business Cloud API (Meta).
// Se usa para enviar mensajes desde el número conectado de cada cliente.
const { prisma } = require('../db');
const { descifrar } = require('../crypto');

const GRAPH_VERSION = process.env.WHATSAPP_GRAPH_VERSION || 'v21.0';

/**
 * Envía un mensaje de texto por WhatsApp usando las credenciales de una conexión.
 * @param {object} conexion  fila de ConexionWhatsApp (con phoneNumberId y tokenCifrado)
 * @param {string} para      número destino en formato internacional (solo dígitos)
 * @param {string} texto
 * @returns {Promise<{ok:boolean, id?:string, error?:string}>}
 */
async function enviarTexto(conexion, para, texto) {
  if (!conexion || !conexion.phoneNumberId || !conexion.tokenCifrado) {
    return { ok: false, error: 'La conexión no tiene credenciales configuradas.' };
  }
  const token = descifrar(conexion.tokenCifrado);
  if (!token) return { ok: false, error: 'No se pudo leer el token de acceso.' };
```

```
  const url = `https://graph.facebook.com/${GRAPH_VERSION}/${conexion.phoneNumberId}/messages`;
  try {
    const resp = await fetch(url, {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${token}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        messaging_product: 'whatsapp',
        to: String(para).replace(/\D/g, ''),
        type: 'text',
        text: { body: texto },
      }),
    });
    const data = await resp.json().catch(() => ({}));
    if (!resp.ok) {
      const msg = (data && data.error && data.error.message) || `Error HTTP ${resp.status}`;
      return { ok: false, error: msg };
    }
    const id = data.messages && data.messages[0] && data.messages[0].id;
    return { ok: true, id };
  } catch (err) {
    return { ok: false, error: err.message };
  }
}

function esperar(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

/**
 * Marca un mensaje entrante como leido y muestra el indicador de "escribiendo..."
 * (hasta 25s o hasta que se manda la respuesta real, lo que pase primero). Es
 * un extra de UX: si falla no debe frenar el procesamiento real del mensaje,
 * por eso el llamador la usa "fire and forget" con su propio try/catch.
 * @returns {Promise<{ok:boolean, error?:string}>}
 */
async function marcarLeidoYEscribiendo(conexion, messageId) {
  if (!conexion || !conexion.phoneNumberId || !conexion.tokenCifrado || !messageId) {
    return { ok: false, error: 'La conexión no tiene credenciales configuradas.' };
  }
  const token = descifrar(conexion.tokenCifrado);
  if (!token) return { ok: false, error: 'No se pudo leer el token de acceso.' };

  const url = `https://graph.facebook.com/${GRAPH_VERSION}/${conexion.phoneNumberId}/messages`;
  try {
    const resp = await fetch(url, {
      method: 'POST',
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify({
        messaging_product: 'whatsapp',
        status: 'read',
        message_id: messageId,
        typing_indicator: { type: 'text' },
      }),
    });
    const data = await resp.json().catch(() => ({}));
    if (!resp.ok) {
      const msg = (data && data.error && data.error.message) || `Error HTTP ${resp.status}`;
      return { ok: false, error: msg };
    }
    return { ok: true };
  } catch (err) {
    return { ok: false, error: err.message };
  }
}

/**
 * Envía una imagen (con caption opcional) por WhatsApp.
 * @returns {Promise<{ok:boolean, id?:string, error?:string}>}
 */
async function enviarImagen(conexion, para, urlImagen, caption) {
  if (!conexion || !conexion.phoneNumberId || !conexion.tokenCifrado) {
    return { ok: false, error: 'La conexión no tiene credenciales configuradas.' };
  }
  const token = descifrar(conexion.tokenCifrado);
  if (!token) return { ok: false, error: 'No se pudo leer el token de acceso.' };

  const url = `https://graph.facebook.com/${GRAPH_VERSION}/${conexion.phoneNumberId}/messages`;
  try {
    const resp = await fetch(url, {
      method: 'POST',
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({
        messaging_product: 'whatsapp',
        to: String(para).replace(/\D/g, ''),
        type: 'image',
        image: { link: urlImagen, caption: caption || '' },
      }),
    });
    const data = await resp.json().catch(() => ({}));
    if (!resp.ok) {
      const msg = (data && data.error && data.error.message) || `Error HTTP ${resp.status}`;
      return { ok: false, error: msg };
    }
    const id = data.messages && data.messages[0] && data.messages[0].id;
    return { ok: true, id };
  } catch (err) {
    return { ok: false, error: err.message };
  }
```

```
}

/**
 * Manda un mensaje interactivo con hasta 3 botones (WhatsApp no permite mas
 * que eso para este tipo de mensaje). Pensado para elecciones cerradas donde
 * el backend ya conoce las opciones reales de antemano (ej. forma de pago):
 * el cliente toca un boton en vez de tipear, y el ID que vuelve es
 * exactamente el que se le paso aca (interpretarMensajeEntrante en server.js
 * ya sabe leer `interactive.button_reply.id`).
 * @param {Array<{id:string, titulo:string}>} botones  maximo 3
 * @returns {Promise<{ok:boolean, id?:string, error?:string}>}
 */
async function enviarBotones(conexion, para, texto, botones) {
  if (!conexion || !conexion.phoneNumberId || !conexion.tokenCifrado) {
    return { ok: false, error: 'La conexión no tiene credenciales configuradas.' };
  }
  if (!Array.isArray(botones) || !botones.length || botones.length > 3) {
    return { ok: false, error: 'Se necesitan entre 1 y 3 botones.' };
  }
  const token = descifrar(conexion.tokenCifrado);
  if (!token) return { ok: false, error: 'No se pudo leer el token de acceso.' };

  const url = `https://graph.facebook.com/${GRAPH_VERSION}/${conexion.phoneNumberId}/messages`;
  try {
    const resp = await fetch(url, {
      method: 'POST',
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({
        messaging_product: 'whatsapp',
        to: String(para).replace(/\D/g, ''),
        type: 'interactive',
        interactive: {
          type: 'button',
          body: { text: texto },
          action: { buttons: botones.map((b) => ({ type: 'reply', reply: { id: b.id, title: b.titulo } })) },
        },
      }),
    });
    const data = await resp.json().catch(() => ({}));
    if (!resp.ok) {
      const msg = (data && data.error && data.error.message) || `Error HTTP ${resp.status}`;
      return { ok: false, error: msg };
    }
    const id = data.messages && data.messages[0] && data.messages[0].id;
    return { ok: true, id };
  } catch (err) {
    return { ok: false, error: err.message };
  }
}

/**
```

```
 * Manda varias imagenes en secuencia con una pequeña pausa entre cada una
 * (sin la pausa, WhatsApp Cloud API acepta todas las llamadas pero en la
 * practica solo entrega la ultima si se mandan demasiado rapido). La primera
 * imagen lleva el caption (la ficha del producto); las demas van limpias.
 */
async function enviarImagenes(conexion, para, urlsImagenes, captionPrimera) {
  const resultados = [];
  let primera = true;
  for (const url of urlsImagenes) {
    resultados.push(await enviarImagen(conexion, para, url, primera ? captionPrimera : undefined));
    primera = false;
    await esperar(1200);
  }
  return resultados;
}

/**
 * Descarga un archivo multimedia (foto o audio) que un cliente envio por
 * WhatsApp. La API de Meta entrega el archivo en 2 pasos: primero resuelve
 * el mediaId a una URL firmada (que expira y requiere el mismo token), y
 * recien esa URL se puede descargar de verdad.
 * @returns {Promise<{buffer:Buffer, mimeType:string}|null>}
 */
async function obtenerMedia(conexion, mediaId) {
  if (!conexion || !conexion.tokenCifrado || !mediaId) return null;
  const token = descifrar(conexion.tokenCifrado);
  if (!token) return null;

  try {
    const metaResp = await fetch(`https://graph.facebook.com/${GRAPH_VERSION}/${mediaId}`, {
      headers: { Authorization: `Bearer ${token}` },
    });
    const meta = await metaResp.json().catch(() => ({}));
    if (!metaResp.ok || !meta.url) return null;

    const fileResp = await fetch(meta.url, { headers: { Authorization: `Bearer ${token}` } });
    if (!fileResp.ok) return null;

    const arrayBuffer = await fileResp.arrayBuffer();
    return { buffer: Buffer.from(arrayBuffer), mimeType: meta.mime_type || fileResp.headers.get('content-type') || '' };
  } catch (err) {
    console.error('Error descargando media de WhatsApp:', err.message);
    return null;
  }
}

/**
 * Verifica la conexión consultando los datos del número en la Graph API.
 * @returns {Promise<{ok:boolean, numero?:string, error?:string}>}
 */
async function verificarConexion(conexion) {
```

```
  if (!conexion || !conexion.phoneNumberId || !conexion.tokenCifrado) {
    return { ok: false, error: 'La conexión no tiene credenciales configuradas.' };
  }
  const token = descifrar(conexion.tokenCifrado);
  const url = `https://graph.facebook.com/${GRAPH_VERSION}/${conexion.phoneNumberId}?
fields=display_phone_number,verified_name`;
  try {
    const resp = await fetch(url, { headers: { Authorization: `Bearer ${token}` } });
    const data = await resp.json().catch(() => ({}));
    if (!resp.ok) {
      const msg = (data && data.error && data.error.message) || `Error HTTP ${resp.status}`;
      return { ok: false, error: msg };
    }
    return { ok: true, numero: data.display_phone_number, nombre: data.verified_name };
  } catch (err) {
    return { ok: false, error: err.message };
  }
}

module.exports = { enviarTexto, enviarImagen, enviarImagenes, enviarBotones, obtenerMedia, verificarConexion,
marcarLeidoYEscribiendo };
```

3.5 Botones de forma de pago (respuesta del backend sin IA)

generarRespuesta: botones de forma de pago preemptivos

Archivo: `lib/services/agente.js` *· líneas 3454–3481*

```
  // DECISION DETERMINISTICA (backend, no la IA): si al cliente en cierre YA
  // le falta UNICAMENTE la forma de pago (nombre, tipo de entrega y ubicacion
  // -si aplica- ya resueltos), se le manda un mensaje con BOTONES reales en
  // vez de dejar que la IA redacte la pregunta en texto libre. Espejo
  // DELIBERADAMENTE acotado de las condiciones que ya valida el handler de
  // confirmar_pedido/crear_pedido (nombreValido, tipoEntrega, direccion
  // resuelta) unas lineas mas arriba: si alguna de esas condiciones todavia
  // no esta lista, este bloque se autoexcluye solo y el flujo normal (IA +
  // ese mismo handler) sigue pidiendo lo que falta como siempre - nunca
  // intenta RESOLVER una direccion aca (eso sigue siendo trabajo exclusivo
  // del handler), solo mira si ya quedo resuelta antes.
  const carritoParaBotonesPago = carritoDe(lead.contexto || {}, conversacionId);
  if (carritoParaBotonesPago.length && !lead.formaPago && nombreValido(lead.nombre) && lead.tipoEntrega
    && (lead.tipoEntrega !== 'DOMICILIO' || Boolean(lead.ubicacionLat))
    && (lead.tipoEntrega !== 'RECOJO' || Boolean(agente.config && agente.config.direccionTienda))) {
    const metodosDePagoBotones = [
      agente.config && agente.config.aceptaQr ? { id: 'QR', titulo: 'QR' } : null,
      !agente.config || agente.config.aceptaEfectivo !== false ? { id: 'EFECTIVO', titulo: 'Efectivo' } : null,
      agente.config && agente.config.aceptaTarjeta ? { id: 'TARJETA', titulo: 'Tarjeta' } : null,
    ].filter(Boolean);
    // Con menos de 2 opciones no hay nada que "elegir" con botones (0 = ni
    // siquiera hay forma de pago configurada, 1 = no hace falta preguntar) -
    // en esos casos se deja que el flujo normal lo resuelva como hoy.
```

```
    if (metodosDePagoBotones.length >= 2) {
      logEtapa('botones_forma_pago_preemptivo', telefonoCliente, { opciones: metodosDePagoBotones.map((m) => m.id) });
      return { ok: true, demo: false, proveedor: 'sistema', modelo: null, respuesta: '¿Cómo preferís pagar?', botones:
metodosDePagoBotones, fotos: [] };
    }
  }
```

3.6 Envío de tarjetas con foto desde las tools

ejecutarFuncion: mostrar_productos / enviar_fotos_producto / ver_mas_productos

Archivo: `lib/services/agente.js` *· líneas 1995–2339*

```
  if (toolCall.name === 'mostrar_productos' || toolCall.name === 'enviar_fotos_producto' || toolCall.name ===
'ver_mas_productos') {
    const yaEnviadas = (clienteFinal.contexto && clienteFinal.contexto.fotosEnviadas) || [];
    // URLs de fotos ya enviadas: el control de duplicados es por imagen, no
    // por producto, para poder mandar otro color del mismo par.
    const urlsEnviadas = [...((clienteFinal.contexto && clienteFinal.contexto.urlsFotosEnviadas) || [])];
    const categoriaActual = categoriaDelLead(contexto, leadActual || {});

    // PEDIDO PUNTUAL: el cliente nombro estos productos y buscar_producto los
    // encontro en el catalogo. No es una busqueda por categoria, es "¿tenes
    // ESTO?" - y eso se contesta mostrandolo, no interrogandolo.
    const idsPorNombre = helpers.idsPedidosPorNombre || [];
    const idsPedidos = toolCall.name === 'mostrar_productos' ? (args.idsProductos || []).map(Number) : [];
    const esPedidoPuntual = idsPedidos.length > 0 && idsPedidos.every((id) => idsPorNombre.includes(id));

    // GATE EN CODIGO: mientras falten los datos que esta categoria marca como
    // imprescindibles, no se muestra NADA. El prompt ya se lo pide, pero el
    // modelo igual intenta mostrar productos apenas escucha una categoria
    // ("zapatillas" -> tres tarjetas), que es justo lo que el negocio no
    // quiere: primero entender, despues mostrar.
    //
    // No aplica al pedido puntual. Bug real con capturas: el cliente escribio
    // "tenes de casualidad ZAPATILLAS TEKKIRA CUP en venta", el sistema la
    // encontro (busqueda_por_nombre, conStock 1) y el gate la bloqueo por
    // falta de "Genero" - o sea, le preguntaba de que genero es el modelo que
    // el mismo acababa de nombrar. Los datos que falten se piden DESPUES de
    // contestarle, cuando hagan falta de verdad (para la talla, el color o el
    // cierre), nunca antes de decirle si lo tenemos.
    if (toolCall.name !== 'enviar_fotos_producto' && leadActual && !esPedidoPuntual) {
      const faltanIniciales = preguntasInicialesFaltantes(config, leadActual, contexto);
      if (faltanIniciales.length) {
        logEtapa('mostrar_bloqueado_preguntas_iniciales', telefonoCliente, { faltan: faltanIniciales });
        return `TODAVIA NO le muestres productos (TOOL_FAILED): falta saber ${faltanIniciales.join(' y ')}, que esta tienda
pide para cualquier busqueda. Preguntá SOLO "${faltanIniciales[0]}" y guardalo con actualizar_datos_lead.`;
      }

      // Mismo orden que seccionProductos y el handler de mostrar_categorias
      // (ver commit anterior): el atributo obligatorio del rubro/categoria va
```

```
      // ANTES del gate de subcategorias, no despues - si no, el rubro con
      // subcategorias Y un atributo propio (ej. Genero) nunca llegaba a
      // preguntarlo, siempre listaba los tipos sin filtrar primero.
      const faltantes = atributosFaltantes(categoriaActual, leadActual, 'OBLIGATORIO');
      if (faltantes.length) {
        logEtapa('mostrar_bloqueado_faltan_datos', telefonoCliente, { faltantes });
        return `TODAVIA NO le muestres productos: falta saber ${faltantes.join(' y ')}. No se envio ninguna tarjeta
(TOOL_FAILED). Preguntaselo primero de forma natural, de a una cosa por mensaje, guardalo con actualizar_datos_lead y recien
despues volve a mostrar.`;
      }

      // El cliente esta parado en un rubro que se subdivide: primero elige el
      // tipo. Sin esto el modelo muestra 3 productos cualquiera del rubro.
      if (categoriaActual && !categoriaActual.padreId) {
        const subs = subcategoriasDe(contexto, categoriaActual.id, leadActual);
        if (subs.length) {
          logEtapa('mostrar_bloqueado_falta_subcategoria', telefonoCliente, { rubro: categoriaActual.nombre });
          return `TODAVIA NO le muestres productos: "${categoriaActual.nombre}" se divide en ${subs.map((x) =>
x.nombre).join(', ')}. No se envio ninguna tarjeta (TOOL_FAILED). Pasale esa lista numerada y preguntale cual quiere ver;
cuando elija, ahi si.`;
        }
      }
    }

    const busqueda = leadActual ? buscarConFallback(contexto, leadActual) : { resultados: [], total: 0, relajado: null };

    // Se encontraron opciones solo aflojando un filtro: no se muestran hasta
    // que el cliente diga que quiere verlas (el documento es explicito: el
    // cliente decide si acepta ampliar la busqueda).
    //
    // El turno importa: no alcanza con "ya se lo pregunte", tiene que haber
    // sido en un turno ANTERIOR. Si no, el modelo pregunta y se autorresponde
    // en la misma vuelta, que es exactamente lo que se quiere evitar.
    if (toolCall.name === 'mostrar_productos' && busqueda.relajado && !esPedidoPuntual) {
      const preguntado = (clienteFinal.contexto || {}).relajadoPreguntado;
      const mismoFiltro = preguntado && preguntado.filtro === busqueda.relajado;
      const respondioElCliente = mismoFiltro && preguntado.turno !== helpers.turno;
      // Tope duro: pedir permiso mas de una vez por el mismo filtro es un
      // bucle, y al cliente lo desespera. Si ya se pregunto dos veces, se
      // muestra igual aclarando que no es exacto. Preferimos mostrar de mas
      // que dejar la conversacion trabada pidiendo permiso para siempre.
      const yaInsistimos = mismoFiltro && (preguntado.intentos || 1) >= 2;

      if (!respondioElCliente && !yaInsistimos) {
        await prisma.clienteFinal.update({
          where: { id: clienteFinal.id },
          data: {
            contexto: {
              ...(clienteFinal.contexto || {}),
              relajadoPreguntado: {
                filtro: busqueda.relajado,
```

```
                turno: helpers.turno,
                intentos: mismoFiltro ? (preguntado.intentos || 1) + 1 : 1,
              },
            },
          },
        });
        logEtapa('mostrar_bloqueado_filtro_relajado', telefonoCliente, { relajado: busqueda.relajado, intentos: mismoFiltro ? (preguntado.intentos || 1) + 1 : 1 });
        return `NO se mostro nada todavia (TOOL_FAILED a proposito): no hay coincidencia exacta y estas opciones aparecen recien aflojando ${NOMBRE_FILTRO_RELAJADO[busqueda.relajado]}. Primero decile eso con claridad y preguntale si quiere verlas igual. Cuando el cliente te conteste que si, volve a llamar mostrar_productos y ahi si se las mando.`;
      }
      if (yaInsistimos) {
        logEtapa('filtro_relajado_forzado', telefonoCliente, { relajado: busqueda.relajado });
      }
    }

    let ids;
    if (toolCall.name === 'enviar_fotos_producto') {
      ids = [Number(args.idProducto)];
    } else if (toolCall.name === 'ver_mas_productos') {
      // La siguiente pagina la elige el backend, no el modelo: es la unica
      // forma de garantizar que no repita ni se saltee resultados reales.
      ids = paginar(busqueda.resultados, yaEnviadas).pagina.map((p) => p.id);
      if (!ids.length) {
        return `Ya le mostraste TODAS las opciones reales de esta busqueda (total_matches = ${busqueda.total}, no queda ninguna sin mostrar). Decile con naturalidad que esas son todas las que hay para lo que pidio, y ofrecele ajustar algo (otro color, otra talla, otro presupuesto) o ver otra categoria.`;
      }
    } else {
      ids = (args.idsProductos || []).slice(0, MAX_PRODUCTOS_A_MOSTRAR).map(Number);
    }

    // NUNCA confiar a ciegas en el ID que manda el modelo: tiene que ser un
    // producto que de verdad salio en la busqueda real de esta conversacion
    // (candidato actual, ya mostrado, o el favorito guardado). Sin esto, si
    // el modelo confunde o inventa un ID (ej. describio productos en texto
    // plano en vez de con mostrar_productos, y despues "adivina" cual era
    // "el primero"), termina mandando la foto de un producto totalmente
    // distinto al que se esta hablando - bug real que ya paso.
    if (leadActual) {
      const idsValidos = new Set([
        ...busqueda.resultados.map((c) => c.id),
        // Lo que se encontro por nombre tambien es mostrable: existe de
        // verdad y el cliente lo pidio explicitamente.
        ...contexto.filter((c) => tieneStock(c)).map((c) => c.id),
        ...(leadActual.productosMostrados || []),
        ...yaEnviadas,
        ...(leadActual.productoFavoritoId ? [leadActual.productoFavoritoId] : []),
      ]);
      const idsInvalidos = ids.filter((id) => !idsValidos.has(id));
```

```
      if (idsInvalidos.length) {
        logEtapa('id_producto_rechazado', telefonoCliente, { tool: toolCall.name, idsInvalidos, idsValidos: [...idsValidos] });
        ids = ids.filter((id) => idsValidos.has(id));
        if (!ids.length) {
          return 'Ese producto no es parte de lo que se busco o mostro de verdad en esta conversacion (no inventes ni supongas un ID). Volve a mirar el bloque de resultados reales de arriba y usa el ID EXACTO del producto correcto - si no estas seguro cual es, primero llama a mostrar_productos con la busqueda actual para confirmarlo.';
        }
      }
    }

    // Si el modelo pide mostrar productos que YA se mandaron en esta misma
    // conversacion, no se reenvian las tarjetas (spam) - se le avisa que ya
    // las vio, para que en su texto siga la charla en vez de repetir.
    if (toolCall.name === 'mostrar_productos') {
      // Solo se bloquea el reenvio DENTRO DEL MISMO TURNO (evita que el modelo
      // mande la misma tarjeta tres veces seguidas). Si el cliente vuelve a
      // pedirlo en otro mensaje, se le manda de nuevo: negarle una tarjeta que
      // pidio porque "ya se la mostraron" hace 20 mensajes es tratarlo mal, y
      // ademas en WhatsApp ya la perdio de vista.
      const mostradosEnTurno = helpers.productosMostradosEnTurno || [];
      const idsNuevos = ids.filter((id) => !mostradosEnTurno.includes(id));
      if (ids.length && idsNuevos.length === 0) {
        const restantes = paginar(busqueda.resultados, yaEnviadas).pagina.length;
        return `Ya le mandaste esas tarjetas en este mismo mensaje: no las repitas. ${restantes ? 'Quedan otras opciones sin mostrar: si quiere ver mas, llama a ver_mas_productos.' : `Son las ${busqueda.total} opciones reales de esta busqueda.`}`;
      }
      // Nunca mas de una pagina de golpe, aunque el modelo pida diez IDs.
      ids = idsNuevos.slice(0, MAX_PRODUCTOS_A_MOSTRAR);

      // ...pero tampoco MENOS de una pagina: si el modelo pidio 1 sola
      // tarjeta habiendo 3 para mostrar, se completan las que faltan. Sin
      // esto la IA manda una tarjeta y "resuelve" el resto describiendolo en
      // texto plano, que es como empezo el bug de la foto equivocada (ver
      // docs/03, punto 7). El tope sigue siendo la pagina actual.
      //
      // EXCEPCION - pedido puntual: si TODO lo que se va a mostrar salio de
      // buscar_producto, es porque el cliente nombro ese producto. Ahi el
      // relleno no ayuda, molesta: pidio uno y recibia tres, con el suyo
      // ultimo. Se le muestra exactamente lo que pidio. Cuando esta
      // explorando ("que modelos tienen?") el relleno sigue igual que antes.
      if (esPedidoPuntual) {
        logEtapa('pedido_puntual_sin_relleno', telefonoCliente, { ids });
      } else {
        for (const c of paginar(busqueda.resultados, mostradosEnTurno).pagina) {
          if (ids.length >= MAX_PRODUCTOS_A_MOSTRAR) break;
          if (!ids.includes(c.id)) ids.push(c.id);
        }
      }
    }
```

```
    let productos = contexto.filter((p) => ids.includes(p.id));
    if (!productos.length) return 'No encontre esos productos para mostrartelos.';

    // Tope duro de fotos por turno, contando TODAS las herramientas que se
    // hayan ejecutado en este mismo turno (el modelo puede llamar mostrar_
    // productos y despues enviar_fotos_producto). Sin esto, el cliente
    // termina con el chat lleno de imagenes.
    const yaMandadasEnEsteTurno = helpers.fotosEnviadasEnTurno || 0;
    const cupo = Math.max(0, MAX_FOTOS_POR_TURNO - yaMandadasEnEsteTurno);
    if (!cupo) {
      return `Ya le mandaste ${MAX_FOTOS_POR_TURNO} fotos en este mismo turno, que es el maximo para no llenarle el chat. NO
mandes mas ahora: respondele con texto, y si quiere ver otras opciones se las mostras cuando te conteste.`;
    }
    productos = productos.slice(0, cupo);

    const nuevasEnviadas = [...yaEnviadas];
    const enviados = [];
    const avisos = [];
    const fallidos = [];

    for (const producto of productos) {
      const caption = fichaProducto(producto, leadActual || {}, moneda);
      const foto = fotoParaMostrar(producto, leadActual || {});

      // Cuando el cliente pide fotos de un color puntual se mandan TODAS las
      // de ese color (hasta el tope del turno), no una sola. Para la tarjeta
      // alcanza con la primera.
      let aEnviar = toolCall.name === 'enviar_fotos_producto'
        ? (foto.urls || []).filter((u) => !urlsEnviadas.includes(u)).slice(0, MAX_FOTOS_POR_TURNO -
(helpers.fotosEnviadasEnTurno || 0))
        : (foto.url ? [foto.url] : []);

      // El control de "ya se la mandé" es por FOTO, no por producto. Antes se
      // cortaba si el producto ya se habia mostrado, asi que pedir otro color
      // del mismo par devolvia "ya te envie todas las fotos" sin mandar nada
      // - y el bot se lo repetia al cliente. Bug real reportado con capturas.
      // Solo cuenta como "ya se la mandaste" si esa foto EXISTE y ya salio.
      // Si el color pedido directamente no tiene imagen, no es un duplicado:
      // se sigue de largo para que el aviso le explique al cliente que de ese
      // color no hay foto (y le ofrezca otra como referencia).
      if (toolCall.name === 'enviar_fotos_producto' && !aEnviar.length && (foto.urls || []).length) {
        const deEseColor = leadActual?.color ? ` de ${leadActual.color}` : '';
        return `Esas fotos${deEseColor} ya se las mandaste en esta conversacion (mira el historial). NO digas que se las estas
mandando de nuevo. ${foto.coloresConFoto.length > 1 ? `Los colores con foto de este producto son: ${foto.coloresConFoto.join(',
')} - si quiere ver otro, decilo y mandale ese.` : 'Preguntale si quiere ver otro producto o avanzar con el pedido.'}`;
      }

      const mediaUrl = aEnviar[0] || null;

      // TOOL_SUCCESS / TOOL_FAILED de verdad: antes se asumia que el envio
      // habia salido bien y el texto decia "te mande la foto" aunque hubiera
```

```
      // fallado. Ahora se mira el resultado real de la API de WhatsApp.
      let envioOk = true;
      let errorEnvio = null;
      if (conexion && conexion.estado === 'CONECTADO' && aEnviar.length) {
        const resultados = await wa.enviarImagenes(conexion, telefonoCliente, aEnviar, caption);
        const fallo = (resultados || []).find((r) => !r.ok);
        envioOk = !fallo;
        errorEnvio = fallo ? fallo.error : null;
      } else if (conexion && conexion.estado === 'CONECTADO') {
        const envio = await wa.enviarTexto(conexion, telefonoCliente, caption);
        envioOk = envio.ok;
        errorEnvio = envio.error || null;
      } else if (aEnviar.length && fotosParaMostrar) {
        // Sin conexion real (modo de prueba en el panel): se registran las
        // fotos para que el chat de prueba las muestre como imagenes de
        // verdad, no solo como texto.
        for (const u of aEnviar) fotosParaMostrar.push({ url: u, caption });
      }
      if (envioOk) urlsEnviadas.push(...aEnviar);

      if (!envioOk) {
        fallidos.push(`${producto.nombre}${errorEnvio ? ` (${errorEnvio})` : ''}`);
        logEtapa('envio_fallido', telefonoCliente, { tool: toolCall.name, productoId: producto.id, error: errorEnvio });
        continue;
      }

      // El cliente pidio un color puntual del que no hay foto propia: se
      // manda la que hay, pero el bot TIENE que aclarar que es referencial.
      // Hacer pasar la foto de un color por otro es justo lo que el negocio
      // marco como error grave.
      avisos.push(...avisosDeFoto(producto, foto));

      // La tarjeta se manda por WhatsApp arriba, pero tambien se guarda como
      // Mensaje real: si no, el historial de la conversacion en el panel
      // queda incompleto (el cliente recibio la foto/ficha, pero el equipo
      // que revisa la conversacion despues no la ve).
      if (conversacionId) {
        const mensajeGuardado = await prisma.mensaje.create({
          data: { conversacionId, rol: 'AGENTE', contenido: caption, mediaUrl, mediaTipo: mediaUrl ? 'imagen' : null },
        });
        emitMensaje(empresaId, {
          conversacionId, rol: 'AGENTE', contenido: caption, mediaUrl, mediaTipo: mensajeGuardado.mediaTipo, createdAt:
mensajeGuardado.createdAt,
        });
      }

      enviados.push(caption);
      helpers.fotosEnviadasEnTurno = (helpers.fotosEnviadasEnTurno || 0) + aEnviar.length;
      if (toolCall.name !== 'enviar_fotos_producto') {
        helpers.tarjetasEnTurno = (helpers.tarjetasEnTurno || 0) + 1;
        helpers.productosMostradosEnTurno = [...(helpers.productosMostradosEnTurno || []), producto.id];
```

```
  }
  if (!nuevasEnviadas.includes(producto.id)) nuevasEnviadas.push(producto.id);
}

if (!enviados.length) {
  logEtapa('tool_ejecutada', telefonoCliente, { tool: toolCall.name, tool_result: 'TOOL_FAILED', fallidos });
  return resultadoDeEnvio({ enviados: 0, fallidos });
}

const mostradosActuales = clienteFinal.productosMostrados || [];
const mostradosNuevos = toolCall.name === 'enviar_fotos_producto'
  ? mostradosActuales
  : [...new Set([...mostradosActuales, ...productos.map((p) => p.id)])];

await prisma.clienteFinal.update({
  where: { id: clienteFinal.id },
  data: {
    contexto: {
      ...(clienteFinal.contexto || {}),
      fotosEnviadas: nuevasEnviadas,
      // Se recortan a las ultimas 60: es memoria de una conversacion, no
      // un historial permanente.
      urlsFotosEnviadas: [...new Set(urlsEnviadas)].slice(-60),
    },
    productosMostrados: mostradosNuevos,
  },
});

// Complemento (no reemplazo) de la tarjeta: un producto puntual -> su
// pagina de detalle; varios -> la lista filtrada con el MISMO criterio
// que ya se uso para buscarlos, nunca el catalogo generico completo.
let linkWeb = null;
if (toolCall.name !== 'enviar_fotos_producto') {
  if (productos.length === 1) {
    linkWeb = await linkCatalogoWeb(helpers, `/producto/${productos[0].id}`);
  } else if (productos.length > 1) {
    const params = new URLSearchParams();
    if (leadActual?.categoriaInteres) params.set('categoria', leadActual.categoriaInteres);
    if (leadActual?.color) params.set('color', leadActual.color);
    if (leadActual?.talla) params.set('talla', leadActual.talla);
    if (leadActual?.marca) params.set('marca', leadActual.marca);
    linkWeb = await linkCatalogoWeb(helpers, `?${params.toString()}`);
  }
}
// Guardado en helpers (no solo en el string que le devolvemos al
// modelo): asi el codigo puede garantizar en la vuelta final que el
// link de verdad le llega al cliente, sin depender de que el modelo se
// acuerde de copiarlo en su texto (bug real: lo mencionaba "opcional" y
// el modelo simplemente no lo mandaba nunca).
if (linkWeb) helpers.linkWebPendiente = linkWeb;
```

```
    const pendientes = paginar(busqueda.resultados, nuevasEnviadas);
    logEtapa('tool_ejecutada', telefonoCliente, {
      tool: toolCall.name,
      tool_result: 'TOOL_SUCCESS',
      productos: productos.map((p) => p.id),
      total_matches: busqueda.total,
      results_returned: enviados.length,
      restantes: pendientes.pagina.length + pendientes.restantes,
    });

    return resultadoDeEnvio({
      enviados: enviados.length,
      fallidos,
      avisos,
      resumen: productos.map((p) => `${p.nombre} (${formatearPrecio(p.precio, moneda)})`).join('; '),
      total: busqueda.total,
      quedan: pendientes.pagina.length + pendientes.restantes,
      linkWeb,
    });
  }
```

ejecutarFuncion: mostrar_categorias

Archivo: `lib/services/agente.js` *· líneas 2767–2863*

```
  if (toolCall.name === 'mostrar_categorias') {
    // Los nombres de las categorias NUNCA dependen de datos del cliente
    // (genero, etc.) - eso solo importa para decidir que PRODUCTOS mostrar
    // adentro de una categoria, no para saber que categorias existen. Esa
    // proteccion real ya vive donde corresponde: mostrar_productos (linea
    // ~1795) y mostrar_tarjeta_categoria ya chequean preguntasInicialesFaltantes
    // antes de mostrar nada con precio/stock/foto. Antes este handler
    // bloqueaba TAMBIEN el menu de nombres, lo cual generaba que el cliente
    // no pudiera ni ver que rubros existen hasta contestar una pregunta que
    // no tenia nada que ver con eso - pedido explicito del dueño del negocio.
    // La lista la arma el sistema con lo que existe DE VERDAD y tiene stock.
    // Antes esto mandaba un link al catalogo web: el cliente tenia que salir
    // de WhatsApp, abrir una pagina y volver. Ahora la conversacion no se
    // interrumpe.
    const arbol = arbolDeCategorias(contexto, leadActual || {});
    if (!arbol.length) {
      return 'No hay ningun producto con stock en este momento (TOOL_FAILED). Decilo con transparencia y ofrece
derivar_a_asesor para avisarle cuando haya reposicion.';
    }

    // Si la tienda vende UN SOLO rubro y ese rubro no se subdivide en tipos,
    // no hay nada que elegir: preguntarle "¿te muestro esta categoria?" es un
    // paso de mas que solo frena la venta (bug real reportado: el cliente
    // contestaba "si, por favor" y el bot igual no mostraba nada). Se le
    // asigna esa categoria directo y se pasa a mostrar sus productos.
    if (leadActual && arbol.length === 1 && !arbol[0].subcategorias.length) {
      const unico = arbol[0];
```

```
      if (leadActual.categoriaId !== unico.id) {
        const datosCategoria = { categoriaId: unico.id, categoriaInteres: unico.nombre };
        await prisma.clienteFinal.update({ where: { id: clienteFinal.id }, data: datosCategoria });
        Object.assign(leadActual, datosCategoria);
      }
      const categoriaCompleta = categoriaDelLead(contexto, leadActual);
      const faltantesObligatorios = atributosFaltantes(categoriaCompleta, leadActual, 'OBLIGATORIO');
      if (faltantesObligatorios.length) {
        logEtapa('menu_rubro_unico_auto', telefonoCliente, { rubro: unico.nombre, faltantesObligatorios });
        return `TOOL_SUCCESS. Esta tienda vende solo "${unico.nombre}" (ya se la asigne como su categoria de interes, no le preguntes si quiere verla). Antes de mostrar productos todavia falta saber ${faltantesObligatorios.join(' y ')}. Preguntaselo de forma natural y guardalo con actualizar_datos_lead.`;
      }
      // Regla del negocio: ni en el rubro unico se saltea a mostrar_productos
      // directo - se manda la tarjeta de categoria como en cualquier otro
      // caso, para mantener la regla sin excepciones raras (Fix D).
      const resultadoTarjeta = await ejecutarFuncion({ name: 'mostrar_tarjeta_categoria', arguments: {} }, contexto, helpers);
      logEtapa('menu_rubro_unico_auto', telefonoCliente, { rubro: unico.nombre, tarjetaCategoria: true });
      return resultadoTarjeta;
    }

    // Si el cliente ya eligio un rubro, el menu baja un nivel: se le muestran
    // los tipos de ESE rubro, no los rubros otra vez. PERO si el mensaje de
    // este turno pide explicitamente ver todo el catalogo ("que mas venden",
    // "solo vendes zapatillas?"), se ignora la categoria ya fijada y se
    // muestran los rubros - sin esto el cliente quedaba pegado en la
    // categoria anterior sin forma de salir (extraerFiltros solo sabe entrar
    // a una categoria, nunca salir de una).
    const quiereCatalogoCompleto = pareceQuererCatalogoCompleto(helpers.mensajeCliente) || pidioVerOtraCosaRespondiendoAlBot(helpers.ultimoMensajeAgente);
    if (quiereCatalogoCompleto) {
      logEtapa('menu_catalogo_completo_forzado', telefonoCliente, {});
    }
    const categoriaActual = quiereCatalogoCompleto ? null : categoriaDelLead(contexto, leadActual || {});
    const rubroElegido = categoriaActual
      ? arbol.find((r) => r.id === categoriaActual.id || r.subcategorias.some((s) => s.id === categoriaActual.id))
      : null;

    if (rubroElegido && rubroElegido.subcategorias.length && !rubroElegido.subcategorias.some((s) => s.id === categoriaActual.id)) {
      // Mismo chequeo que ya hace el camino de "un solo rubro" unas lineas
      // arriba, antes de listar los tipos: si el rubro tiene un atributo
      // obligatorio propio (ej. Genero), hay que preguntarlo primero - sin
      // esto, la lista de tipos salia sin filtrar por ese dato (bug real
      // reportado: pregunta abierta e improvisada en vez de la cerrada que
      // corresponde).
      const faltantesObligatorios = atributosFaltantes(categoriaActual, leadActual, 'OBLIGATORIO');
      if (faltantesObligatorios.length) {
        logEtapa('menu_subcategorias_bloqueado_atributos', telefonoCliente, { rubro: rubroElegido.nombre, faltantesObligatorios });
        return `TOOL_SUCCESS. Antes de mostrarte los tipos de "${rubroElegido.nombre}" falta saber
```

```
${faltantesObligatorios.join(' y ')}. Preguntaselo de forma natural y guardalo con actualizar_datos_lead.`;
      }
      const lista = rubroElegido.subcategorias.map((s, i) => `${i + 1}. ${s.nombre}`).join('\n');
      logEtapa('menu_subcategorias', telefonoCliente, { rubro: rubroElegido.nombre, opciones: rubroElegido.subcategorias.length
});
      // Bandera APARTE de tarjetasEnTurno (que es especificamente "se mando
      // una foto real") - un menu de texto tambien es contenido real
      // entregado este turno, pero no es una tarjeta: si se reusara
      // tarjetasEnTurno aca, partesDelSystem le inyectaria al modelo la
      // instruccion "ya mandaste una tarjeta, no repitas el menu de
      // categorias" justo despues de haber mandado... el menu de categorias -
      // contradictorio. Bug real reportado: sin ninguna bandera para esto,
      // afirmaQueMostroAlgoSinTarjeta rechazaba como "invencion" una frase
      // honesta como "aqui tienes las opciones" despues de este mismo menu,
      // empujando al modelo a inventar un "tuve un problema" en la vuelta
      // siguiente por una lista de texto que SI se habia mandado de verdad.
      helpers.menuTextoEnTurno = (helpers.menuTextoEnTurno || 0) + 1;
      return `TOOL_SUCCESS. Dentro de "${rubroElegido.nombre}" hay estos tipos (son los reales con stock, no inventes otros ni
los renombres):\n${lista}\n\nTu texto ahora: pasale esta lista tal cual, numerada, una por linea, y preguntale cual quiere ver.
Nada de descripciones ni de mandar productos todavia.`;
    }

    const lista = arbol.map((r, i) => `${i + 1}. ${r.nombre}`).join('\n');
    logEtapa('menu_rubros', telefonoCliente, { opciones: arbol.length });
    // Mismo motivo que arriba: el menu de rubros tambien es contenido real
    // entregado en este turno (ver menuTextoEnTurno).
    helpers.menuTextoEnTurno = (helpers.menuTextoEnTurno || 0) + 1;
    return `TOOL_SUCCESS. Esto es lo que vende la tienda (rubros reales con stock, no inventes otros ni los
renombres):\n${lista}\n\nTu texto ahora: pasale esta lista tal cual, numerada, una por linea, y preguntale cual le interesa. NO
mandes ningun link, NO describas los rubros y NO muestres productos todavia: primero que elija.`;
  }
```

ejecutarFuncion: mostrar_tarjeta_categoria

Archivo: `lib/services/agente.js` *· líneas 2865–2956*

```
  if (toolCall.name === 'mostrar_tarjeta_categoria') {
    const categoriaActual = categoriaDelLead(contexto, leadActual || {});
    if (!categoriaActual) {
      return 'Todavia no hay ninguna categoria puntual elegida (TOOL_FAILED): primero hay que saber que categoria le interesa
antes de poder mandar esta tarjeta.';
    }

    // GATE EN CODIGO, mismo que mostrar_productos (linea ~1773): mientras
    // falten datos que la tienda marco como imprescindibles (ej. Genero), no
    // se manda ninguna tarjeta - ni de categoria ni de producto. Bug real
    // detectado en test: al redirigir mostrar_productos hacia esta tool
    // (Fix B, regla de "solo tarjeta de categoria"), se colaba la tarjeta
    // sin haber preguntado el atributo obligatorio todavia.
    const faltanIniciales = preguntasInicialesFaltantes(config, leadActual, contexto);
    if (faltanIniciales.length) {
      logEtapa('tarjeta_categoria_bloqueada_preguntas_iniciales', telefonoCliente, { faltan: faltanIniciales });
```

```
      return `TODAVIA NO le mandes esta tarjeta (TOOL_FAILED): falta saber ${faltanIniciales.join(' y ')}, que esta tienda pide para cualquier busqueda. Preguntá SOLO "${faltanIniciales[0]}" y guardalo con actualizar_datos_lead.`;
    }
    const faltantesObligatorios = atributosFaltantes(categoriaActual, leadActual, 'OBLIGATORIO');
    if (faltantesObligatorios.length) {
      logEtapa('tarjeta_categoria_bloqueada_atributos', telefonoCliente, { categoria: categoriaActual.nombre, faltantesObligatorios });
      return `TODAVIA NO le mandes esta tarjeta (TOOL_FAILED): falta saber ${faltantesObligatorios.join(' y ')} de "${categoriaActual.nombre}", que esta tienda marco como imprescindible. Preguntá SOLO "${faltantesObligatorios[0]}" y guardalo con actualizar_datos_lead.`;
    }

    // Una sola vez por categoria en la conversacion: repetirla es spam.
    const yaMostradas = (clienteFinal.contexto || {}).tarjetasCategoriaMostradas || [];
    if (yaMostradas.includes(categoriaActual.id)) {
      return `Ya le mandaste esto de "${categoriaActual.nombre}" antes en esta conversacion (TOOL_FAILED a proposito, no se repite). Si el cliente ya nombro un modelo puntual por su nombre, mostraselo con mostrar_productos; si no nombro nada puntual, preguntale por su nombre cual modelo le interesa - no vuelvas a mandar tarjetas sueltas sin que las nombre.`;
    }

    const resumen = resumenCategoria(contexto, categoriaActual.id);
    if (!resumen.cantidad) {
      return `No hay ningun producto con stock real en "${categoriaActual.nombre}" ahora mismo (TOOL_FAILED, no se mando nada). Decilo con transparencia, no inventes disponibilidad.`;
    }

    const params = new URLSearchParams();
    params.set('categoria', categoriaActual.nombre);
    const linkWeb = await linkCatalogoWeb(helpers, `?${params.toString()}`);

    const caption = `*${categoriaActual.nombre}*\n\n`
      + `🛍️ Tenemos ${resumen.cantidad} modelo${resumen.cantidad === 1 ? '' : 's'} disponible${resumen.cantidad === 1 ? '' : 's'}\n`
      + `💰 Desde ${formatearPrecio(resumen.precioDesde, moneda)}\n`
      + (resumen.destacados.length ? `✨ Por ejemplo: ${resumen.destacados.join(', ')}\n` : '')
      + (linkWeb ? `\n👉 Ver todos los modelos con foto, talla y color: ${linkWeb}` : '');

    // Si la tienda no cargo una foto de categoria a mano, se usa la de uno
    // de sus productos reales (ver fotoDestacada en resumenCategoria) - la
    // subida a mano sigue teniendo prioridad si existe.
    const imagenParaTarjeta = categoriaActual.imagenUrl || resumen.fotoDestacada;

    // Mismo patron que mostrar_productos: solo se afirma exito si el envio
    // real lo confirmo. Sin ninguna foto disponible, se manda el mismo
    // texto sin foto - nunca se inventa una imagen que no existe.
    let envioOk = true;
    if (imagenParaTarjeta && conexion && conexion.estado === 'CONECTADO') {
      const envio = await wa.enviarImagen(conexion, telefonoCliente, imagenParaTarjeta, caption);
      envioOk = envio.ok;
    } else if (conexion && conexion.estado === 'CONECTADO') {
      const envio = await wa.enviarTexto(conexion, telefonoCliente, caption);
```

```
      envioOk = envio.ok;
    } else if (imagenParaTarjeta && fotosParaMostrar) {
      // Modo de prueba (sin WhatsApp real conectado): se registra igual que
      // el resto de las tarjetas, para que el chat de prueba la muestre.
      fotosParaMostrar.push({ url: imagenParaTarjeta, caption });
    } else if (fotosParaMostrar) {
      // Bug real detectado probando en vivo: sin conexion real Y sin
      // ninguna foto disponible (ni de categoria ni de sus productos), no
      // entraba en NINGUNA de las ramas de arriba - el texto con
      // precio/cantidad/link se perdia entero, aunque la IA (o el bloque
      // pre-emptivo) igual confirmaba "TOOL_SUCCESS" y le decia al cliente
      // "echale un vistazo" a algo que nunca le llego. url:null -> el chat
      // de prueba lo muestra como texto (ver burbujaFoto en agente.ejs), sin
      // imagen rota.
      fotosParaMostrar.push({ url: null, caption });
    }

    if (!envioOk) {
      logEtapa('tarjeta_categoria_fallida', telefonoCliente, { categoria: categoriaActual.nombre });
      return 'No se pudo mandar la tarjeta de categoria (TOOL_FAILED, fallo el envio real). NO digas que se la mandaste.';
    }

    await prisma.clienteFinal.update({
      where: { id: clienteFinal.id },
      data: { contexto: { ...(clienteFinal.contexto || {}), tarjetasCategoriaMostradas: [...new Set([...yaMostradas,
categoriaActual.id])] } },
    });
    // Igual que mostrar_productos (linea ~2050): marca que este turno SI
    // mando una tarjeta real, para que los detectores de texto vago/mentiras
    // ("todavia no mostraste nada") no disparen en falso despues de un envio
    // exitoso de la tarjeta de categoria.
    helpers.tarjetasEnTurno = (helpers.tarjetasEnTurno || 0) + 1;
    logEtapa('tarjeta_categoria_enviada', telefonoCliente, { categoria: categoriaActual.nombre, cantidad: resumen.cantidad,
conImagen: Boolean(imagenParaTarjeta), imagenPropia: Boolean(categoriaActual.imagenUrl) });
    return `TOOL_SUCCESS: se le mando la tarjeta real de "${categoriaActual.nombre}" (${imagenParaTarjeta ? 'con foto' : 'sin
foto, ni la tienda ni sus productos tienen una cargada'}).\nTu texto ahora: UNA linea corta y natural invitandolo a mirar los
${categoriaActual.nombre.toLowerCase()} (ej: "Echale un vistazo a los ${categoriaActual.nombre.toLowerCase()} que tenemos, a
ver si alguno te gusta"). NO digas la palabra "tarjeta" ni "te envie/mande esto" - hablale del producto/categoria directamente,
no de lo que le llego. NO repitas los datos (cantidad, precio, modelos, link) que ya estan ahi arriba.`;
  }
```

fichaProducto (texto de la tarjeta de producto)

Archivo: `lib/services/agente.js` *· líneas 1260–1317*

```
function fichaProducto(p, lead = {}, moneda = 'BOB') {
  const lineas = [`· *Precio*: ${formatearPrecio(p.precio, moneda)}`];
  const atributos = p.atributos || {};
  if (atributos.Marca) lineas.push(`· *Marca*: ${atributos.Marca}`);
  if (atributos.Material) lineas.push(`· *Material*: ${atributos.Material}`);

  const variantes = (p.variantes || []).filter((v) => v.activa && v.stock > 0);
```

```
  if (variantes.length) {
    // La tarjeta muestra TODO lo que hay con stock, no solo lo que el cliente
    // pidio. Antes se filtraba por su talla y color, y cuando esa combinacion
    // tenia stock en un solo lugar la tarjeta quedaba con una linea sola: un
    // producto con 6 variantes parecia tener una. El cliente tiene que ver el
    // abanico real para poder elegir (reporte del dueño con capturas).
    const aMostrar = variantes;

    // Lo que el cliente pidio no se esconde: se le dice si esa combinacion
    // puntual esta o no, arriba de la lista completa.
    const tallasPedidas = tokensDeLista(lead.talla);
    const coloresPedidos = tokensDeLista(lead.color);
    if (tallasPedidas.length || coloresPedidos.length) {
      const calza = variantes.some((v) => (
        (!tallasPedidas.length || tallasPedidas.includes(normalizarSimple(v.atributos?.Talla)))
        && (!coloresPedidos.length || coloresPedidos.some((c) => valoresEquivalentes(c, v.atributos?.Color || '')))
      ));
      const pedido = [lead.talla ? `talla ${lead.talla}` : null, lead.color].filter(Boolean).join(' en ');
      lineas.push(calza
        ? `· *Lo que buscabas* (${pedido}): disponible ✅`
        : `· *Lo que buscabas* (${pedido}): sin stock por ahora, pero mira lo que si hay 👇`);
    }

    // Agrupadas por talla (una linea por talla con sus colores), no una
    // linea por cada combinacion talla+color: mucho mas facil de leer.
    const porTalla = new Map();
    for (const v of aMostrar) {
      const talla = v.atributos?.Talla || 'Unica';
      if (!porTalla.has(talla)) porTalla.set(talla, []);
      if (v.atributos?.Color) porTalla.get(talla).push(v.atributos.Color);
    }
    const entradasPorTalla = [...porTalla.entries()].sort((a, b) => compararTallas(a[0], b[0]));
    // Si TODAS las tallas vienen en exactamente los mismos colores (muy
    // comun: el mismo modelo en varias tallas), una linea por talla es puro
    // ruido repetido - se colapsa en una sola linea con el rango de tallas.
    const firmaColores = (colores) => [...colores].sort().join('|');
    const mismosColoresEnTodas = entradasPorTalla.length > 1
      && entradasPorTalla.every(([, colores]) => firmaColores(colores) === firmaColores(entradasPorTalla[0][1]));
    const lineasVariantes = mismosColoresEnTodas
      ? `· Tallas ${entradasPorTalla.map(([talla]) => talla).join(', ')}: ${entradasPorTalla[0][1].length ? entradasPorTalla[0]
[1].join(', ') : 'disponible'}`
      : entradasPorTalla
        .map(([talla, colores]) => `· Talla ${talla}: ${colores.length ? colores.join(', ') : 'disponible'}`)
        .join('\n');
    lineas.push(`\n*Tallas y colores disponibles*:\n${lineasVariantes}`);
  } else {
    lineas.push(`· *Disponibilidad*: ${p.stock > 0 ? (p.stock <= 3 ? `¡Ultimas ${p.stock} unidades!` : `${p.stock} en stock`) :
'Agotado'}`);
  }
  if (p.caracteristicas?.length) lineas.push(`\n${p.caracteristicas.map((c) => `✔ ${c}`).join('\n')}`);
  if (p.descripcion) lineas.push(`\n${p.descripcion}`);
```

```
  return `*${p.nombre}*\n\n${lineas.join('\n')}`;
}
```

catalogo.js: fotoParaMostrar

Archivo: `lib/services/catalogo.js` *· líneas 777–829*

```
/**
 * Decide QUE foto mandar y, sobre todo, si esa foto representa de verdad el
 * color que pidio el cliente.
 *
 * Nunca se hace pasar la foto de un color por otro: si el cliente pidio gris
 * y solo hay foto del blanco, se devuelve la del blanco pero marcada como
 * referencial (esDelColorPedido: false, colorDeLaFoto: 'Blanco') para que el
 * bot lo diga explicitamente. La talla no influye a proposito: las fotos se
 * comparten entre tallas de un mismo color por diseño.
 *
 * @returns {{url: string|null, colorDeLaFoto: string|null, esDelColorPedido: boolean,
 *            colorPedido: string|null, coloresConFoto: string[], coloresSinFoto: string[]}}
 */
function fotoParaMostrar(p, lead = {}) {
  const { conFoto, sinFoto } = coloresDeVariantes(p);
  const base = { coloresConFoto: conFoto, coloresSinFoto: sinFoto, colorPedido: lead.color || null };

  const coloresPedidos = tokensDeLista(lead.color);
  if (coloresPedidos.length) {
    const variantes = (p.variantes || []).filter((v) => v.activa && v.fotos?.length);
    // TODAS las variantes de ese color, no solo la primera: un mismo color
    // suele estar repartido en varias tallas y cada una puede aportar fotos
    // distintas del mismo par.
    const delColor = variantes.filter((v) => coloresPedidos.some((c) => valoresEquivalentes(c, v.atributos?.Color || '')));
    if (delColor.length) {
      const urls = [...new Set(delColor.flatMap((v) => v.fotos))];
      return { ...base, url: urls[0], urls, colorDeLaFoto: delColor[0].atributos?.Color || null, esDelColorPedido: true };
    }
    // Pidio un color puntual y no hay foto de ESE color. NO se manda la de
    // otro por las suyas: mandarle una zapatilla negra a quien pidio blancas
    // es peor que no mandar nada (paso en produccion). Se devuelve sin foto,
    // pero se informa cual se podria ofrecer COMO REFERENCIA si el cliente
    // acepta - que es lo que pide el punto 22 del documento del negocio.
    const referencia = variantes[0];
    return {
      ...base,
      url: null,
      urls: [],
      colorDeLaFoto: null,
      esDelColorPedido: false,
      referenciaDisponible: referencia ? (referencia.atributos?.Color || null) : null,
    };
  }

  // Sin color pedido no hay nada que aclarar: la foto generica es correcta.
```

```
  return {
    ...base,
    url: p.fotos?.length ? p.fotos[0] : null,
    urls: [...(p.fotos || [])],
    colorDeLaFoto: null,
    esDelColorPedido: true,
  };
}
```

---

4. Carrito de compras

El carrito vive en `ClienteFinal.contexto.carrito` (JSON), asociado al `conversacionId`. Máximo 20 líneas (`MAX_ITEMS`), vigencia 24 h (`HORAS_DE_VIDA`).

4.1 Módulo del carrito (carrito.js completo)

carrito.js completo

Archivo: `lib/services/carrito.js` · líneas 1–100

```
// Carrito de la conversacion.
//
// El cliente va agregando lo que quiere ("me interesa la Ginger Tav en blanco
// talla 9"), el bot le pregunta si desea ver algo mas, y al final se cierra
// todo junto con un resumen. Antes el pedido se armaba de una sola vez y el
// modelo tenia que "acordarse" de los items: si se olvidaba uno, se vendia de
// menos.
//
// Vive en ClienteFinal.contexto.carrito, no en una tabla: por decision del
// negocio el carrito NO sobrevive a la conversacion (si el cliente vuelve al
// dia siguiente, arranca limpio). Se guarda junto al id de conversacion para
// poder detectar exactamente eso.
//
// Puro y determinista: sin DB ni red, se testea sin levantar nada.

const MAX_ITEMS = 20;

// Mismo plazo que ya dura el link de la web del catalogo (sesionWeb.js,
// MINUTOS_DE_VIDA_POR_DEFECTO): pasadas las 24h el cliente ya no puede seguir
// agregando nada de todos modos, asi que un carrito mas viejo que eso es
// abandonado de verdad, no una charla que sigue activa.
const HORAS_DE_VIDA = 24;

/**
 * El carrito de ESTA conversacion. Si el guardado quedo de una conversacion
 * anterior, o quedo mas de HORAS_DE_VIDA sin tocarse (carrito abandonado, ej.
 * armado desde la web y nunca confirmado), se devuelve vacio: el cliente que
 * vuelve otro dia empieza de cero. No se borra el JSON de la base a la fuerza
 * -no hace falta un job aparte-, simplemente se deja de ver como "abierto"
 * en cualquier lugar que lo lea, mismo criterio que ya usa el chequeo de
 * conversacionId de aca abajo.
 */
function carritoDe(contexto = {}, conversacionId = null, ahora = Date.now()) {
  const guardado = contexto.carrito;
  if (!guardado || !Array.isArray(guardado.items)) return [];
  if (conversacionId && guardado.conversacionId && guardado.conversacionId !== conversacionId) return [];
  if (guardado.actualizadoEn) {
    const horasPasadas = (ahora - new Date(guardado.actualizadoEn).getTime()) / (60 * 60 * 1000);
    if (horasPasadas > HORAS_DE_VIDA) return [];
  }
```

```
  return guardado.items;
}

function guardarCarrito(contexto = {}, conversacionId, items, ahora = Date.now()) {
  return { ...contexto, carrito: { conversacionId: conversacionId || null, items: items.slice(0, MAX_ITEMS), actualizadoEn: new
Date(ahora).toISOString() } };
}

function contextoSinCarrito(contexto = {}) {
  return { ...contexto, carrito: null };
}

// Dos lineas son la misma si apuntan al mismo producto Y a la misma variante:
// la misma zapatilla en talla 9 y en talla 10 son items distintos.
function mismaLinea(a, b) {
  return a.productoId === b.productoId && (a.varianteId || null) === (b.varianteId || null);
}

/**
 * Agrega (o suma cantidad si ya estaba). Devuelve el carrito nuevo, sin mutar
 * el anterior.
 */
function agregarItem(items, item) {
  const existente = items.find((i) => mismaLinea(i, item));
  if (existente) {
    return items.map((i) => (mismaLinea(i, item) ? { ...i, cantidad: i.cantidad + item.cantidad } : i));
  }
  return [...items, item];
}

function quitarItem(items, { productoId, varianteId = null }) {
  return items.filter((i) => !mismaLinea(i, { productoId, varianteId }));
}

function totalCarrito(items) {
  return items.reduce((suma, i) => suma + Number(i.precio) * i.cantidad, 0);
}

/**
 * Resumen legible del carrito, con el formateador de precio que corresponda a
 * la moneda del negocio (se inyecta para no acoplar esto a agente.js).
 */
function resumenCarrito(items, formatearPrecio) {
  if (!items.length) return 'El carrito esta vacio.';
  const lineas = items.map((i) => `- ${i.cantidad}x ${i.nombre} — ${formatearPrecio(i.precio)} c/u`);
  return `${lineas.join('\n')}\nTotal: ${formatearPrecio(totalCarrito(items))}`;
}

// Los items en el formato que espera crear_pedido. Se lleva agregadoEn (si el
// item lo tiene) para que confirmar_pedido pueda avisar cuando algo lleva
// horas en el carrito de una sesion que no se llego a cerrar - nunca se
```

```
// inventa, si el item no lo tiene queda null.
function itemsParaPedido(items) {
  return items.map((i) => ({ idProducto: i.productoId, idVariante: i.varianteId || undefined, cantidad: i.cantidad, agregadoEn:
i.agregadoEn || null }));
}

module.exports = {
  carritoDe, guardarCarrito, contextoSinCarrito,
  agregarItem, quitarItem, totalCarrito, resumenCarrito, itemsParaPedido,
  MAX_ITEMS,
};
```

4.2 Tools del carrito: agregar, ver y quitar

ejecutarFuncion: agregar_al_carrito / ver_carrito / quitar_del_carrito

Archivo: `lib/services/agente.js` *· líneas 2383–2483*

```
  if (toolCall.name === 'agregar_al_carrito' || toolCall.name === 'ver_carrito' || toolCall.name === 'quitar_del_carrito') {
    const precioTexto = (monto) => formatearPrecio(monto, moneda);
    let items = carritoDe(clienteFinal.contexto || {}, conversacionId);

    if (toolCall.name === 'ver_carrito') {
      const resumen = resumenCarrito(items, precioTexto);
      // 100% determinista (solo muestra estado ya calculado): se salta la
      // vuelta extra de la IA, ver el short-circuit en el loop principal.
      return {
        paraModelo: `Esto lleva el cliente:
${resumen}
${items.length ? 'Leeselo tal cual. Si quiere cerrar, pedile lo que falte (nombre, entrega, forma de pago) y llama a
confirmar_pedido.' : 'Esta vacio: invitalo a elegir algo de lo que le mostraste.'}`,
        respuestaDirecta: items.length
          ? `Esto es lo que llevás:\n${resumen}`
          : 'Tu carrito está vacío. ¿Querés que te muestre opciones?',
      };
    }

    if (toolCall.name === 'quitar_del_carrito') {
      const antes = items.length;
      items = quitarItem(items, { productoId: Number(args.idProducto), varianteId: args.idVariante ? Number(args.idVariante) :
null });
      await prisma.clienteFinal.update({
        where: { id: clienteFinal.id },
        data: { contexto: guardarCarrito(clienteFinal.contexto || {}, conversacionId, items) },
      });
      logEtapa('carrito_quitar', telefonoCliente, { quedan: items.length });
      if (antes === items.length) {
        return {
          paraModelo: 'Ese producto no estaba en el carrito. Deciselo con naturalidad y mostrale lo que si lleva.',
          respuestaDirecta: null, // requiere que la IA describa lo que SI lleva, con naturalidad
        };
```

```
      }
      const resumen = resumenCarrito(items, precioTexto);
      return {
        paraModelo: `Listo, lo saque. Ahora el carrito tiene:
${resumen}`,
        respuestaDirecta: `Listo, lo saqué. Ahora tu carrito tiene:\n${resumen}`,
      };
    }

    // --- agregar_al_carrito ---
    const producto = await prisma.producto.findFirst({
      where: { id: Number(args.idProducto), empresaId, activo: true },
      include: { variantes: { where: { activa: true } } },
    });
    if (!producto) return 'Ese producto no existe o no esta activo. Usa el ID exacto de una tarjeta que le hayas mostrado.';

    const cantidad = Math.max(1, Number(args.cantidad) || 1);
    let variante = null;
    if (producto.variantes.length) {
      if (!args.idVariante) {
        const opciones = producto.variantes
          .filter((v) => v.stock > 0)
          .map((v) => `- [Variante ID ${v.id}] ${formatearAtributosVariante(v.atributos)} - Stock: ${v.stock}`)
          .join('\n');
        return `Falta saber cual combinacion eligio de "${producto.nombre}". Esto es lo que hay con stock (ya lo vio en la
tarjeta):
${opciones}
Preguntaselo en una sola frase y volve a llamar agregar_al_carrito con el idVariante.`;
      }
      variante = producto.variantes.find((v) => v.id === Number(args.idVariante));
      if (!variante) return `Esa variante no es de "${producto.nombre}". Mira las de la tarjeta y usa el ID exacto.`;
      if (variante.stock < cantidad) {
        return `No hay stock suficiente de "${producto.nombre}" (${formatearAtributosVariante(variante.atributos)}): quedan
${variante.stock}. Deciselo y ofrecele esa cantidad u otra combinacion.`;
      }
    } else if (producto.stock < cantidad) {
      return `No hay stock suficiente de "${producto.nombre}": quedan ${producto.stock}.`;
    }

    const precioUnitario = Number(variante ? (variante.precio ?? producto.precio) : producto.precio);
    items = agregarItem(items, {
      productoId: producto.id,
      varianteId: variante ? variante.id : null,
      nombre: variante ? `${producto.nombre} (${formatearAtributosVariante(variante.atributos)})` : producto.nombre,
      precio: precioUnitario,
      cantidad,
      agregadoEn: new Date().toISOString(),
    });

    await prisma.clienteFinal.update({
      where: { id: clienteFinal.id },
```

```
      data: {
        contexto: guardarCarrito(clienteFinal.contexto || {}, conversacionId, items),
        productoFavoritoId: producto.id,
        varianteFavoritaId: variante ? variante.id : null,
        estadoConversacion: 'INTENCION_DE_COMPRA',
      },
    });
    if (leadActual) leadActual.productoFavoritoId = producto.id;
    logEtapa('carrito_agregar', telefonoCliente, { productoId: producto.id, varianteId: variante?.id || null, cantidad,
itemsEnCarrito: items.length });

    // 100% determinista (el dato ya se guardo, solo falta confirmar): se
    // salta la vuelta extra de la IA, ver el short-circuit en el loop
    // principal.
    return {
      paraModelo: `TOOL_SUCCESS: agregado al carrito.
${resumenCarrito(items, precioTexto)}

Tu texto ahora: confirmale en una linea que lo agregaste y preguntale SI DESEA VER ALGO MAS. NUNCA le ofrezcas ni menciones
otro producto por tu cuenta en este mensaje: si el mismo pide ver algo especifico, ahi si lo buscas. Si dice que si quiere ver
mas, llama a mostrar_categorias. Si dice que no, cerra: pedile nombre, como quiere recibirlo y forma de pago, y despues llama a
confirmar_pedido.`,
      respuestaDirecta: fraseAlAzar(CONFIRMACIONES_AGREGADO),
    };
  }
```

Frases fijas de confirmación del carrito (respuesta sin segunda llamada a la IA)

Archivo: `lib/services/agente.js` *· líneas 1956–1967*

```
// Frases de tramite para las operaciones de carrito que son 100%
// deterministas (ver "respuestaDirecta" en ejecutarFuncion): varias opciones
// equivalentes, elegidas al azar, para no sonar repetitivo turno a turno -
// mismo espiritu que la regla de variacion del prompt (partesDelSystem).
const CONFIRMACIONES_AGREGADO = [
  '¡Listo, lo agregué a tu carrito! ¿Querés ver algo más?',
  'Perfecto, ya está en tu carrito. ¿Buscás algo más?',
  'Agregado ✅ ¿Te muestro algo más o seguimos con esto?',
];
function fraseAlAzar(opciones) {
  return opciones[Math.floor(Math.random() * opciones.length)];
}
```

4.3 Detectores relacionados con el carrito

afirmaAgregoAlCarrito (detecta "ya lo agregué" sin haberlo hecho)

Archivo: `lib/services/agente.js` *· líneas 980–997*

```
/**
 * Detecta que el bot afirma que un producto YA esta en el carrito. Se cruza
 * siempre con el carrito REAL (ver uso en el loop de vueltas): si el carrito
```

```
 * esta vacio, la afirmacion es necesariamente falsa - no hace falta entender
 * de que producto habla, alcanza con que diga "ya esta" y no haya nada.
 *
 * Bug real con capturas: el cliente dijo "estoy viendo la Park St 2.0" (solo
 * la estaba mirando, sin talla ni color) y el bot respondio "ya la tienes en
 * tu carrito" en el mismo mensaje en el que recien preguntaba la talla -
 * agregar_al_carrito exige la variante, asi que en verdad no se habia
 * agregado nada. El cliente se entera del pedido incompleto recien al leer
 * el resumen final.
 */
function afirmaAgregoAlCarrito(texto) {
  if (!texto) return false;
  const t = normalizarSimple(texto);
  return /(ya (lo|la|los|las) tien[e]?s? en tu carrito|ya (esta|quedo) en tu carrito|(lo|la|los|las) agregu[eé] a tu
carrito|agregad[oa]s? a tu carrito|carrito ya (tiene|cuenta con))/.test(t);
}
```

5. Catálogo web y su conexión con WhatsApp

El catálogo público no tiene login: el bot genera un link con un token firmado (empresa + teléfono + conversación) y la web usa ese token para escribir en **el mismo** `contexto.carrito` que usa el bot. Al agregar desde la web se deja `carritoWebPendiente` para que el bot reaccione en el próximo turno.

5.1 Token de sesión (sesionWeb.js completo)

sesionWeb.js completo

Archivo: `lib/services/sesionWeb.js` *· líneas 1–71*

```
// Token de sesion para el catalogo web publico.
//
// El catalogo (/catalogo/:slug) no tiene login: el cliente llega desde un
// link que le manda el bot por WhatsApp. Para que la web sepa "quien sos"
// sin pedirle usuario/contrasena, el bot firma un token con la identidad que
// YA conoce (empresa + telefono + conversacion) y se lo pasa en la URL. La
// web valida ese token en cada accion que toca el carrito; sin token valido
// se puede mirar el catalogo pero no agregar nada (evita que cualquiera
// escriba el carrito de otro cliente adivinando o copiando una URL vieja).
//
// Puro: nada de DB ni red, se testea sin levantar nada (igual que carrito.js).

const crypto = require('crypto');

const SEPARADOR = '.';
const MINUTOS_DE_VIDA_POR_DEFECTO = 60 * 24; // 24h: el link no deberia durar mas que la conversacion misma.

// Reusa la misma variable de entorno que ya protege los tokens de WhatsApp
// cifrados (ver lib/crypto.js) en vez de pedir una nueva. Mismo fallback de
// desarrollo que ahi: en produccion, lib/crypto.js ya revienta el arranque
// si falta, asi que para cuando esto se ejecuta la variable real ya esta.
const SECRETO = process.env.APP_ENCRYPTION_KEY || 'clave-de-desarrollo-cambia-esto-en-produccion';

function firmar(payloadTexto) {
  return crypto.createHmac('sha256', SECRETO).update(payloadTexto).digest('base64url');
}

/**
 * Genera un token para que el cliente pueda agregar al carrito desde la web
 * sin loguearse. Codifica lo minimo indispensable: con que empresa/telefono/
 * conversacion actuar, y cuando vence.
 */
function generarTokenSesion({ empresaId, telefono, conversacionId }, minutosDeVida = MINUTOS_DE_VIDA_POR_DEFECTO) {
  if (!empresaId || !telefono) throw new Error('generarTokenSesion necesita empresaId y telefono.');
  const vence = Date.now() + minutosDeVida * 60 * 1000;
  const payload = { empresaId, telefono, conversacionId: conversacionId || null, vence };
  const payloadTexto = Buffer.from(JSON.stringify(payload)).toString('base64url');
  const firma = firmar(payloadTexto);
  return `${payloadTexto}${SEPARADOR}${firma}`;
```

```
}

/**
 * Valida un token recibido por query param. Devuelve el payload si es
 * valido y no vencio, o null si es invalido/manipulado/vencido - nunca
 * tira una excepcion por un token malformado (puede venir de cualquier
 * lado, un usuario pegando una URL rota no puede romper la pagina).
 */
function verificarTokenSesion(token) {
  if (!token || typeof token !== 'string' || !token.includes(SEPARADOR)) return null;
  const [payloadTexto, firma] = token.split(SEPARADOR);
  if (!payloadTexto || !firma) return null;

  const firmaEsperada = firmar(payloadTexto);
  // Comparacion en tiempo constante: es una firma de seguridad, no un id
  // cualquiera - no queremos filtrar por timing cuanto de la firma coincide.
  const bufA = Buffer.from(firma);
  const bufB = Buffer.from(firmaEsperada);
  if (bufA.length !== bufB.length || !crypto.timingSafeEqual(bufA, bufB)) return null;

  let payload;
  try {
    payload = JSON.parse(Buffer.from(payloadTexto, 'base64url').toString('utf8'));
  } catch {
    return null;
  }
  if (!payload || typeof payload.vence !== 'number' || Date.now() > payload.vence) return null;
  if (!payload.empresaId || !payload.telefono) return null;
  return payload;
}

module.exports = { generarTokenSesion, verificarTokenSesion, MINUTOS_DE_VIDA_POR_DEFECTO };
```

5.2 Generación del link desde el bot

linkCatalogoWeb

Archivo: `lib/services/agente.js` *· líneas 1208–1234*

```
/**
 * Que se le responde al modelo despues de intentar mandar las tarjetas.
 *
 * La regla del documento es que la IA no puede afirmar que hizo algo si el
 * sistema no confirmo que ocurrio: por eso el resultado siempre arranca con
 * TOOL_SUCCESS o TOOL_FAILED segun lo que paso DE VERDAD con el envio.
 */
// Link al catalogo web para "ver mejor antes de decidir": foto grande,
// descripcion completa, selector de talla/color con stock real. Es un
// COMPLEMENTO de la tarjeta que ya se mando, nunca un reemplazo - y solo
// tiene sentido mandarlo cuando ya se mostro algo real (esta funcion se
// llama desde el resultado de mostrar_productos/ver_mas_productos, nunca
```

```
// antes). Sin baseUrl o slug (ej. en tests sin request HTTP real) devuelve
// null y no se menciona nada - nunca un link roto.
async function linkCatalogoWeb(helpers, ruta) {
  if (!helpers.baseUrl || !helpers.empresaSlug) return null;
  try {
    const token = generarTokenSesion({ empresaId: helpers.empresaId, telefono: helpers.telefonoCliente, conversacionId:
helpers.conversacionId });
    const destino = `${helpers.baseUrl}/catalogo/${helpers.empresaSlug}${ruta}${ruta.includes('?') ? '&' :
'?'}s=${encodeURIComponent(token)}`;
    // El link con el token es largo (se ve feo y desconfia por WhatsApp) -
    // se manda acortado; server.js resuelve /l/:codigo al destino real.
    const corto = await prisma.linkCorto.create({ data: { destino } });
    return `${helpers.baseUrl}/l/${corto.id.toString(36)}`;
  } catch {
    return null;
  }
}
```

conLinkPendienteSiFalta

Archivo: `lib/services/agente.js` *· líneas 1236–1247*

```
// Si en este turno se calculo un link real al catalogo web (ver
// linkCatalogoWeb) y el texto final del modelo no lo incluyo, se agrega en
// codigo antes de mandarlo. No se puede confiar en que el modelo copie un
// link solo porque el prompt se lo sugiere como "opcional" - bug real: el
// cliente pedia ver mas modelos y el link nunca le llegaba.
function conLinkPendienteSiFalta(texto, helpers) {
  const link = helpers.linkWebPendiente;
  if (!link) return texto;
  helpers.linkWebPendiente = null; // una sola vez por turno
  if (texto && texto.includes(link)) return texto;
  return `${texto || ''}\n\n👉 Para ver más con foto grande y elegir talla/color: ${link}`;
}
```

5.3 Rutas del catálogo web (server.js)

GET /catalogo/:slug (listado)

Archivo: `server.js` *· líneas 287–369*

```
// Catalogo publico de una empresa (sin login): el agente manda este link
// cuando el cliente pide "el catálogo" en general, en vez de un producto
// puntual. Reutiliza el slug de la empresa (el mismo que ya usa cada tenant).
app.get('/catalogo/:slug', async (req, res, next) => {
  try {
    const empresa = await prisma.empresa.findUnique({
      where: { slug: req.params.slug },
      include: { agentes: { take: 1, include: { config: true } } },
    });
    if (!empresa) return res.status(404).render('404', { title: 'Página no encontrada' });
```

```
    const productosDb = await prisma.producto.findMany({
      where: { empresaId: empresa.id, activo: true },
      orderBy: [{ categoria: { nombre: 'asc' } }, { nombre: 'asc' }],
      include: { variantes: { where: { activa: true } }, categoria: { include: { padre: true } } },
    });
    // Si el producto tiene variantes, el stock real es la suma de sus
    // variantes (Producto.stock queda en 0 a proposito en ese caso) - mismo
    // criterio que /panel/productos, si no el catalogo publico muestra
    // "Agotado" en productos que si tienen stock real.
    const productos = productosDb.map((p) => ({
      ...p,
      stockMostrado: p.variantes.length ? p.variantes.reduce((suma, v) => suma + v.stock, 0) : p.stock,
    }));
    const agente = empresa.agentes[0];
    const config = agente ? agente.config : null;

    // El bot manda links con filtro real (categoria/color/talla/marca):
    // mismas funciones que ya usa el chat, para que la web nunca muestre un
    // criterio distinto al del bot. Sin filtros, se ve el catalogo entero
    // agrupado por categoria (comportamiento de siempre).
    const filtro = {
      categoria: (req.query.categoria || '').trim(),
      color: (req.query.color || '').trim(),
      talla: (req.query.talla || '').trim(),
      marca: (req.query.marca || '').trim(),
    };
    const hayFiltro = Boolean(filtro.categoria || filtro.color || filtro.talla || filtro.marca);
    const productosAMostrar = hayFiltro
      ? buscarProductosFiltrados(
          productos,
          { categoriaInteres: filtro.categoria || null, color: filtro.color || null, talla: filtro.talla || null, marca:
filtro.marca || null },
          { estrictoColor: true, estrictoMarca: true }
        )
      : productos;

    // Agrupa por categoria (orden alfabetico, "Otros" al final para
    // productos sin categoria) para que el catalogo se pueda navegar por
    // secciones en vez de una sola lista larga. Con filtro activo se
    // muestra todo en una sola seccion (ya es un resultado acotado).
    const grupos = new Map();
    for (const p of productosAMostrar) {
      const cat = hayFiltro ? 'Resultados' : (p.categoria?.padre?.nombre || p.categoria?.nombre || 'Otros');
      if (!grupos.has(cat)) grupos.set(cat, []);
      grupos.get(cat).push(p);
    }
    const categorias = [...grupos.entries()]
      .sort(([a], [b]) => (a === 'Otros' ? 1 : b === 'Otros' ? -1 : a.localeCompare(b, 'es')))
      .map(([nombre, items]) => ({ nombre, productos: items }));

    // El token de sesion (si vino uno valido en la URL) se propaga a los
```

```
    // links de "ver producto" para que la pagina de detalle sepa quien es
    // sin pedirle nada al cliente.
    const sesion = verificarTokenSesion(req.query.s);

    // Open Graph: para que WhatsApp (y cualquier otra app) arme una vista
    // previa real cuando alguien comparte este link - complementa (no
    // reemplaza) la tarjeta con foto real que ya manda el bot.
    const ogTitle = hayFiltro ? `${filtro.categoria || 'Catálogo'} · ${empresa.marca || empresa.nombre}` : `Catálogo · ${empresa.marca || empresa.nombre}`;
    const ogDescripcion = `${productosAMostrar.length} producto${productosAMostrar.length === 1 ? '' : 's'} disponible${productosAMostrar.length === 1 ? '' : 's'} en ${empresa.marca || empresa.nombre}`;
    const ogImagen = (config && config.logoUrl) || (productosAMostrar.find((p) => p.fotos && p.fotos.length)?.fotos[0]) || null;
    const ogUrl = `${req.protocol}://${req.get('host')}${req.originalUrl}`;

    res.render(`catalogo-${vistaCatalogo(config)}`, {
      title: `Catálogo · ${empresa.marca || empresa.nombre}`,
      empresa, config, productos: productosAMostrar, categorias,
      simboloCatalogo: simboloMoneda(empresa.moneda),
      numeroWhatsapp: agente ? agente.numeroWhatsapp : null,
      filtro, hayFiltro, tokenSesion: sesion ? req.query.s : null,
      ogTitle, ogDescripcion, ogImagen, ogUrl,
    });
  } catch (err) { next(err); }
});
```

GET /catalogo/:slug/producto/:id (detalle)

Archivo: `server.js` · líneas 371–438

```
// Ficha de UN producto puntual: foto(s), descripcion completa, selector de
// variante con stock real, y "agregar al carrito" si vino con un token de
// sesion valido (lo manda el bot cuando ya hay interes real en ESE
// producto). Sin token se puede mirar pero no agregar nada.
app.get('/catalogo/:slug/producto/:id', async (req, res, next) => {
  try {
    const empresa = await prisma.empresa.findUnique({
      where: { slug: req.params.slug },
      include: { agentes: { take: 1, include: { config: true } } },
    });
    if (!empresa) return res.status(404).render('404', { title: 'Página no encontrada' });

    const producto = await prisma.producto.findFirst({
      where: { id: Number(req.params.id), empresaId: empresa.id, activo: true },
      include: { variantes: { where: { activa: true } }, categoria: true },
    });
    if (!producto) return res.status(404).render('404', { title: 'Producto no encontrado' });

    const agente = empresa.agentes[0];
    const config = agente ? agente.config : null;
    const sesion = verificarTokenSesion(req.query.s);
```

```
    const ogTitle = `${producto.nombre} · ${empresa.marca || empresa.nombre}`;
    const ogDescripcion = producto.descripcion || `${simboloMoneda(empresa.moneda)} ${Number(producto.precio).toFixed(2)}`;
    const ogImagen = (producto.fotos && producto.fotos[0]) || (config && config.logoUrl) || null;
    const ogUrl = `${req.protocol}://${req.get('host')}${req.originalUrl}`;

    const colores = coloresConFotoDeVariantes(producto);

    // Mismo filtro (categoria/color/talla/marca) que ya usa /catalogo/:slug:
    // si el cliente llego aca desde una vista filtrada, "Volver al catalogo"
    // tiene que devolverlo a ESA vista, no al catalogo entero (bug real
    // reportado: se perdia el filtro al entrar al detalle de un producto).
    const filtro = {
      categoria: (req.query.categoria || '').trim(),
      color: (req.query.color || '').trim(),
      talla: (req.query.talla || '').trim(),
      marca: (req.query.marca || '').trim(),
    };
    const hayFiltro = Boolean(filtro.categoria || filtro.color || filtro.talla || filtro.marca);

    // El link "volver a WhatsApp" lleva un mensaje ya armado: si el cliente
    // recien agrego esto al carrito, el mensaje refleja EXACTAMENTE que
    // eligio (variante y cantidad reales, no lo que dijo el modelo) - asi
    // llega a la conversacion real diciendo lo mismo que ya confirmo aca.
    const agregado = req.query.agregado === '1';
    const varianteAgregada = agregado && req.query.idVariante
      ? producto.variantes.find((v) => v.id === Number(req.query.idVariante))
      : null;
    const mensajeWhatsapp = mensajeWhatsappProducto(producto, {
      variante: varianteAgregada,
      cantidad: agregado ? Math.max(1, Number(req.query.cantidad) || 1) : null,
      accion: agregado ? 'Quiero' : 'Estoy viendo',
    });

    res.render('catalogo-producto', {
      title: `${producto.nombre} · ${empresa.marca || empresa.nombre}`,
      empresa, config, producto, colores,
      simboloCatalogo: simboloMoneda(empresa.moneda),
      numeroWhatsapp: agente ? agente.numeroWhatsapp : null,
      tokenSesion: sesion ? req.query.s : null,
      agregado,
      mensajeWhatsapp,
      filtro, hayFiltro,
      ogTitle, ogDescripcion, ogImagen, ogUrl,
    });
  } catch (err) { next(err); }
});
```

limiteCarritoWeb (límite de frecuencia)

Archivo: `server.js` *· líneas 444–459*

```
function limiteCarritoWeb(req, res, next) {
  const clave = `${req.ip}:${req.params.slug}`;
  const ahora = Date.now();
  const ventana = 60 * 1000;
  const registro = intentosCarritoWeb.get(clave) || { cuenta: 0, desde: ahora };
  if (ahora - registro.desde > ventana) {
    registro.cuenta = 0;
    registro.desde = ahora;
  }
  registro.cuenta += 1;
  intentosCarritoWeb.set(clave, registro);
  if (registro.cuenta > 20) {
    return res.status(429).send('Demasiados intentos, esperá un momento.');
  }
  next();
}
```

POST /catalogo/:slug/producto/:id/carrito (agregar desde la web)

Archivo: `server.js` *· líneas 461–533*

```
// Agregar al carrito desde la web: mismo carrito real que usa el bot
// (ClienteFinal.contexto.carrito), asi que cuando el cliente vuelve a
// escribir por WhatsApp ya lo tiene ahi.
app.post('/catalogo/:slug/producto/:id/carrito', limiteCarritoWeb, async (req, res, next) => {
  try {
    const sesion = verificarTokenSesion((req.body || {}).s || req.query.s);
    if (!sesion) return res.status(401).send('Este link ya venció. Volvé a pedirlo por WhatsApp.');

    const empresa = await prisma.empresa.findUnique({ where: { slug: req.params.slug } });
    if (!empresa || empresa.id !== sesion.empresaId) return res.status(404).send('No encontrado.');

    const producto = await prisma.producto.findFirst({
      where: { id: Number(req.params.id), empresaId: empresa.id, activo: true },
      include: { variantes: { where: { activa: true } } },
    });
    if (!producto) return res.status(404).send('Producto no encontrado.');

    const idVariante = (req.body || {}).idVariante ? Number(req.body.idVariante) : null;
    const cantidad = Math.max(1, Number((req.body || {}).cantidad) || 1);

    let variante = null;
    if (producto.variantes.length) {
      if (!idVariante) return res.status(400).send('Elegí una combinación (talla/color).');
      variante = producto.variantes.find((v) => v.id === idVariante);
      if (!variante) return res.status(400).send('Esa combinación no existe.');
      if (variante.stock < cantidad) return res.status(400).send('No hay stock suficiente de esa combinación.');
    } else if (producto.stock < cantidad) {
      return res.status(400).send('No hay stock suficiente.');
```

```
    }

    const clienteFinal = await prisma.clienteFinal.upsert({
      where: { empresaId_telefono: { empresaId: empresa.id, telefono: sesion.telefono } },
      update: {},
      create: { empresaId: empresa.id, telefono: sesion.telefono },
    });

    const precioUnitario = Number(variante ? (variante.precio ?? producto.precio) : producto.precio);
    const nombreItem = variante
      ? `${producto.nombre} (${Object.entries(variante.atributos || {}).map(([k, v]) => `${k}: ${v}`).join(', ')})`
      : producto.nombre;
    const itemsActuales = carritoDe(clienteFinal.contexto || {}, sesion.conversacionId);
    const itemsNuevos = agregarItem(itemsActuales, {
      productoId: producto.id, varianteId: variante ? variante.id : null, nombre: nombreItem, precio: precioUnitario, cantidad,
      agregadoEn: new Date().toISOString(),
    });

    await prisma.clienteFinal.update({
      where: { id: clienteFinal.id },
      data: {
        contexto: {
          ...guardarCarrito(clienteFinal.contexto || {}, sesion.conversacionId, itemsNuevos),
          // El bot no vio este cambio (paso afuera de una tool call): esta
          // marca le avisa en el proximo turno para que reaccione solo, sin
          // que el cliente tenga que repetirselo (ver generarRespuesta).
          carritoWebPendiente: { productoId: producto.id, varianteId: variante ? variante.id : null, agregadoEn: new
Date().toISOString() },
        },
        productoFavoritoId: producto.id,
        varianteFavoritaId: variante ? variante.id : null,
        estadoConversacion: 'INTENCION_DE_COMPRA',
      },
    });

    const paramsVuelta = new URLSearchParams({ s: req.body.s || req.query.s, agregado: '1', cantidad: String(cantidad) });
    if (variante) paramsVuelta.set('idVariante', String(variante.id));
    // El filtro (categoria/color/talla/marca) viaja en campos ocultos del
    // form: sin esto, agregar al carrito perdia el filtro con el que el
    // cliente habia llegado (mismo bug que "Volver al catalogo").
    for (const campo of ['categoria', 'color', 'talla', 'marca']) {
      if (req.body[campo]) paramsVuelta.set(campo, req.body[campo]);
    }
    res.redirect(`/catalogo/${req.params.slug}/producto/${req.params.id}?${paramsVuelta.toString()}`);
  } catch (err) { next(err); }
});
```

catalogo.js: mensajeWhatsappProducto (link "volver a WhatsApp")

Archivo: `lib/services/catalogo.js` *· líneas 752–775*

```
/**
 * Mensaje prellenado (formato WhatsApp) para el link "wa.me" del catalogo
 * web: la idea es que el cliente vuelva a WhatsApp con un texto ya armado y
 * prolijo en vez de tener que describir de nuevo lo que quiere. A proposito
 * no es un formato rigido de un solo caso - sirve tanto para "solo estoy
 * mirando" (sin variante ni cantidad) como para "esto es justo lo que quiero
 * comprar" (con variante y cantidad), y para cualquier boton nuevo que en el
 * futuro necesite mandar al cliente de vuelta a WhatsApp con contexto.
 *
 * @param {object} producto  al menos { nombre }.
 * @param {object} [opciones]
 * @param {object|null} [opciones.variante]  variante elegida, si aplica ({ atributos }).
 * @param {number|null} [opciones.cantidad]  si es mas de 1, se antepone "Nx".
 * @param {string} [opciones.accion]  verbo inicial ("Estoy viendo", "Quiero", etc.).
 */
function mensajeWhatsappProducto(producto, opciones = {}) {
  const { variante = null, cantidad = null, accion = 'Estoy viendo' } = opciones || {};
  const nombre = (producto && producto.nombre) || '';
  const prefijoCantidad = cantidad && cantidad > 1 ? `${cantidad}x ` : '';
  const detalle = variante && variante.atributos
    ? Object.entries(variante.atributos).map(([k, v]) => `${k}: ${v}`).join(', ')
    : '';
  return `${accion} *${prefijoCantidad}${nombre}*${detalle ? ` (${detalle})` : ''}`;
}
```

5.4 Cómo el bot se entera de lo agregado en la web

generarRespuesta: consumo de carritoWebPendiente (avisoCarritoWeb)

Archivo: `lib/services/agente.js` *· líneas 3197–3219*

```
  // El cliente agrego algo al carrito desde el catalogo web (fuera de una
  // tool call del bot, ver POST /catalogo/:slug/producto/:id/carrito en
  // server.js) - se le avisa al modelo como un hecho ya ocurrido para que
  // reaccione solo ("¡buena eleccion!") en vez de que el cliente tenga que
  // repetirselo. Se limpia la marca de una: si el proveedor de IA falla a
  // mitad de turno, se prefiere perder el aviso una vez a repetirlo en bucle.
  let avisoCarritoWeb = null;
  if (lead.contexto && lead.contexto.carritoWebPendiente) {
    const pendiente = lead.contexto.carritoWebPendiente;
    const productoAgregado = productos.find((p) => p.id === pendiente.productoId);
    if (productoAgregado) {
      avisoCarritoWeb = `RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente en este mensaje - paso hace un momento en el catalogo web): el cliente ACABA de agregar esto a su carrito desde la web:\n${fichaProducto(productoAgregado, lead, empresa.moneda)}\n\nTu texto AHORA: saludalo con entusiasmo natural por esa eleccion ("¡buena eleccion!" o similar), confirmale que ya quedo en su carrito, y preguntale si quiere ver algo mas. NUNCA le ofrezcas ni menciones otro producto por tu cuenta. NO le preguntes que producto eligio: ya lo sabes.`;
    }
```

```
    try {
      await prisma.clienteFinal.update({
        where: { id: clienteFinal.id },
        data: { contexto: { ...lead.contexto, carritoWebPendiente: null } },
      });
      lead = { ...lead, contexto: { ...lead.contexto, carritoWebPendiente: null } };
    } catch (err) {
      logErrorEtapa('memoria', telefonoCliente, err);
    }
  }
```

---

6. Pedidos y pagos

Datos del cliente: se extraen por código (`extraerFiltros`, ubicación nativa de WhatsApp en `interpretarMensajeEntrante`) y por la tool `actualizar_datos_lead`. El pedido se arma **siempre desde el carrito** y se crea en dos pasos protegidos: `confirmar_pedido` (muestra el resumen real) y, en un turno posterior, `crear_pedido`.

6.1 Captura de datos del cliente

datosDeActualizacionDeLead

Archivo: `lib/services/agente.js` · líneas 1883–1954

```
/**
 * Traduce los argumentos de `actualizar_datos_lead` al objeto que se guarda en
 * ClienteFinal. Puro a proposito: aca vive la memoria del cliente, que es lo
 * mas facil de romper sin darse cuenta.
 *
 * EL ORDEN IMPORTA. Si el cliente cambia de categoria, la limpieza deja las
 * listas depuradas en `datos`, y los bloques que acumulan (atributos,
 * descartados) tienen que construir SOBRE esa limpieza, no sobre el valor
 * viejo del cliente: un mismo mensaje puede cambiar de rubro y traer atributos
 * nuevos a la vez, y leyendo el valor viejo se resucitan los de la categoria
 * anterior.
 */
function datosDeActualizacionDeLead(args = {}, clienteFinal = {}, productos = []) {
  const datos = {};
  if (args.nombre) datos.nombre = String(args.nombre).slice(0, 120);
  if (args.categoriaInteres) {
    datos.categoriaInteres = String(args.categoriaInteres).slice(0, 120);
    // Resuelve tambien la categoria REAL (para saber sus atributos
    // obligatorios) con el mismo matching que ya usa extraerFiltros.
    const resuelto = productos.find((p) => p.categoria && coincideTexto(datos.categoriaInteres, p.categoria.nombre));
    datos.categoriaId = resuelto ? resuelto.categoria.id : null;

    // El cliente cambio de rubro ("mejor quiero zapatillas urbanas"): lo que
    // seguia siendo valido se mantiene (genero, talla, presupuesto), pero
    // todo lo que apuntaba a la categoria vieja deja de tener sentido y hay
    // que soltarlo. Sin esto el bot se quedaba enfocado en cerrar la venta
    // de un producto que el cliente ya descarto.
    if (datos.categoriaId !== clienteFinal.categoriaId) {
      Object.assign(datos, limpiezaPorCambioDeCategoria(clienteFinal, productos, datos.categoriaId));
    }
  }
  if (args.presupuesto) {
    datos.presupuesto = String(args.presupuesto).slice(0, 60);
    // El texto queda para repetirselo al cliente; el filtrado real usa el
    // rango numerico ya interpretado (nunca se reparsea en cada busqueda).
    const { min, max } = interpretarPresupuesto(datos.presupuesto);
    datos.presupuestoMin = min;
    datos.presupuestoMax = max;
```

```
  }
  if (args.cantidad) datos.cantidad = String(args.cantidad).slice(0, 20);
  if (args.marca) datos.marca = String(args.marca).slice(0, 60);
  if (args.talla) datos.talla = String(args.talla).slice(0, 30);
  if (args.color) datos.color = String(args.color).slice(0, 30);
  if (args.direccionEntrega) datos.direccionEntrega = String(args.direccionEntrega).slice(0, 300);
  if (args.tipoEntrega) datos.tipoEntrega = String(args.tipoEntrega).toUpperCase();
  if (args.formaPago) datos.formaPago = String(args.formaPago).toUpperCase();
  if (args.observaciones) datos.observaciones = String(args.observaciones).slice(0, 500);
  if (args.nivelInteres) datos.nivelInteres = String(args.nivelInteres).toUpperCase();
  if (args.estadoConversacion) datos.estadoConversacion = String(args.estadoConversacion).toUpperCase();
  if (args.productoFavoritoId) datos.productoFavoritoId = Number(args.productoFavoritoId);
  if (args.varianteFavoritaId) datos.varianteFavoritaId = Number(args.varianteFavoritaId);
  if (Array.isArray(args.productosDescartadosIds) && args.productosDescartadosIds.length) {
    const actuales = datos.productosDescartados ?? clienteFinal.productosDescartados ?? [];
    datos.productosDescartados = [...new Set([...actuales, ...args.productosDescartadosIds.map(Number)])];
  }
  if (args.atributosCategoria && typeof args.atributosCategoria === 'object') {
    const actuales = datos.atributosLead ?? clienteFinal.atributosLead ?? {};
    const nuevos = {};
    for (const [k, v] of Object.entries(args.atributosCategoria)) {
      if (!v) continue;
      // El modelo escribe el nombre del atributo a su gusto ("Genero" una vez,
      // "género" la siguiente). Si ya existe una clave que significa lo mismo,
      // se reusa esa: si no, el lead termina con dos entradas para el mismo
      // dato y el gate que busca una nunca ve la otra.
      const yaExiste = Object.keys(actuales).find((existente) => normalizarSimple(existente) === normalizarSimple(k));
      nuevos[yaExiste || String(k).slice(0, 60)] = String(v).slice(0, 120);
    }
    datos.atributosLead = { ...actuales, ...nuevos };
  }
  datos.estadoLead = 'EN_CONVERSACION';
  return datos;
}
```

ejecutarFuncion: actualizar_datos_lead

Archivo: `lib/services/agente.js` *· líneas 1982–1993*

```
  if (toolCall.name === 'actualizar_datos_lead') {
    const datos = datosDeActualizacionDeLead(args, clienteFinal, contexto);
    await prisma.clienteFinal.update({ where: { id: clienteFinal.id }, data: datos });
    // Si en este mismo turno la IA tambien llama mostrar_productos despues de
    // esto (muy comun: "guarda que pidio talla L" + "mostrale las opciones"),
    // ese siguiente tool call necesita ver la talla/color recien guardados
    // para poder filtrar la ficha - leadActual viene de un fetch de ANTES de
    // este turno, asi que se actualiza en memoria aca tambien.
    if (leadActual) Object.assign(leadActual, datos);
    logEtapa('memoria', telefonoCliente, { datos: Object.keys(datos) });
    return null;
  }
```

nombreValido

Archivo: `lib/services/agente.js` *· líneas 832–839*

```
function nombreValido(nombre) {
  if (!nombre) return false;
  const limpio = String(nombre).trim();
  if (limpio.length < 2) return false;
  if (!/[a-zA-ZñÑáéíóúÁÉÍÓÚ]{2,}/.test(limpio)) return false;
  if (/^(cliente|usuario|hola|si|no|nn|xx)$/i.test(limpio)) return false;
  return true;
}
```

primeraPreguntaDeCierre (qué dato falta para cerrar)

Archivo: `lib/services/agente.js` *· líneas 1446–1466*

```
/**
 * La UNICA cosa que falta para poder cerrar el pedido, en el mismo orden
 * que ya exige confirmar_pedido/crear_pedido - nunca inventa nada, solo lee
 * los datos reales del cliente. Se usa cuando hay que forzar una respuesta
 * de cierre en codigo (sin llamar de nuevo al modelo), asi la pregunta
 * siempre es honesta con lo que de verdad falta. Devuelve null si ya esta
 * todo listo para leerle el resumen.
 */
function primeraPreguntaDeCierre(cliente, config) {
  if (!nombreValido(cliente.nombre)) return '¿Me confirmás tu nombre para el pedido?';
  if (!cliente.tipoEntrega) {
    return config && config.direccionTienda
      ? '¿Lo querés a domicilio o pasás a retirarlo por la tienda?'
      : '¿Confirmamos que la entrega es a domicilio?';
  }
  if (cliente.tipoEntrega === 'DOMICILIO' && (!cliente.direccionEntrega || !cliente.ubicacionLat)) {
    return 'Para la entrega necesito tu ubicación real: compartila desde WhatsApp (el clip → Ubicación) o pegame el link de
Google Maps.';
  }
  if (!cliente.formaPago) return '¿Cómo preferís pagar?';
  return null;
}
```

6.2 Confirmar y crear el pedido, y envío del QR

ejecutarFuncion: confirmar_pedido / crear_pedido (incluye envío de QR/ubicación de tienda)

Archivo: `lib/services/agente.js` *· líneas 2485–2765*

```
  if (toolCall.name === 'confirmar_pedido' || toolCall.name === 'crear_pedido') {
    const accion = toolCall.name === 'crear_pedido' ? 'crees el pedido' : 'sigas';
    if (!nombreValido(clienteFinal.nombre)) {
      return `TODAVIA NO ${accion}: falta el nombre del cliente. Preguntaselo, guardalo con actualizar_datos_lead y recien ahi
volve a llamar.`;
    }
```

```
    if (!clienteFinal.tipoEntrega) {
      const opciones = config && config.direccionTienda
        ? 'a domicilio o retiro en la tienda'
        : 'a domicilio (esta tienda no tiene cargada la direccion de su local, asi que no ofrezcas retiro)';
      return `TODAVIA NO ${accion}: falta saber como quiere recibirlo. Preguntale si lo quiere ${opciones}, guardalo con actualizar_datos_lead (tipoEntrega) y volve a llamar.`;
    }
    if (clienteFinal.tipoEntrega === 'DOMICILIO' && !clienteFinal.direccionEntrega) {
      return `TODAVIA NO ${accion}: es una entrega a domicilio y falta la ubicacion. Pedile que comparta su UBICACION (el pin de WhatsApp) o que pegue el link de Google Maps - nunca le aceptes una direccion escrita a mano como unico dato. Guardala con actualizar_datos_lead y volve a llamar.`;
    }
    // No alcanza con tener texto en direccionEntrega: tiene que resolver a
    // coordenadas reales. Antes se aceptaba cualquier direccion escrita a
    // mano ("Av. Ballivian 1234"), que despues no se puede ubicar en un
    // mapa ni usar para coordinar la entrega. Si ya comparte el pin nativo
    // de WhatsApp, ubicacionLat/Lng ya estan guardadas y esto ni se corre.
    if (clienteFinal.tipoEntrega === 'DOMICILIO' && !clienteFinal.ubicacionLat) {
      const resuelto = await resolverCoordenadas(clienteFinal.direccionEntrega);
      if (resuelto) {
        await prisma.clienteFinal.update({
          where: { id: clienteFinal.id },
          data: { ubicacionLat: resuelto.lat, ubicacionLng: resuelto.lng },
        }).catch(() => {});
        clienteFinal.ubicacionLat = resuelto.lat;
        clienteFinal.ubicacionLng = resuelto.lng;
      } else {
        return `TODAVIA NO ${accion}: lo que guardaste como direccion ("${clienteFinal.direccionEntrega}") no es una ubicacion real que se pueda ubicar en un mapa. Pedile especificamente que comparta su UBICACION desde WhatsApp (el clip -> Ubicacion) o que pegue el link de Google Maps de donde esta - una direccion escrita a mano NO alcanza. Guardala con actualizar_datos_lead y volve a llamar.`;
      }
    }
    if (clienteFinal.tipoEntrega === 'RECOJO' && !(config && config.direccionTienda)) {
      return `TODAVIA NO ${accion}: el cliente quiere retirar en la tienda, pero este negocio no tiene cargada la direccion de su local. NUNCA inventes una direccion: avisale que un asesor le va a pasar la ubicacion exacta, u ofrecele entrega a domicilio.`;
    }
    const metodosHabilitados = {
      QR: Boolean(config && config.aceptaQr),
      EFECTIVO: !config || config.aceptaEfectivo !== false,
      TARJETA: Boolean(config && config.aceptaTarjeta),
    };
    if (!clienteFinal.formaPago) {
      return `TODAVIA NO ${accion}: falta la forma de pago. Preguntasela (solo entre las formas de pago reales de esta tienda, ver bloque de arriba), guardala con actualizar_datos_lead y recien ahi volve a llamar.`;
    }
    if (!metodosHabilitados[clienteFinal.formaPago]) {
      const disponibles = Object.entries(metodosHabilitados).filter(([, ok]) => ok).map(([m]) => m.toLowerCase()).join(', ');
      return `TODAVIA NO ${accion}: esta tienda no tiene habilitada esa forma de pago. Las formas de pago reales son: ${disponibles || 'ninguna configurada, avisale que un asesor lo va a contactar para coordinar el pago'}. Preguntale de nuevo
```

```
entre esas, guardala con actualizar_datos_lead y recien ahi volve a llamar.`;
    }

    // El carrito es la UNICA fuente de verdad, siempre - nunca lo que el
    // modelo diga que hay que cobrar. Antes se le permitia mandar su propia
    // lista de items "por si acaso", y esa via de escape es justo lo que
    // volvio a perder un producto real: el modelo mando solo lo ultimo que
    // habia conversado (la camisa) y el carrito real tenia ademas unas
    // zapatillas, que desaparecieron del pedido sin ningun aviso. El
    // carrito ya es confiable (se arma con agregar_al_carrito y desde la
    // web, nunca a mano por el modelo), asi que no hace falta ni es seguro
    // dejar que lo reemplace.
    const itemsPedidos = itemsParaPedido(carritoDe(clienteFinal.contexto || {}, conversacionId));
    if (!itemsPedidos.length) {
      return 'El carrito esta vacio: no hay nada que confirmar. Invitalo a elegir algo de lo que le mostraste.';
    }
    // Aparte de `items` (que va tal cual a Prisma en crear_pedido, sin
    // campos extra): un mapa productoId:varianteId -> agregadoEn, solo para
    // poder anotar en el resumen de confirmar_pedido que items estan
    // esperando de una sesion anterior.
    const agregadoEnPorLinea = new Map(itemsPedidos.map((i) => [`${i.idProducto}:${i.idVariante || 0}`, i.agregadoEn]));

    const items = [];
    let total = 0;
    for (const item of itemsPedidos) {
      const producto = await prisma.producto.findFirst({
        where: { id: Number(item.idProducto), empresaId },
        include: { variantes: { where: { activa: true } } },
      });
      if (!producto) continue;
      const cantidad = Math.max(1, Number(item.cantidad) || 1);

      let variante = null;
      if (producto.variantes.length) {
        if (!item.idVariante) {
          const opciones = producto.variantes
            .map((v) => `- [Variante ID ${v.id}] ${formatearAtributosVariante(v.atributos)} - Precio:
${formatearPrecio(v.precio ?? producto.precio, moneda)} - Stock: ${v.stock}`)
            .join('\n');
          return `TODAVIA NO ${accion} de "${producto.nombre}": ese producto tiene variantes y falta saber cual eligio el
cliente. Preguntaselo y volve a llamar con el idVariante correcto entre estas opciones reales:\n${opciones}`;
        }
        variante = producto.variantes.find((v) => v.id === Number(item.idVariante));
        if (!variante) return `No encontre esa variante de "${producto.nombre}". Ofrece una de las opciones reales
disponibles.`;
        if (variante.stock < cantidad) {
          return `No hay stock suficiente de "${producto.nombre}" (${formatearAtributosVariante(variante.atributos)}) - quedan
${variante.stock}. Ofrece la cantidad disponible u otra variante.`;
        }
      } else if (producto.stock < cantidad) {
        return `No hay stock suficiente de ${producto.nombre} (quedan ${producto.stock}). Ofrece la cantidad disponible o un
```

```
producto alternativo.`;
      }

      const precioUnitario = Number(variante ? (variante.precio ?? producto.precio) : producto.precio);
      total += precioUnitario * cantidad;
      items.push({
        productoId: producto.id,
        varianteId: variante ? variante.id : null,
        nombre: variante ? `${producto.nombre} (${formatearAtributosVariante(variante.atributos)})` : producto.nombre,
        precio: precioUnitario,
        cantidad,
      });
    }
    if (!items.length) return 'No encontre esos productos para armar el pedido.';

    // Firma de lo que se esta por comprar. crear_pedido exige que sea la
    // MISMA que se le confirmo al cliente: asi un pedido nunca se crea sin
    // que la persona haya visto y aceptado el resumen exacto.
    const firmaItems = items.map((i) => `${i.productoId}:${i.varianteId || 0}:${i.cantidad}`).sort().join('|');

    if (toolCall.name === 'confirmar_pedido') {
      // Bug real reportado con capturas de WhatsApp: el cliente confirmaba
      // el resumen ("si") y el modelo, en vez de llamar a crear_pedido,
      // volvia a llamar a confirmar_pedido - que re-mostraba el MISMO
      // resumen y pisaba resumenConfirmadoEnTurno con el turno actual. Eso
      // dejaba a crear_pedido bloqueado para siempre (su chequeo de "mismo
      // turno" nunca dejaba de cumplirse) y el bot quedaba en un loop
      // infinito reshowing el resumen, con el cliente contestando "ok"/
      // "si"/"?" sin que nada avanzara. Si este MISMO pedido (misma firma)
      // ya se confirmo en un turno ANTERIOR, no se vuelve a mostrar: se le
      // dice al modelo que llame crear_pedido directo.
      const contextoPrevio = clienteFinal.contexto || {};
      if (contextoPrevio.resumenConfirmado === firmaItems
        && contextoPrevio.resumenConfirmadoEnTurno != null
        && contextoPrevio.resumenConfirmadoEnTurno !== helpers.turno) {
        return 'TOOL_FAILED: este pedido (mismos items, mismo total) YA fue confirmado por el cliente en un mensaje anterior.
NO vuelvas a mostrar el resumen ni le preguntes de nuevo: llama AHORA a crear_pedido con estos mismos items.';
      }
      await prisma.clienteFinal.update({
        where: { id: clienteFinal.id },
        data: {
          // resumenConfirmadoEnTurno (mismo patron que relajadoPreguntado.turno
          // mas arriba): guarda EN QUE TURNO se armo este resumen, no solo que
          // se armo. crear_pedido exige que sea un turno DISTINTO - si no,
          // nada impide que el modelo llame confirmar_pedido y crear_pedido
          // seguidos en la misma vuelta/turno, sin que el resumen real
          // (items, precios, total) le haya llegado al cliente todavia como
          // mensaje propio. Bug real reportado: el cliente contesto "si" a
          // "necesito tu nombre", y en ESE MISMO turno el pedido termino
          // "confirmado" sin que nunca se le mostrara el desglose.
          contexto: { ...(clienteFinal.contexto || {}), resumenConfirmado: firmaItems, resumenConfirmadoEnTurno: helpers.turno
```

```
},
          estadoConversacion: 'DATOS_DE_PEDIDO',
        },
      });
      // Items agregados hace rato (de una sesion que no se llego a cerrar)
      // se marcan aparte: el cliente nunca deberia sorprenderse con algo
      // que ya se habia olvidado mezclado en silencio en su pedido nuevo.
      const UMBRAL_ITEM_VIEJO_MS = 2 * 60 * 60 * 1000;
      const ahora = Date.now();
      const esItemViejo = (i) => {
        const agregadoEn = agregadoEnPorLinea.get(`${i.productoId}:${i.varianteId || 0}`);
        return agregadoEn && (ahora - new Date(agregadoEn).getTime()) > UMBRAL_ITEM_VIEJO_MS;
      };
      const hayItemsViejos = items.some(esItemViejo);
      const lineas = items.map((i) => `- ${i.cantidad}x ${i.nombre} — ${formatearPrecio(i.precio, moneda)} c/u${esItemViejo(i)
? ' (ya lo tenia en el carrito de antes)' : ''}`).join('\n');
      const entrega = clienteFinal.tipoEntrega === 'RECOJO'
        ? `Retira en la tienda: ${config.direccionTienda}`
        : `Entrega a domicilio: ${clienteFinal.direccionEntrega}`;
      logEtapa('resumen_confirmado', telefonoCliente, { items: items.length, total, tipoEntrega: clienteFinal.tipoEntrega,
hayItemsViejos });
      return `TOOL_SUCCESS. Resumen REAL del pedido (todos estos datos salen de la base, usalos tal cual, no los cambies ni
redondees):
${lineas}
Total: ${formatearPrecio(total, moneda)}
A nombre de: ${clienteFinal.nombre}
${entrega}
Forma de pago: ${clienteFinal.formaPago.toLowerCase()}

Tu texto ahora: leele este resumen al cliente de forma clara y ordenada y preguntale si esta todo correcto.${hayItemsViejos ? '
Los items marcados "(ya lo tenia en el carrito de antes)" son de una compra que el cliente no llego a cerrar hace un rato -
mencionaselo con naturalidad ("ademas segui con lo que tenias de antes: X"), por si ya no los quiere.' : ''} NO llames a
crear_pedido en este turno: espera a que el cliente confirme que si.`;
    }

    const contextoActual = clienteFinal.contexto || {};
    if (contextoActual.resumenConfirmado !== firmaItems) {
      return 'TODAVIA NO crees el pedido: el cliente no confirmo ESTE pedido exacto todavia (o cambio algo desde el resumen
anterior). Llama primero a confirmar_pedido con estos mismos items, leele el resumen, y crea el pedido recien cuando te diga
que esta todo bien.';
    }
    if (contextoActual.resumenConfirmadoEnTurno === helpers.turno) {
      // El resumen se armo EN ESTE MISMO turno (misma vuelta u otra vuelta
      // del mismo mensaje): el cliente todavia no tuvo la oportunidad real
      // de leerlo como un mensaje propio y contestar. Se bloquea aunque los
      // items coincidan exacto - el resumen tiene que llegar primero como
      // respuesta de ESTE turno, y recien el PROXIMO mensaje del cliente
      // puede crear el pedido.
      return 'TODAVIA NO crees el pedido en este mismo turno: el resumen recien se armo ahora. Tu texto de este turno tiene que
ser el resumen (leeselo tal cual), y espera al proximo mensaje del cliente confirmando que esta bien antes de llamar a
crear_pedido.';
```

```
    }

    // El cliente muchas veces NO comparte su ubicacion en vivo (eso es lo
    // unico que llenaba ubicacionLat/Lng antes): pega el link de Google
    // Maps como texto en direccionEntrega. Si todavia no hay coordenadas
    // guardadas, se intenta resolver ese link ahora - asi el pedido puede
    // tener un mapa real sin depender de que comparta el pin nativo. Nunca
    // inventa nada: si no se puede resolver, sigue sin coordenadas y el
    // pedido queda igual que antes (solo la direccion en texto).
    let ubicacionLat = clienteFinal.ubicacionLat;
    let ubicacionLng = clienteFinal.ubicacionLng;
    if (clienteFinal.tipoEntrega !== 'RECOJO' && !ubicacionLat && clienteFinal.direccionEntrega) {
      const resuelto = await resolverCoordenadas(clienteFinal.direccionEntrega);
      if (resuelto) {
        ubicacionLat = resuelto.lat;
        ubicacionLng = resuelto.lng;
        // Se guarda tambien en el cliente para que una proxima compra ya
        // la tenga sin volver a resolver el link.
        await prisma.clienteFinal.update({ where: { id: clienteFinal.id }, data: { ubicacionLat, ubicacionLng } }).catch(() =>
{});
      }
    }

    const pedido = await prisma.$transaction(async (tx) => {
      const creado = await tx.pedido.create({
        data: {
          empresaId,
          clienteId: clienteFinal.id,
          conversacionId: conversacionId || null,
          total,
          tipoEntrega: clienteFinal.tipoEntrega,
          direccionEntrega: clienteFinal.tipoEntrega === 'RECOJO' ? null : clienteFinal.direccionEntrega,
          entregaLat: clienteFinal.tipoEntrega === 'RECOJO' ? null : ubicacionLat,
          entregaLng: clienteFinal.tipoEntrega === 'RECOJO' ? null : ubicacionLng,
          notas: clienteFinal.observaciones,
          formaPago: clienteFinal.formaPago,
          items: { create: items },
        },
      });
      for (const item of items) {
        if (item.varianteId) {
          await tx.variante.update({ where: { id: item.varianteId }, data: { stock: { decrement: item.cantidad } } });
        } else {
          await tx.producto.update({ where: { id: item.productoId }, data: { stock: { decrement: item.cantidad } } });
        }
      }
      await tx.clienteFinal.update({
        where: { id: clienteFinal.id },
        data: {
          estadoLead: 'PEDIDO_CREADO',
          estadoConversacion: 'PEDIDO_COMPLETADO',
```

```
          contexto: contextoSinCarrito({ ...(clienteFinal.contexto || {}), resumenConfirmado: null }),
        },
      });
      return creado;
    });
    logEtapa('pedido_creado', telefonoCliente, {
      pedidoId: pedido.id, items: items.length, total, moneda,
      tipoEntrega: clienteFinal.tipoEntrega,
      selected_variant: items.map((i) => i.varianteId).filter(Boolean),
    });

    const resumen = items.map((i) => `${i.cantidad}x ${i.nombre}`).join(', ');

    // Retiro en tienda: se manda la ubicacion REAL que cargo el negocio.
    // Igual que con el QR, solo se puede afirmar que se mando si el envio
    // por WhatsApp devolvio exito de verdad.
    let avisoEntrega = '';
    if (clienteFinal.tipoEntrega === 'RECOJO' && config && config.direccionTienda) {
      const textoUbicacion = `📍 Nos encontrás en: ${config.direccionTienda}${config.tiendaLat && config.tiendaLng ?
`\nhttps://www.google.com/maps?q=${config.tiendaLat},${config.tiendaLng}` : ''}`;
      if (conexion && conexion.estado === 'CONECTADO') {
        const envio = await wa.enviarTexto(conexion, telefonoCliente, textoUbicacion);
        avisoEntrega = envio.ok
          ? ` La ubicacion real de la tienda SI se le mando por WhatsApp (TOOL_SUCCESS): "${config.direccionTienda}".`
          : ' La ubicacion de la tienda NO se pudo mandar (fallo el envio): no le digas que se la mandaste, avisale que se la
vas a pasar en un momento.';
      } else {
        avisoEntrega = ` [Modo de prueba] La direccion de la tienda es: ${config.direccionTienda}.`;
      }
    }

    // Si eligio QR, se manda la imagen real que subio el negocio (estatica,
    // no una pasarela de pago). Nunca se afirma que se mando si no hubo
    // TOOL_SUCCESS real: si falla o no hay imagen cargada, se le avisa tal
    // cual al modelo para que sea honesto con el cliente.
    let avisoQr = '';
    if (clienteFinal.formaPago === 'QR') {
      if (!config || !config.qrCobroUrl) {
        avisoQr = ' No se pudo mandar el QR porque la tienda todavia no cargo la imagen: avisale con naturalidad que un asesor
se lo va a mandar.';
      } else if (conexion && conexion.estado === 'CONECTADO') {
        const envio = await wa.enviarImagen(conexion, telefonoCliente, config.qrCobroUrl, 'Este es el QR para el pago 📲');
        avisoQr = envio.ok
          ? ' El QR de pago SI se mando por WhatsApp (TOOL_SUCCESS): podes decirle que te fijes que lo recibio.'
          : ' El QR de pago NO se pudo mandar (fallo el envio): no le digas que se lo mandaste, avisale que un asesor se lo va
a mandar.';
      } else if (fotosParaMostrar) {
        fotosParaMostrar.push({ url: config.qrCobroUrl, caption: 'QR de pago' });
        avisoQr = ' [Modo de prueba] El QR de pago se muestra como imagen en el chat de prueba.';
      }
    }
```

```
    return `TOOL_SUCCESS: pedido #${pedido.id} creado con: ${resumen}. Total ${formatearPrecio(total,
moneda)}.${avisoEntrega}${avisoQr} Tu texto ahora: confirma el pedido al cliente con calidez, y explica que un asesor
coordinara ${clienteFinal.tipoEntrega === 'RECOJO' ? 'el retiro' : 'la entrega'}. Si tiene dudas de pago, ofrece
derivar_a_asesor.`;
  }
```

6.3 Cierre forzado por código cuando la IA falla (crear pedido / mostrar resumen)

generarRespuesta: cierre forzado (pedido_creado_forzado_preemptivo / resumen_mostrado_forzado_preemptivo)

Archivo: `lib/services/agente.js` *· líneas 3922–3995*

```
            // Bug real reportado: el cliente ya habia visto el resumen y
            // contesto "si, todo" / "si" - pero la IA fallo las 3 vueltas sin
            // llamar a crear_pedido (a veces por texto de mas, a veces por
            // llamar de nuevo a confirmar_pedido en vez de crear_pedido), y
            // este mismo bloque le volvia a preguntar "¿confirmas que esta
            // todo bien?" IGUAL, como si el cliente no hubiera contestado
            // nada. Antes de repetir la pregunta: si no falta ningun dato
            // (primeraPreguntaDeCierre da null) se intenta CREAR el pedido
            // por codigo. Esto nunca depende de interpretar lo que dijo el
            // cliente en este mensaje - crear_pedido ya exige por su cuenta
            // que resumenConfirmado coincida EXACTO con el carrito actual
            // (ver el handler, unas lineas mas arriba), asi que si el
            // resumen nunca se confirmo de verdad, esto simplemente falla
            // silencioso y cae a la pregunta de siempre, sin ningun riesgo
            // de crear un pedido que el cliente no confirmo.
            const preguntaPendiente = primeraPreguntaDeCierre(leadActual, agente.config);
            if (!preguntaPendiente) {
              helpers.leadActual = leadActual;
              const resultadoCierre = await ejecutarFuncion({ name: 'crear_pedido', arguments: {} }, productos, helpers);
              if (typeof resultadoCierre === 'string' && resultadoCierre.startsWith('TOOL_SUCCESS')) {
                logEtapa('pedido_creado_forzado_preemptivo', telefonoCliente, { vuelta });
                return {
                  ok: true,
                  demo: false,
                  proveedor,
                  modelo,
                  respuesta: '¡Perfecto! Tu pedido quedó confirmado ✅ Ya lo estamos preparando. Cualquier duda, quedo
atento.',
                  fotos: fotosParaMostrar,
                };
              }
              // TOOL_FAILED (el resumen todavia no se confirmo de verdad, o
              // cambio algo desde entonces): antes de caer a la pregunta
              // generica de siempre, se intenta mostrar el resumen REAL
              // (items, precios, total) por codigo. Bug real reportado: sin
              // esto, un cliente podia llegar a "confirmado" sin haber visto
              // NUNCA el desglose - la IA fallaba las vueltas, caia a esta
```

```
              // pregunta vaga ("¿confirmas que esta todo bien?") sin decir
              // que era lo que habia que confirmar. El resumen mostrado a
              // partir de aca cuenta como "armado en este turno"
              // (resumenConfirmadoEnTurno queda en el turno actual, ver el
              // handler de confirmar_pedido) - recien el proximo mensaje del
              // cliente puede terminar de crear el pedido.
              const resultadoResumen = await ejecutarFuncion({ name: 'confirmar_pedido', arguments: {} }, productos, helpers);
              if (typeof resultadoResumen === 'string' && resultadoResumen.startsWith('TOOL_SUCCESS')) {
                const bloqueDatos = resultadoResumen
                  .replace(/^TOOL_SUCCESS\.[^\n]*\n/, '')
                  .split('\n\nTu texto ahora:')[0]
                  .trim();
                logEtapa('resumen_mostrado_forzado_preemptivo', telefonoCliente, { vuelta });
                return {
                  ok: true,
                  demo: false,
                  proveedor,
                  modelo,
                  respuesta: `Este es el resumen de tu pedido:\n${bloqueDatos}\n\n¿Confirmás que está todo bien para
procesarlo?`,
                  fotos: fotosParaMostrar,
                };
              }
              // TOOL_FAILED de confirmar_pedido tambien (ej. carrito vacio,
              // stock insuficiente detectado justo ahora) -> cae a la
              // pregunta de siempre, es un caso real que no se puede resolver
              // por codigo aca.
            }

            logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true, cierreForzado: true
});
            return {
              ok: true,
              demo: false,
              proveedor,
              modelo,
              respuesta: preguntaPendiente || 'Ya tengo todo listo para tu pedido — ¿confirmás que está todo bien para
procesarlo?',
              fotos: fotosParaMostrar,
            };
          }
```

6.4 Estados del pedido y panel

pedidos.js completo

Archivo: `lib/services/pedidos.js` *· líneas 1–32*

```
// Reglas de transicion de estado de un pedido. Logica pura (sin DB) para que
// sea facil de testear - la aplicacion real (incluido el restock) vive en el
// handler de server.js.
```

```
const TRANSICIONES_VALIDAS = {
  NUEVO: ['CONFIRMADO', 'CANCELADO'],
  CONFIRMADO: ['ENTREGADO', 'CANCELADO'],
  ENTREGADO: [], // estado final: no se puede reabrir un pedido ya entregado
  CANCELADO: [], // estado final: no se puede reabrir un pedido ya cancelado
};

function transicionValida(estadoActual, estadoNuevo) {
  return (TRANSICIONES_VALIDAS[estadoActual] || []).includes(estadoNuevo);
}

// El stock se descuenta al CREAR el pedido (ver crear_pedido en agente.js),
// no al entregarlo - asi que solo hay que devolverlo si el pedido se
// cancela, y solo una vez (nunca si ya estaba cancelado).
function requiereRestock(estadoActual, estadoNuevo) {
  return estadoNuevo === 'CANCELADO' && estadoActual !== 'CANCELADO';
}

const VENTANA_24H_MS = 24 * 60 * 60 * 1000;

// Meta solo deja mandar un mensaje de texto libre (no plantilla aprobada) si
// el cliente escribio hace menos de 24 horas. Se usa Conversacion.ultimoMensajeAt
// como referencia de "cuando escribio por ultima vez".
function dentroDeVentana24h(ultimoMensajeAt, ahora = new Date()) {
  if (!ultimoMensajeAt) return false;
  return ahora.getTime() - new Date(ultimoMensajeAt).getTime() < VENTANA_24H_MS;
}

module.exports = { TRANSICIONES_VALIDAS, transicionValida, requiereRestock, dentroDeVentana24h };
```

POST /panel/pedidos/:id/estado

Archivo: `server.js` · líneas 1253–1298

```
// Cambia el estado de un pedido (Confirmar / Marcar entregado / Cancelar).
// Al cancelar se devuelve el stock que se habia descontado al crear el
// pedido (ver crear_pedido en lib/services/agente.js) - confirmar/entregar
// NO tocan el stock, ya se reservo al crearse.
app.post('/panel/pedidos/:id/estado', requireCliente, async (req, res, next) => {
  // La lista y el detalle comparten esta misma accion: cada form manda de
  // vuelta a donde estaba parado el usuario (hidden input "volver"), asi no
  // lo saca del detalle del pedido que esta gestionando.
  const volverA = req.body.volver === 'detalle' ? `/panel/pedidos/${req.params.id}` : '/panel/pedidos';
  try {
    const empresaId = req.session.empresaId;
    const pedido = await prisma.pedido.findFirst({
      where: { id: Number(req.params.id), empresaId },
      include: { items: true },
    });
    if (!pedido) return res.redirect('/panel/pedidos');

    const estadoNuevo = String(req.body.estado || '').toUpperCase();
```

```
    if (!transicionValida(pedido.estado, estadoNuevo)) {
      return res.redirect(`${volverA}?err=` + encodeURIComponent('Ese cambio de estado no es válido.'));
    }

    await prisma.$transaction(async (tx) => {
      if (requiereRestock(pedido.estado, estadoNuevo)) {
        for (const item of pedido.items) {
          if (item.varianteId) {
            await tx.variante.update({ where: { id: item.varianteId }, data: { stock: { increment: item.cantidad } } });
          } else if (item.productoId) {
            await tx.producto.update({ where: { id: item.productoId }, data: { stock: { increment: item.cantidad } } });
          }
        }
      }
      await tx.pedido.update({ where: { id: pedido.id }, data: { estado: estadoNuevo } });
    });

    let mensajeOk = 'Pedido actualizado.';
    if (TEXTOS_CAMBIO_DE_ESTADO[estadoNuevo]) {
      const ETIQUETAS_ESTADO = { CONFIRMADO: 'confirmado', ENTREGADO: 'marcado como entregado', CANCELADO: 'cancelado' };
      const notificado = await notificarCambioDeEstadoPedido(empresaId, pedido, estadoNuevo);
      const etiqueta = ETIQUETAS_ESTADO[estadoNuevo];
      mensajeOk = notificado ? `Pedido ${etiqueta} y cliente avisado por WhatsApp.` : `Pedido ${etiqueta} (no se pudo avisar
por WhatsApp: sin conexión o fuera de la ventana de 24hs).`;
    }

    res.redirect(`${volverA}?ok=` + encodeURIComponent(mensajeOk));
  } catch (err) { next(err); }
});
```

6.5 Pagos de la plataforma (paquetes/planes) — confirmación manual desde el admin

POST /admin/pagos/:id/confirmar

Archivo: `server.js` · líneas 3867–3906

```
// Confirmar un pago pendiente: activa lo que corresponda segun el tipo.
// Antes esta pantalla era solo de lectura, un pago quedaba "PENDIENTE" para
// siempre y jamas pasaba nada (los paquetes ya se acreditaban solos al
// comprar, sin relacion real con si se pagaba o no).
app.post('/admin/pagos/:id/confirmar', requireAuth, async (req, res, next) => {
  try {
    const pago = await prisma.pago.findUnique({ where: { id: Number(req.params.id) } });
    if (!pago || pago.estado !== 'PENDIENTE') return res.redirect('/admin/pagos');

    await prisma.pago.update({ where: { id: pago.id }, data: { estado: 'CONFIRMADO' } });

    if (pago.tipo === 'PRIMER_PAGO' || pago.tipo === 'MENSUALIDAD') {
      const suscripcion = await prisma.suscripcion.findUnique({ where: { empresaId: pago.empresaId } });
      if (suscripcion) {
        await prisma.suscripcion.update({
```

```
          where: { id: suscripcion.id },
          data: {
            estado: 'ACTIVA',
            periodoInicio: new Date(),
            periodoFin: new Date(Date.now() + 30 * 24 * 60 * 60 * 1000),
          },
        });
      }
      await notif.crear(pago.empresaId, {
        titulo: 'Tu pago fue confirmado',
        mensaje: 'Confirmamos tu pago: tu plan ya está activo por 30 días más.',
        tipo: 'PAGO_CONFIRMADO',
      });
    } else if (pago.tipo === 'PAQUETE' && pago.compraPaqueteId) {
      await prisma.compraPaquete.update({ where: { id: pago.compraPaqueteId }, data: { estado: 'ACTIVA' } });
      await notif.crear(pago.empresaId, {
        titulo: 'Tu paquete ya está disponible',
        mensaje: 'Confirmamos tu pago: las conversaciones del paquete ya se sumaron a tu saldo.',
        tipo: 'PAGO_CONFIRMADO',
      });
    }

    res.redirect('/admin/pagos?ok=' + encodeURIComponent('Pago confirmado.'));
  } catch (err) { next(err); }
});
```

POST /admin/pagos/:id/rechazar

Archivo: `server.js` · líneas 3908–3929

```
// Rechazar un pago pendiente: si era de un paquete, revoca esa compra (nunca
// llego a dar saldo real gracias a que se crea en PENDIENTE).
app.post('/admin/pagos/:id/rechazar', requireAuth, async (req, res, next) => {
  try {
    const pago = await prisma.pago.findUnique({ where: { id: Number(req.params.id) } });
    if (!pago || pago.estado !== 'PENDIENTE') return res.redirect('/admin/pagos');

    await prisma.pago.update({ where: { id: pago.id }, data: { estado: 'FALLIDO' } });

    if (pago.tipo === 'PAQUETE' && pago.compraPaqueteId) {
      await prisma.compraPaquete.update({ where: { id: pago.compraPaqueteId }, data: { estado: 'RECHAZADA' } });
    }

    await notif.crear(pago.empresaId, {
      titulo: 'No pudimos confirmar tu pago',
      mensaje: 'Tu pago no pudo confirmarse. Contáctanos para resolverlo y activar tu servicio.',
      tipo: 'PAGO_RECHAZADO',
    });

    res.redirect('/admin/pagos?ok=' + encodeURIComponent('Pago marcado como fallido.'));
```

```
  } catch (err) { next(err); }
});
```

---

7. Bucle de llamadas: vueltas, detectores, reintentos y respuestas sin IA

`generarRespuesta` completa, en cinco tramos contiguos (sin omitir líneas). Los tramos 2 a 5 son el bucle; el tramo 1 contiene las respuestas del backend que cuestan 0 llamadas. Algunos fragmentos de estos tramos ya aparecen citados en las secciones 3, 5 y 6; se repiten acá para que la función pueda leerse completa y de corrido.

7.1 Tramo 1 — preparación del turno y respuestas del backend sin IA

generarRespuesta (tramo 1: carga de datos, memoria del lead, historial, modelo, preemptivos sin IA)

Archivo: `lib/services/agente.js` · líneas 2987–3481

```
/**
 * Genera la respuesta del agente para un mensaje del cliente, ejecutando el
 * loop completo de function-calling (hasta 3 vueltas): el modelo llama
 * tools, el sistema las ejecuta de verdad (actualiza el cliente, manda fotos
 * reales por WhatsApp, crea el pedido validando stock) y recien entonces el
 * modelo redacta el mensaje final.
 *
 * Si algo falla de verdad (no se pudo consultar el catalogo, o el proveedor
 * de IA no respondio), NUNCA se cae a una respuesta que "adivine"
 * disponibilidad: se devuelve un mensaje honesto de error tecnico. Eso es
 * distinto del modo demo (sin API key configurada), que es un modo de
 * prueba conocido, no una falla.
 *
 * @param {number} agenteId
 * @param {string} telefonoCliente
 * @param {Array<{rol:string, contenido:string}>} historial  mensajes previos (de Mensaje)
 * @param {string} mensajeCliente
 * @param {number} [conversacionId]
 * @param {{ llamarInyectado?: Function }} [opciones]  solo para tests: permite inyectar un "llamar" falso sin pegarle a un
proveedor real.
 */
async function generarRespuesta(agenteId, telefonoCliente, historial, mensajeCliente, conversacionId, opciones = {}) {
  logEtapa('mensaje_recibido', telefonoCliente, { agenteId, mensaje: mensajeCliente.slice(0, 200), conversacionId });

  let agente;
  try {
    agente = await prisma.agente.findUnique({
      where: { id: agenteId },
      include: {
        config: true,
        conexion: true,
        empresa: {
          include: {
            suscripcion: { include: { plan: true } },
            productos: {
              where: { activo: true },
              orderBy: { nombre: 'asc' },
              take: 200,
              // "padre" trae el rubro al que pertenece la categoria del
```

```
              // producto: con eso se arma el menu de dos niveles sin una
              // consulta aparte (ver arbolDeCategorias en catalogo.js).
              // padre.atributos: sin esto, categoriaDelLead devolviendo el
              // RUBRO (via p.categoria.padre, cuando el lead esta parado en
              // el rubro y el producto es de una subcategoria) traia un
              // objeto sin atributos - atributosFaltantes nunca podia ver un
              // atributo obligatorio configurado en el rubro mismo. Bug real
              // encontrado escribiendo el test de este fix.
              include: { variantes: { where: { activa: true }, orderBy: { id: 'asc' } }, categoria: { include: { atributos:
true, padre: { include: { atributos: true } } } } },
            },
          },
        },
      },
    });
  } catch (err) {
    logErrorEtapa('base_de_datos', telefonoCliente, err);
    return { ok: true, demo: false, error: true, proveedor: 'error', respuesta: respuestaErrorTecnico() };
  }
  if (!agente) return { ok: false, respuesta: 'Agente no encontrado.', demo: true, proveedor: 'demo' };

  const { empresa } = agente;
  const productos = empresa.productos;

  let clienteFinal;
  try {
    clienteFinal = await prisma.clienteFinal.upsert({
      where: { empresaId_telefono: { empresaId: empresa.id, telefono: telefonoCliente } },
      update: {},
      create: { empresaId: empresa.id, telefono: telefonoCliente },
    });
  } catch (err) {
    logErrorEtapa('base_de_datos', telefonoCliente, err);
    return { ok: true, demo: false, error: true, proveedor: 'error', respuesta: respuestaErrorTecnico() };
  }

  // Si el cliente respondio con un numero suelto ("2") a algo que el agente
  // ofrecio, se resuelve a que opcion real corresponde ANTES de extraer
  // filtros (si no, "2" no matchea contra ninguna categoria real).
  const ultimoAgente = [...historial].reverse().find((h) => h.rol === 'AGENTE');
  const opcionElegida = resolverSeleccionMenu(mensajeCliente, ultimoAgente && ultimoAgente.contenido);
  const textoParaExtraccion = opcionElegida || mensajeCliente;

  // Si paso MUCHISIMO tiempo sin escribir, la intencion de compra (categoria,
  // talla, color, marca, presupuesto, favorito) tambien se olvida - ver
  // intencionReiniciadaPorInactividad. Se calcula ANTES de extraer filtros
  // para que, si el mensaje de hoy SI menciona una categoria, no quede
  // protegida de menos por la categoria vieja que se esta por descartar.
  const intencionFresca = intencionReiniciadaPorInactividad(clienteFinal);
  if (intencionFresca) {
    logEtapa('reinicio_por_inactividad_intencion', telefonoCliente, { minutos: MINUTOS_PARA_REINICIAR_INTENCION });
```

```
  }

  // Extraccion determinista por codigo: se aplica ANTES de armar el prompt,
  // sin depender de que el modelo llame a actualizar_datos_lead.
  const detectadosDeEsteMensaje = extraerFiltros(textoParaExtraccion, productos, intencionFresca ? null :
clienteFinal.categoriaId);
  // Lo que reseteamos por inactividad son valores por defecto: si el mensaje
  // de HOY ya trae algo (ej. menciono una categoria), eso gana.
  const detectados = intencionFresca ? { ...intencionFresca, ...detectadosDeEsteMensaje } : detectadosDeEsteMensaje;

  // Lo mismo para los datos que la tienda pide ANTES de mostrar nada (el gate
  // de preguntasIniciales, tipicamente el Genero). Sin este respaldo, el unico
  // camino para guardar la respuesta era que el modelo llamara a
  // actualizar_datos_lead: cuando no lo hacia, el cliente contestaba "hombre"
  // y el bot volvia a preguntarle el genero una y otra vez.
  const inicialesResueltas = resolverDatosIniciales(textoParaExtraccion, agente.config, clienteFinal, productos);
  if (inicialesResueltas) {
    Object.assign(detectados, inicialesResueltas);
    logEtapa('datos_iniciales_resueltos', telefonoCliente, inicialesResueltas);
  }

  // El cliente estuvo un rato largo sin escribir: se olvida QUE se le mostro,
  // para que si vuelve y pide las zapatillas otra vez se las mande de nuevo en
  // vez de contestarle "ya te las mostre". Lo que dijo (categoria, talla,
  // color, nombre, direccion) se conserva por mas tiempo (ver arriba),
  // pero eventualmente tambien se pierde si paso demasiado.
  const contextoFresco = contextoReiniciadoPorInactividad(clienteFinal.contexto || {});
  if (contextoFresco) {
    detectados.contexto = contextoFresco;
    logEtapa('reinicio_por_inactividad', telefonoCliente, { minutos: MINUTOS_PARA_REINICIAR_VISTOS });
  } else {
    detectados.contexto = { ...(clienteFinal.contexto || {}), ultimoTurnoAt: new Date().toISOString() };
  }

  // El cliente contesto "cualquiera" / "me da igual": eso ES una respuesta.
  // Se anota que no tiene preferencia por lo que se le estaba preguntando, asi
  // el gate se destraba y la busqueda NO se filtra por ese atributo. Sin esto
  // el bot le contestaba "no, tenes que escoger un color antes de seguir" y no
  // salia nunca de ahi (reportado por el negocio).
  if (expresaSinPreferencia(mensajeCliente)) {
    const leadParcial = { ...clienteFinal, ...detectados };
    const categoriaParaPendientes = categoriaDelLead(productos, leadParcial);
    const pendientes = [
      ...preguntasInicialesFaltantes(agente.config, leadParcial, productos),
      ...atributosFaltantes(categoriaParaPendientes, leadParcial, 'OBLIGATORIO'),
      ...atributosFaltantes(categoriaParaPendientes, leadParcial, 'RECOMENDADO'),
    ];
    if (pendientes.length) {
      const yaAnotados = (detectados.contexto || {}).sinPreferencia || [];
      const sinPreferencia = [...new Set([...yaAnotados, ...pendientes])];
      detectados.contexto = { ...detectados.contexto, sinPreferencia };
```

```
    logEtapa('cliente_sin_preferencia', telefonoCliente, { atributos: pendientes });
  }
}

// Cuenta cuantas veces se le pidio al cliente el mismo dato inicial sin
// conseguirlo. Al llegar al tope, preguntasInicialesFaltantes deja de
// bloquear y el cliente puede ver el catalogo igual: es preferible mostrar
// de mas que dejarlo atrapado contestando siempre lo mismo.
const leadTrasResolver = { ...clienteFinal, ...detectados };
const pedidasIniciales = (agente.config && agente.config.preguntasIniciales) || [];
// OJO: se mira leadYaTiene y no preguntasInicialesFaltantes. Esa funcion ya
// devuelve [] cuando el contador llego al tope (gate liberado), asi que
// usarla aca reseteaba el contador justo despues de liberarlo y el gate se
// volvia a armar en el turno siguiente: el mismo bucle, mas lento.
const resueltasTodas = pedidasIniciales.every((n) => leadYaTiene(leadTrasResolver, n));
if (pedidasIniciales.length && !resueltasTodas) {
  const intentos = Number((detectados.contexto || {}).intentosPreguntaInicial || 0) + 1;
  detectados.contexto = { ...detectados.contexto, intentosPreguntaInicial: intentos };
  if (intentos === MAX_INTENTOS_PREGUNTA_INICIAL) {
    logEtapa('gate_inicial_liberado', telefonoCliente, { faltaban: pedidasIniciales, intentos });
  }
} else if ((clienteFinal.contexto || {}).intentosPreguntaInicial) {
  detectados.contexto = { ...detectados.contexto, intentosPreguntaInicial: 0 };
}

let lead = clienteFinal;
if (Object.keys(detectados).length) {
  // El cliente cambio de rubro y lo detecto el codigo (no la IA): hay que
  // soltar lo que era de la categoria vieja igual que en
  // actualizar_datos_lead, o el bot sigue empujando un producto que ya no
  // viene al caso.
  const cambioDeCategoriaEsteTurno = Boolean(detectados.categoriaId) && detectados.categoriaId !== clienteFinal.categoriaId;
  if (cambioDeCategoriaEsteTurno) {
    // OJO: esto pisa detectados.contexto entero (vuelve a partir de
    // clienteFinal.contexto, no del detectados.contexto ya armado arriba),
    // asi que cualquier cosa que dependa de contexto tiene que calcularse
    // DESPUES de esta linea, no antes - ver el contador de frustracion
    // mas abajo, que por eso corre aca y no mas arriba con el resto.
    Object.assign(detectados, limpiezaPorCambioDeCategoria(clienteFinal, productos, detectados.categoriaId));
  }

  // Contador de turnos sin avanzar de estadoConversacion (backstop de
  // frustracion acumulada, ver UMBRAL_TURNOS_SIN_AVANZAR arriba). Compara
  // el estado AL EMPEZAR este turno contra el que tenia al empezar el
  // turno anterior (guardado la vez pasada): si es el mismo, no hubo
  // progreso en el turno anterior y se suma uno. Se guarda con un turno de
  // margen a proposito -no hace falta medir tambien al final de este
  // turno- para no duplicar el punto de medicion.
  //
  // TAMBIEN cuenta como progreso que el cliente haya pedido una categoria
  // DISTINTA en este mismo mensaje: sin esto, un cliente navegando el
```

```
    // catalogo sin trabarse ("que tienes en zapatillas" -> "y en gorras" ->
    // "y camisas") sumaba al contador en cada mensaje solo porque
    // estadoConversacion se queda en EXPLORANDO mientras esta mirando - el
    // backstop es para conversaciones ESTANCADAS, no para exploracion activa.
    {
      const estadoAlEmpezarEsteTurno = clienteFinal.estadoConversacion;
      const contextoPrevio = clienteFinal.contexto || {};
      const contadorPrevio = Number(contextoPrevio.turnosSinAvanzarEstado || 0);
      const sinCambioDeEstado = contextoPrevio.estadoConversacionAnterior != null
        && contextoPrevio.estadoConversacionAnterior === estadoAlEmpezarEsteTurno;
      const turnosSinAvanzarEstado = (sinCambioDeEstado && !cambioDeCategoriaEsteTurno) ? contadorPrevio + 1 : 0;
      detectados.contexto = { ...detectados.contexto, estadoConversacionAnterior: estadoAlEmpezarEsteTurno,
turnosSinAvanzarEstado };
    }

    try {
      lead = await prisma.clienteFinal.update({ where: { id: clienteFinal.id }, data: detectados });
    } catch (err) {
      logErrorEtapa('memoria', telefonoCliente, err);
      return { ok: true, demo: false, error: true, proveedor: 'error', respuesta: respuestaErrorTecnico() };
    }
  }
  // El cliente agrego algo al carrito desde el catalogo web (fuera de una
  // tool call del bot, ver POST /catalogo/:slug/producto/:id/carrito en
  // server.js) - se le avisa al modelo como un hecho ya ocurrido para que
  // reaccione solo ("¡buena eleccion!") en vez de que el cliente tenga que
  // repetirselo. Se limpia la marca de una: si el proveedor de IA falla a
  // mitad de turno, se prefiere perder el aviso una vez a repetirlo en bucle.
  let avisoCarritoWeb = null;
  if (lead.contexto && lead.contexto.carritoWebPendiente) {
    const pendiente = lead.contexto.carritoWebPendiente;
    const productoAgregado = productos.find((p) => p.id === pendiente.productoId);
    if (productoAgregado) {
      avisoCarritoWeb = `RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente en este mensaje - paso hace un momento en el
catalogo web): el cliente ACABA de agregar esto a su carrito desde la web:\n${fichaProducto(productoAgregado, lead,
empresa.moneda)}\n\nTu texto AHORA: saludalo con entusiasmo natural por esa eleccion ("¡buena eleccion!" o similar), confirmale
que ya quedo en su carrito, y preguntale si quiere ver algo mas. NUNCA le ofrezcas ni menciones otro producto por tu cuenta. NO
le preguntes que producto eligio: ya lo sabes.`;
    }
    try {
      await prisma.clienteFinal.update({
        where: { id: clienteFinal.id },
        data: { contexto: { ...lead.contexto, carritoWebPendiente: null } },
      });
      lead = { ...lead, contexto: { ...lead.contexto, carritoWebPendiente: null } };
    } catch (err) {
      logErrorEtapa('memoria', telefonoCliente, err);
    }
  }

  const categoriaActual = categoriaDelLead(productos, lead);
```

```
const faltantesObligatorios = atributosFaltantes(categoriaActual, lead, 'OBLIGATORIO');
logEtapa('atributos_detectados', telefonoCliente, {
  ...detectados,
  presupuesto: lead.presupuesto,
  marca: lead.marca,
  talla: lead.talla,
  purchase_stage: lead.estadoConversacion,
  missing_attributes: faltantesObligatorios,
});

const busquedaActual = faltantesObligatorios.length
  ? { resultados: [], total: 0, relajado: null }
  : buscarConFallback(productos, lead);
const candidatosActuales = busquedaActual.resultados;
logEtapa('busqueda_ejecutada', telefonoCliente, {
  categoria: lead.categoriaInteres || null,
  total_matches: busquedaActual.total,
  results_returned: paginar(candidatosActuales, (lead.contexto || {}).fotosEnviadas || []).pagina.length,
  filtro_relajado: busquedaActual.relajado,
  bloqueado_por_datos_faltantes: faltantesObligatorios.length > 0,
});

// Deterministico (en codigo, no depende del modelo): el cliente nombro EN
// SU PROPIO MENSAJE un producto real, ya sea de los candidatos actuales o
// de algo que ya se le mostro antes en esta conversacion. Si es asi, mas
// abajo se fuerza que la tarjeta de ESE producto sea la que se manda este
// turno, sin importar que IDs haya elegido el modelo. Bug real reportado:
// el cliente escribio "Zapatillas Park St 2.0" (ya mostrada) y el bot le
// mando tres tarjetas totalmente distintas, mintiendo en el texto que
// esa era una de ellas.
const universoNombrable = [
  ...candidatosActuales,
  ...productos.filter((p) => (lead.productosMostrados || []).includes(p.id)),
];
const idsYaVistos = new Set();
const productoNombradoPorCliente = universoNombrable.find((p) => {
  if (idsYaVistos.has(p.id)) return false;
  idsYaVistos.add(p.id);
  return nombreDeProductoEnTexto(mensajeCliente, p.nombre);
}) || null;

const previos = historial.map((m) => ({ role: m.rol === 'CLIENTE' ? 'user' : 'assistant', content: m.contenido }));
// El historial ya incluye el mensaje actual del cliente (se persiste antes
// de generar la respuesta), asi que "primer mensaje" NO es historial vacio:
// es que el agente todavia nunca respondio nada en esta conversacion.
const esPrimerMensaje = !historial.some((h) => h.rol === 'AGENTE');

const proveedor = proveedorActivo();
const plan = empresa.suscripcion && empresa.suscripcion.plan;

// Ruteo por costo: lo dificil (y lo que justifica el modelo caro) es
```

```
  // entender que quiere el cliente y elegir bien que mostrarle - no saludar
  // ni pasar el menu de categorias. Mientras el cliente todavia no eligio
  // ninguna categoria (candidatosActuales SIEMPRE tiene algo si hay stock,
  // aunque no haya dicho nada: sin categoria, buscarConFallback no filtra
  // nada y devuelve todo el catalogo - por eso no sirve como señal) ni tiene
  // nada en el carrito/favorito, el turno es "de orientacion": saludo, ayuda
  // generica, ver el menu de categorias. Ahi se usa el modelo economico sin
  // importar el plan. En cuanto sabemos que categoria le interesa (lo
  // detecta el codigo, no la IA, apenas lo dice) o ya eligio un favorito, se
  // pasa al modelo del plan: ahi es donde de verdad se juega entender la
  // necesidad del cliente y no trabarse mostrando.
  const esTurnoSimple = !lead.categoriaInteres && !lead.productoFavoritoId;
  const modelo = esTurnoSimple ? modeloEconomico(proveedor) : modeloParaPlan(proveedor, plan);
  logEtapa('modelo_elegido', telefonoCliente, { proveedor, modelo, esTurnoSimple, plan: plan && plan.codigo });
  const llamar = opciones.llamarInyectado || (proveedor === 'openai' ? llamarOpenAI : proveedor === 'anthropic' ?
llamarAnthropic : null);

  if (!llamar) {
    logEtapa('modo_demo', telefonoCliente, { motivo: 'sin_api_key' });
    return { ok: true, demo: true, proveedor: 'demo', respuesta: respuestaDemo(empresa, productos, mensajeCliente) };
  }

  // Fotos que las tools van registrando en este turno para el modo de
  // prueba (sin WhatsApp real conectado), para que el chat de prueba las
  // pueda mostrar como imagenes de verdad.
  const fotosParaMostrar = [];
  // "turno" identifica este intercambio puntual: sirve para exigir que ciertas
  // confirmaciones (ej: aceptar ver alternativas con un filtro aflojado) pasen
  // por una respuesta REAL del cliente y no se resuelvan solas en la misma vuelta.
  //
  // Se cuentan los mensajes del CLIENTE en la conversacion, no historial.length:
  // el historial viene cortado a los ultimos 20 mensajes, asi que en una charla
  // larga su largo se queda clavado y el numero de turno deja de avanzar. Con
  // eso, el sistema nunca se enteraba de que el cliente habia contestado y le
  // volvia a pedir permiso una y otra vez - bucle real en produccion: el
  // cliente dijo "si quiero ver" tres veces y el bot seguia preguntando.
  const turno = conversacionId
    ? await prisma.mensaje.count({ where: { conversacionId, rol: 'CLIENTE' } }).catch(() => historial.length)
    : historial.length;

  // Compras reales anteriores del mismo cliente (Opcion A de
  // docs/05-propuesta-personalizacion-ia.md): se lee UNA vez por turno, no
  // se inventa nada - si no hay pedidos, el prompt simplemente no menciona
  // el bloque. Se busca por telefono/empresa (no por clienteFinal.id) por si
  // el registro de lead se recreo en algun momento.
  const pedidosPrevios = await prisma.pedido.findMany({
    where: { empresaId: empresa.id, cliente: { telefono: telefonoCliente }, estado: { not: 'CANCELADO' } },
    orderBy: { createdAt: 'desc' },
    take: 2,
    include: { items: { include: { variante: true, producto: { include: { categoria: true } } } } },
  }).catch(() => []);
```

```
  // Bug real probando en vivo: el chat de prueba del panel ("Probar mi
  // agente") usa un telefono sintetico (ej. "demo-201674"), NUNCA un numero
  // real. Cuando la empresa YA tiene WhatsApp conectado de verdad, las tools
  // (mostrar_tarjeta_categoria, mostrar_productos) solo miran
  // conexion.estado==='CONECTADO' para decidir si intentan un envio real - y
  // lo intentaban igual contra ese telefono falso, sin ninguna foto
  // terminando en fotosParaMostrar (asi que el chat de prueba nunca mostraba
  // imagenes) y gastando llamadas reales a la API de WhatsApp para nada. Con
  // opciones.modoPrueba, se fuerza a que las tools se comporten como si no
  // hubiera conexion real - exactamente el mismo camino de "modo de prueba"
  // que ya usan las empresas que todavia no conectaron WhatsApp.
  const helpers = { empresaId: empresa.id, empresaSlug: empresa.slug, telefonoCliente, conexion: opciones.modoPrueba ? null :
agente.conexion, fotosParaMostrar, conversacionId, baseUrl: opciones.baseUrl, config: agente.config, moneda: empresa.moneda ||
'BOB', turno, mensajeCliente, ultimoMensajeAgente: ultimoAgente && ultimoAgente.contenido };

  // DECISION DETERMINISTICA (backend, no la IA), CON PRIORIDAD sobre los
  // bloques de abajo: "quiero salir de esta categoria / ver todo lo que
  // venden" se detecta SIEMPRE antes de la IA, sin importar si ya hay una
  // categoria fijada en el lead o si esta o no en cierre. Antes de este
  // bloque, pareceQuererCatalogoCompleto solo se evaluaba DENTRO del
  // handler de la tool mostrar_categorias - si la IA, ese turno, decidia no
  // llamarla (por ejemplo, describiendo de memoria lo que ya habia mostrado
  // antes), la señal nunca se aplicaba. Bug real probando en vivo: "Ya vi
  // poleras, que mas tienes" (sin carrito, sin decir "otra categoria")
  // repitio las subcategorias de Poleras en vez de subir al menu completo.
  // Va PRIMERO (antes del bloque de "categoria recien elegida" de abajo) a
  // proposito: si el cliente pide "ver todo" tiene prioridad sobre
  // cualquier categoria que el mismo mensaje tambien haya nombrado.
  if ((pareceQuererCatalogoCompleto(mensajeCliente) || pidioVerOtraCosaRespondiendoAlBot(ultimoAgente &&
ultimoAgente.contenido)) && !productoNombradoPorCliente) {
    const arbolPre = arbolDeCategorias(productos, lead);
    if (arbolPre.length) {
      const lista = arbolPre.map((r, i) => `${i + 1}. ${r.nombre}`).join('\n');
      logEtapa('catalogo_completo_forzado_preemptivo', telefonoCliente, {});
      return { ok: true, demo: false, proveedor: 'sistema', modelo: null, respuesta: `Esto es lo que
tenemos:\n${lista}\n\n¿Cuál te interesa?`, fotos: [] };
    }
  }

  // DECISION DETERMINISTICA (backend, no la IA), un paso antes de la tarjeta
  // de categoria de abajo: sin ninguna categoria elegida todavia (ni rubro
  // ni subcategoria) y sin nombrar nada puntual, el MENU de rubros se manda
  // directo - es siempre la misma lista real, no hay nada que interpretar.
  // Pedido explicito del dueño despues de un bug real: el cliente contesto
  // el genero que la tienda pide y el bot respondio "tuve un problema" antes
  // de mostrar el menu, aunque mostrar_categorias nunca falla de verdad (es
  // una consulta pura a la tabla Categoria) - lo que fallaba era dejar que
  // la IA decidiera turno a turno si llamar a esa funcion o probar otra
  // cosa. El menu de NOMBRES de categorias nunca depende de datos del
  // cliente (genero, etc.) - eso solo importa para filtrar PRODUCTOS
  // adentro de una categoria puntual (ver Fix A en el handler de
```

```
  // mostrar_categorias, unas lineas mas abajo).
  if (!lead.categoriaId && !productoNombradoPorCliente && !lead.productoFavoritoId &&
!ESTADOS_DE_CIERRE.has(lead.estadoConversacion) && !(lead.productosMostrados || []).length) {
    const arbolPre = arbolDeCategorias(productos, lead);
    if (arbolPre.length) {
      helpers.leadActual = lead;
      // Mismo mensaje de bienvenida que la tienda ya configuro para el
      // camino AI-driven (config.mensajeBienvenida, ver partesDelSystem) -
      // si no configuraron uno, se arma una frase cordial de respaldo, nunca
      // el nombre pelado como antes ("Soy Raul de Tienda Demo." sonaba a
      // formulario, no a un vendedor real).
      const bienvenidaConfigurada = (agente.config && agente.config.mensajeBienvenida) || '';
      const saludo = esPrimerMensaje
        ? (bienvenidaConfigurada
          ? `${bienvenidaConfigurada} `
          : `¡Hola! Soy ${agente.nombre || 'el asistente de ventas'} de ${empresa.marca || empresa.nombre}. Encantado de
ayudarte a encontrar justo lo que buscas. `)
        : '';
      if (arbolPre.length === 1 && !arbolPre[0].subcategorias.length) {
        // Mismo camino que menu_rubro_unico_auto (dentro del handler de
        // mostrar_categorias): un solo rubro sin subcategorias, se reusa la
        // tool para que se autoasigne y mande la tarjeta - misma fuente de
        // verdad que ya usa el camino AI-driven, no se reimplementa.
        const resultadoUnico = await ejecutarFuncion({ name: 'mostrar_categorias', arguments: {} }, productos, helpers);
        if (typeof resultadoUnico === 'string' && resultadoUnico.startsWith('TOOL_SUCCESS') && fotosParaMostrar.length) {
          logEtapa('menu_rubros_forzado_preemptivo', telefonoCliente, { rubroUnico: arbolPre[0].nombre });
          return { ok: true, demo: false, proveedor: 'sistema', modelo: null, respuesta: `${saludo}Echale un vistazo a
${arbolPre[0].nombre.toLowerCase()} que tenemos 👆`, fotos: fotosParaMostrar };
        }
        // TOOL_FAILED (atributo obligatorio de ESA categoria todavia falta,
        // ej. Genero marcado como obligatorio en esa categoria puntual) ->
        // no se improvisa nada, sigue el flujo normal con la IA.
      } else {
        const lista = arbolPre.map((r, i) => `${i + 1}. ${r.nombre}`).join('\n');
        logEtapa('menu_rubros_forzado_preemptivo', telefonoCliente, { opciones: arbolPre.length });
        return { ok: true, demo: false, proveedor: 'sistema', modelo: null, respuesta: `${saludo}Esto es lo que
tenemos:\n${lista}\n\n¿Cuál te interesa?`, fotos: [] };
      }
    }
  }

  // DECISION DETERMINISTICA (backend, no la IA): si el cliente ELIGIO LA
  // CATEGORIA RECIEN, EN ESTE MISMO MENSAJE (detectados.categoriaId, no una
  // que ya estaba fijada de turnos anteriores), no nombro nada puntual, y
  // todavia no vio la tarjeta de esa categoria, se manda DIRECTO - no se le
  // pregunta al modelo. Mismo principio que menu_rubro_unico_auto (ver mas
  // abajo): el backend decide y ejecuta, la IA no entra en esta decision.
  // Pedido explicito del dueño del negocio despues de varios bugs reales
  // donde el modelo no llamaba a mostrar_tarjeta_categoria (a veces
  // describia la categoria en texto plano sin llamar ninguna tool, a veces
  // intentaba mostrar_productos sin categoria elegida) - los detectores
```

```
  // reactivos que ya existen quedan como red de seguridad para los demas
  // casos (producto puntual, cierre, rubro con subcategorias, mensajes de
  // seguimiento que no vuelven a nombrar la categoria), pero el momento
  // exacto de elegir categoria ya no depende de que la IA decida bien.
  //
  // El gate exige detectados.categoriaId (no solo lead.categoriaId) a
  // proposito: bug real encontrado probando esto - sin ese chequeo, el
  // bloque interceptaba CUALQUIER mensaje de una conversacion ya en curso
  // (preguntar por una foto de color, confirmar un pedido, etc.) solo
  // porque la categoria seguia fijada de turnos anteriores y el mensaje no
  // nombraba un producto por su nombre completo.
  if (detectados.categoriaId && !productoNombradoPorCliente && !lead.productoFavoritoId &&
!ESTADOS_DE_CIERRE.has(lead.estadoConversacion) && !(lead.productosMostrados || []).length) {
    const categoriaPre = categoriaDelLead(productos, lead);
    if (categoriaPre) {
      const yaMostradaPre = ((lead.contexto || {}).tarjetasCategoriaMostradas || []).includes(categoriaPre.id);
      const esRubroPre = !categoriaPre.padreId && subcategoriasDe(productos, categoriaPre.id, lead).length;
      if (!yaMostradaPre && !esRubroPre) {
        // helpers.leadActual normalmente se setea recien adentro del loop de
        // vueltas (linea ~3010) - como este bloque corre ANTES del loop,
        // hace falta setearlo aca a mano, o el handler de
        // mostrar_tarjeta_categoria ve leadActual undefined y falla siempre
        // (categoriaDelLead(contexto, {}) da null -> "TOOL_FAILED, todavia
        // no hay categoria elegida", aunque si la haya).
        helpers.leadActual = lead;
        const resultadoPre = await ejecutarFuncion({ name: 'mostrar_tarjeta_categoria', arguments: {} }, productos, helpers);
        if (typeof resultadoPre === 'string' && resultadoPre.startsWith('TOOL_SUCCESS')) {
          logEtapa('tarjeta_categoria_forzada_preemptiva', telefonoCliente, { categoria: categoriaPre.nombre });
          return { ok: true, demo: false, proveedor: 'sistema', modelo: null, respuesta: `Echale un vistazo a
${categoriaPre.nombre.toLowerCase()} que tenemos 👆`, fotos: fotosParaMostrar };
        }
        // TOOL_FAILED (falta atributo obligatorio, sin stock, envio real
        // fallo) -> no se improvisa nada aca, sigue el flujo normal con la
        // IA, que ya sabe manejar esos casos (pedir el dato que falta,
        // avisar sin stock, etc.) exactamente como hoy.
      }
    }
  }

  // DECISION DETERMINISTICA (backend, no la IA): si al cliente en cierre YA
  // le falta UNICAMENTE la forma de pago (nombre, tipo de entrega y ubicacion
  // -si aplica- ya resueltos), se le manda un mensaje con BOTONES reales en
  // vez de dejar que la IA redacte la pregunta en texto libre. Espejo
  // DELIBERADAMENTE acotado de las condiciones que ya valida el handler de
  // confirmar_pedido/crear_pedido (nombreValido, tipoEntrega, direccion
  // resuelta) unas lineas mas arriba: si alguna de esas condiciones todavia
  // no esta lista, este bloque se autoexcluye solo y el flujo normal (IA +
  // ese mismo handler) sigue pidiendo lo que falta como siempre - nunca
  // intenta RESOLVER una direccion aca (eso sigue siendo trabajo exclusivo
  // del handler), solo mira si ya quedo resuelta antes.
  const carritoParaBotonesPago = carritoDe(lead.contexto || {}, conversacionId);
```

```
  if (carritoParaBotonesPago.length && !lead.formaPago && nombreValido(lead.nombre) && lead.tipoEntrega
    && (lead.tipoEntrega !== 'DOMICILIO' || Boolean(lead.ubicacionLat))
    && (lead.tipoEntrega !== 'RECOJO' || Boolean(agente.config && agente.config.direccionTienda))) {
    const metodosDePagoBotones = [
      agente.config && agente.config.aceptaQr ? { id: 'QR', titulo: 'QR' } : null,
      !agente.config || agente.config.aceptaEfectivo !== false ? { id: 'EFECTIVO', titulo: 'Efectivo' } : null,
      agente.config && agente.config.aceptaTarjeta ? { id: 'TARJETA', titulo: 'Tarjeta' } : null,
    ].filter(Boolean);
    // Con menos de 2 opciones no hay nada que "elegir" con botones (0 = ni
    // siquiera hay forma de pago configurada, 1 = no hace falta preguntar) -
    // en esos casos se deja que el flujo normal lo resuelva como hoy.
    if (metodosDePagoBotones.length >= 2) {
      logEtapa('botones_forma_pago_preemptivo', telefonoCliente, { opciones: metodosDePagoBotones.map((m) => m.id) });
      return { ok: true, demo: false, proveedor: 'sistema', modelo: null, respuesta: '¿Cómo preferís pagar?', botones:
metodosDePagoBotones, fotos: [] };
    }
  }
```

7.2 Tramo 2 — inicio del bucle y llamada al modelo

generarRespuesta (tramo 2: bucle, parámetros, llamada, rescate 429, registro de uso)

Archivo: `lib/services/agente.js` *· líneas 3483–3591*

```
  const mensajesTurno = avisoCarritoWeb ? [{ role: 'user', content: avisoCarritoWeb }] : [];
  const MAX_VUELTAS = 3;

  // Una vez que se lo freno por "no sabes que combinacion se lleva", la
  // pregunta por talla/color de este turno es LEGITIMA: es completar la
  // compra, no filtrar antes de mostrar. Sin esta marca los dos detectores
  // se pelean - el de "pedis preferencia sin mostrar" atrapa la pregunta que
  // el otro acaba de pedir, y el turno termina reenviando tarjetas.
  let frenadoPorVariante = false;

  try {
    for (let vuelta = 0; vuelta < MAX_VUELTAS; vuelta++) {
      // El prompt se rearma en cada vuelta con el lead recalculado.
      const leadActual = vuelta === 0 ? lead : await prisma.clienteFinal.findUnique({ where: { id: clienteFinal.id } });
      helpers.leadActual = leadActual;
      const esUltimaVuelta = vuelta === MAX_VUELTAS - 1;
      const pidioProductoPuntual = Boolean(productoNombradoPorCliente) || Boolean((helpers.idsPedidosPorNombre || []).length);
      const { fijo: systemFijo, variable: systemVariable } = partesDelSystem(empresa, productos, agente.config, leadActual,
esUltimaVuelta, esPrimerMensaje, agente.nombre, Boolean(helpers.tarjetasEnTurno), pedidosPrevios, pidioProductoPuntual);
      const system = `${systemFijo}\n\n${systemVariable}`;

      // DECISION DETERMINISTICA (backend, no la IA): mismo criterio exacto
      // que ya usan, cada uno por su lado, los detectores backstop
      // debioLlamarTarjetaCategoria/nombrado/listadoEnTexto/pidiendoPreferencia
      // (ver mas abajo, dentro de esta misma vuelta) - hay candidatos reales
      // para mostrar, el cliente no nombro nada puntual, y no esta en pleno
      // cierre. En ESE estado exacto la unica respuesta valida es una tool
```

```
      // call (mostrar_tarjeta_categoria/mostrar_productos), nunca texto
      // libre - es justo el estado que hoy genero "Zapatillas Urbanas Flex"
      // inventado. Fuera de esa condicion (ej. falta un atributo
      // obligatorio y hay que preguntarlo en lenguaje natural),
      // candidatosActuales.length es 0 y el gate se auto-excluye solo, sin
      // tocar ese camino legitimo de pregunta libre.
      const yaEnCierrePre = Boolean(leadActual.productoFavoritoId)
        || carritoDe(leadActual.contexto || {}, conversacionId).length > 0
        || ESTADOS_DE_CIERRE.has(leadActual.estadoConversacion);
      const soloAccionValida = !esUltimaVuelta && !yaEnCierrePre && candidatosActuales.length && !pidioProductoPuntual;

      // Backstop de frustracion acumulada (ver UMBRAL_TURNOS_SIN_AVANZAR):
      // si ya se avisaron varios mensajes reales seguidos sin que la
      // conversacion avance de estado, se fuerza la derivacion sin esperar a
      // que la IA "sienta" que el cliente esta molesto en ESTE turno
      // puntual. No aplica si ya se derivo antes (estadoLead ya en
      // DERIVADO_A_ASESOR): ahi no hay nada mas que forzar.
      const debeEscalarPorFrustracion = !esUltimaVuelta
        && Number((leadActual.contexto || {}).turnosSinAvanzarEstado || 0) >= UMBRAL_TURNOS_SIN_AVANZAR
        && leadActual.estadoLead !== 'DERIVADO_A_ASESOR';

      const paramsLlamada = {
        system,
        systemFijo,
        systemVariable,
        mensajes: [...previos, { role: 'user', content: mensajeCliente }, ...mensajesTurno],
        tools: esUltimaVuelta
          ? []
          : (debeEscalarPorFrustracion ? TOOLS.filter((t) => t.function.name === 'derivar_a_asesor') :
toolsSegunEstado(leadActual, conversacionId)),
        toolChoice: (soloAccionValida || debeEscalarPorFrustracion) ? 'requerido' : 'auto',
        modelo,
      };
      let resp;
      try {
        resp = await llamar(paramsLlamada);
      } catch (errLlamada) {
        // Rate limit (429) del proveedor: bug real en produccion, dos
        // conversaciones simultaneas saturaron el limite de tokens/minuto de
        // gpt-4o y el cliente se quedo sin respuesta ("no pude consultar la
        // disponibilidad") aunque el catalogo si tenia todo. En vez de
        // fallar el turno entero, se reintenta UNA vez con el modelo
        // economico del mismo proveedor (bucket de limite distinto, casi
        // siempre con margen de sobra) antes de rendirse de verdad.
        const modeloDeRescate = modeloEconomico(proveedor);
        if (errLlamada?.status === 429 && modelo !== modeloDeRescate) {
          logEtapa('proveedor_ia_rate_limit_reintentado', telefonoCliente, { modeloOriginal: modelo, modeloDeRescate });
          resp = await llamar({ ...paramsLlamada, modelo: modeloDeRescate });
        } else {
          throw errLlamada;
        }
```

```
}
if (resp.usage) {
  logEtapa('uso_tokens', telefonoCliente, { proveedor, modelo, vuelta, ...resp.usage });
  try {
    // OpenAI y Anthropic devuelven la forma de "usage" con nombres de
    // campo distintos (prompt_tokens/completion_tokens vs.
    // input_tokens/output_tokens/cache_read_input_tokens) - se
    // normaliza aca antes de guardar en columnas fijas.
    const u = resp.usage;
    const promptTokens = proveedor === 'anthropic'
      ? (u.input_tokens || 0) + (u.cache_read_input_tokens || 0) + (u.cache_creation_input_tokens || 0)
      : (u.prompt_tokens || 0);
    const cachedTokens = proveedor === 'anthropic'
      ? (u.cache_read_input_tokens || 0)
      : (u.prompt_tokens_details?.cached_tokens || 0);
    const completionTokens = proveedor === 'anthropic' ? (u.output_tokens || 0) : (u.completion_tokens || 0);
    await prisma.usoIA.create({
      data: {
        empresaId: empresa.id,
        conversacionId: conversacionId || null,
        proveedor,
        modelo,
        proposito: vuelta === 0 ? 'respuesta' : 'reintento_forzado',
        vuelta,
        promptTokens,
        cachedTokens,
        completionTokens,
      },
    });
  } catch (errUso) {
    logErrorEtapa('uso_ia_registro', telefonoCliente, errUso);
  }
}
```

7.3 Tramo 3 — la IA contestó solo texto: detectores y reintentos forzados

generarRespuesta (tramo 3: detectores y RECORDATORIO DEL SISTEMA con continue)

Archivo: `lib/services/agente.js` *· líneas 3593–3808*

```
if (!resp.tool_calls?.length) {
  // Si el cliente ya eligio un producto, preguntarle la talla NO es
  // filtrar: es completar la compra. Ahi si corresponde. Y si ya esta
  // en pleno cierre (favorito o algo en el carrito), es NORMAL que el
  // texto nombre el producto ("tus Park St 2.0 llegan a esa direccion")
  // - forzar una tarjeta ahi interrumpe el cierre en vez de seguirlo
  // (bucle real en produccion: pedia confirmar la direccion y el bot
  // le reenviaba la tarjeta sin llegar nunca a crear_pedido).
  const yaEligio = Boolean(lead.productoFavoritoId);
  // ESTADOS_DE_CIERRE se suma aca tambien: cubre el caso de
  // productoFavoritoId limpio (cambio de categoria a mitad del
```

```
        // cierre) pero estadoConversacion ya avanzado - bug real donde el
        // "rescate de ultima vuelta" volvia a mostrar tarjetas de producto
        // en vez de seguir el cierre.
        const yaEnCierre = Boolean(lead.productoFavoritoId)
          || carritoDe(leadActual.contexto || {}, conversacionId).length > 0
          || ESTADOS_DE_CIERRE.has(leadActual.estadoConversacion);

        // Los 4 detectores de "esta hablando en vez de actuar": cada uno
        // atrapa una forma distinta de texto vago (pedir preferencia sin
        // mostrar, nombrar un producto sin tarjeta, un listado con precios,
        // varias preguntas juntas, o prometer una busqueda que no llega).
        const pidiendoPreferencia = !yaEligio && !frenadoPorVariante && !helpers.tarjetasEnTurno && candidatosActuales.length
&& pidePreferenciaSinMostrar(resp.content);
        const nombrado = !yaEnCierre && !helpers.tarjetasEnTurno && candidatosActuales.length ?
nombraUnProductoReal(resp.content, candidatosActuales) : null;
        const listadoEnTexto = !yaEnCierre && !helpers.tarjetasEnTurno && candidatosActuales.length &&
pareceListadoDeProductosEnTexto(resp.content);
        // Modelo inventado como "ejemplo" (ver mencionaModeloInventado): a
        // diferencia de los otros detectores de arriba, esto aplica SIEMPRE
        // que haya candidatos reales contra los que comparar, incluso en
        // cierre - inventar un nombre de producto es grave en cualquier
        // etapa de la conversacion.
        const modeloInventado = candidatosActuales.length ? mencionaModeloInventado(resp.content, candidatosActuales) : null;
        const interrogatorio = pareceInterrogatorio(resp.content);
        const anuncioDeBusqueda = pareceAnuncioDeBusqueda(resp.content);
        // Con el carrito vacio, "ya esta en tu carrito" es necesariamente
        // falso (agregar_al_carrito exige variante y esta vuelta no llamo
        // ninguna tool) - ver afirmaAgregoAlCarrito.
        const carritoFalso = !carritoDe(leadActual.contexto || {}, conversacionId).length &&
afirmaAgregoAlCarrito(resp.content);
        // "No tenemos stock" es necesariamente falso si la busqueda REAL de
        // este turno (candidatosActuales, calculada por codigo antes del
        // loop) ya encontro algo con esos mismos filtros - ver
        // afirmaFaltaDeStockSinRespaldo.
        const faltaStockFalsa = candidatosActuales.length > 0 && afirmaFaltaDeStockSinRespaldo(resp.content);
        // El bot dice "te muestro esto"/"aqui puedes revisar" pero este turno
        // NO se mando ninguna tarjeta real (ni de producto ni de categoria) -
        // ver afirmaQueMostroAlgoSinTarjeta. Bug real: mostrar_tarjeta_categoria
        // fallo (TOOL_FAILED, ej. el envio real no salio) y el modelo escribio
        // igual un texto confiado como si hubiera mandado algo.
        //
        // Tambien cuenta como "algo real ya se mando" un menu de texto
        // (menuTextoEnTurno, ver mostrar_categorias) - sin esto, presentar un
        // menu de rubros/subcategorias real con una frase honesta como "aqui
        // tienes las opciones" se rechazaba igual que si no se hubiera
        // mandado nada, y el modelo terminaba inventando un "tuve un
        // problema" en la vuelta siguiente. Bug real reportado por WhatsApp.
        const afirmoSinMostrar = !helpers.tarjetasEnTurno && !helpers.menuTextoEnTurno &&
afirmaQueMostroAlgoSinTarjeta(resp.content);
        // Espejo de afirmoSinMostrar: el modelo inventa un FALLO tecnico
        // ("hubo un inconveniente", "intento mandartelas de nuevo") que
```

```
        // nunca ocurrio. Solo es invencion si NO hubo un TOOL_FAILED real en
        // este turno - si de verdad fallo un envio, la disculpa es honesta y
        // no hay que corregirla (ver afirmaQueMostroAlgoSinTarjeta arriba,
        // que exige el mismo tipo de evidencia real para el caso inverso).
        const huboFalloRealEnEsteTurno = mensajesTurno.some((m) => m.role === 'tool' && /TOOL_FAILED/.test(String(m.content ||
''')));
        const falloInventado = !huboFalloRealEnEsteTurno && afirmaFalloTecnicoInventado(resp.content);
        // No existe NINGUN campo de tiempo de entrega en toda la base (ver
        // prisma/schema.prisma) - un plazo especifico ("2 a 3 dias habiles")
        // es siempre inventado, sin importar el contexto. Bug real: el
        // cliente pregunto "¿en cuanto tiempo me llegara?" con el pedido ya
        // creado y el bot inventa un plazo de la nada.
        const plazoEntregaInventado = afirmaPlazoDeEntregaInventado(resp.content);
        // Bug real: el modelo NO llamo a NINGUNA tool (resp.tool_calls vacio,
        // por eso se esta en esta rama) y en vez de eso describio la
        // categoria en texto libre ("tenemos una excelente seleccion de
        // zapatillas, varios modelos disponibles...") sin mandar la tarjeta
        // real. afirmaQueMostroAlgoSinTarjeta no lo agarra porque el texto no
        // dice "te muestro"/"aqui tenes" - solo describe, sin afirmar que
        // mando algo. Esto es mas fuerte que un chequeo de frases: reusa
        // EXACTAMENTE el mismo gate que seccionProductos usa en el prompt
        // para forzar mostrar_tarjeta_categoria (agente.js linea ~740) - si
        // esa condicion esta activa y el modelo no llamo a la tool, esta mal
        // sin importar que haya escrito.
        const categoriaParaTarjeta = pidioProductoPuntual || yaEnCierre || frenadoPorVariante || !candidatosActuales.length ?
null : categoriaDelLead(productos, leadActual);
        const esRubroSinSubcategoriaAun = Boolean(categoriaParaTarjeta && !categoriaParaTarjeta.padreId &&
subcategoriasDe(productos, categoriaParaTarjeta.id, leadActual).length);
        const yaMostroTarjetaCategoria = Boolean(categoriaParaTarjeta && ((leadActual.contexto ||
{}).tarjetasCategoriaMostradas || []).includes(categoriaParaTarjeta.id));
        const debioLlamarTarjetaCategoria = Boolean(categoriaParaTarjeta) && !esRubroSinSubcategoriaAun &&
!yaMostroTarjetaCategoria && !helpers.tarjetasEnTurno;
        // Sin gate de candidatosActuales a proposito: una tool call escrita
        // como texto plano es invalida siempre, haya o no candidatos.
        const llamadaEnTexto = pareceLlamadaDeHerramientaEnTexto(resp.content);

        // Se va al cierre (entrega/pago/nombre) sin saber la combinacion
        // exacta: ni variante elegida, ni nada en el carrito, pero ya le
        // mostro productos que TIENEN talla y color con stock. Primero se
        // pregunta que se lleva; el resto viene despues.
        const productoPendienteDeVariante = (!lead.varianteFavoritaId
          && !carritoDe(leadActual.contexto || {}, conversacionId).length
          && pideDatosDeCierre(resp.content))
          ? (lead.productosMostrados || [])
            .map((id) => productos.find((p) => p.id === id))
            .find((p) => p && (p.variantes || []).some((v) => v.activa !== false && v.stock > 0))
          : null;

        // Si todavia quedan vueltas, se le da la chance de corregirse solo
        // (mejor: el modelo elige que decir). Cada uno con su propio
        // recordatorio especifico, por orden de especificidad.
```

```
        if (!esUltimaVuelta) {
          if (carritoFalso) {
            logEtapa('carrito_falso_rechazado', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): dijiste que el producto ya esta en el carrito, pero el carrito esta VACIO - eso nunca paso. Si el cliente ya eligio un producto, llama a agregar_al_carrito con su idVariante (preguntale la variante primero si todavia no la sabes); si todavia no eligio, no digas que lo agregaste, segui mostrando/ayudando a decidir.' });
            continue;
          }
          if (productoPendienteDeVariante) {
            frenadoPorVariante = true;
            logEtapa('cierre_sin_variante', telefonoCliente, { vuelta, productoId: productoPendienteDeVariante.id });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: `RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): le estas pidiendo datos de entrega o pago, pero todavia NO sabes que combinacion se lleva de "${productoPendienteDeVariante.nombre}". Un "si, me interesa" no dice ni la talla ni el color. Preguntale AHORA eso -una sola cosa por mensaje, arrancando por la talla-, guardalo con actualizar_datos_lead y agregalo al carrito con agregar_al_carrito. La entrega y el pago se preguntan DESPUES.` });
            continue;
          }
          if (pidiendoPreferencia) {
            logEtapa('pregunta_preferencia_sin_mostrar', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): le estas pidiendo color/talla/marca cuando YA tenes productos para mostrarle. Primero se muestra, despues se afina. Llama AHORA a mostrar_productos con los IDs del bloque de resultados; si despues hace falta acotar, se lo preguntas cuando ya haya visto las opciones.' });
            continue;
          }
          if (faltaStockFalsa) {
            logEtapa('falta_stock_falsa_rechazada', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): dijiste que no hay stock, pero el bloque de resultados de arriba SI tiene productos reales para esto. Nunca digas que no hay algo que esta cargado. Llama AHORA a mostrar_productos con los IDs del bloque de resultados.' });
            continue;
          }
          if (afirmoSinMostrar) {
            logEtapa('afirmo_mostro_sin_tarjeta_rechazado', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): le dijiste al cliente que le mostraste/mandaste algo, pero en este turno NO se envio ninguna tarjeta real (revisa arriba: si hubo un TOOL_FAILED, el envio real fallo). PROHIBIDO decir "te muestro", "aqui tienes" o similar sin haber mandado nada de verdad. Si todavia no llamaste a mostrar_tarjeta_categoria o mostrar_productos en este turno, hacelo ahora; si ya lo intentaste y fallo, decile la verdad con naturalidad (ej. "tuve un problema para mandarte las opciones, ¿te las intento de nuevo?"), nunca afirmes que le llego algo.' });
            continue;
          }
          if (debioLlamarTarjetaCategoria) {
            logEtapa('debio_llamar_tarjeta_categoria_rechazado', telefonoCliente, { vuelta, categoria:
```

```
categoriaParaTarjeta?.nombre });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: `RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): describiste "${categoriaParaTarjeta?.nombre || 'la categoria'}" en tu texto pero NO llamaste a ninguna funcion, asi que el cliente no ve tarjeta, foto ni link. Llama AHORA a mostrar_tarjeta_categoria - es la unica forma de mostrarle algo real en este punto, no lo describas vos de memoria.` });
            continue;
          }
          if (falloInventado) {
            logEtapa('fallo_tecnico_inventado_rechazado', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): dijiste que hubo un problema/inconveniente tecnico, pero NO llamaste a ninguna funcion este turno - ese fallo nunca paso, te lo inventaste como excusa. PROHIBIDO inventar fallas tecnicas que no ocurrieron. Contesta lo que el cliente realmente dijo, con normalidad; si estas en pleno cierre, segui pidiendo lo que falte (nombre, entrega, forma de pago) o llama a confirmar_pedido si ya tenes todo.' });
            continue;
          }
          if (plazoEntregaInventado) {
            logEtapa('plazo_entrega_inventado_rechazado', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): dijiste un plazo de entrega concreto (dias/horas/semanas), pero el sistema NO tiene ese dato en ningun lado - te lo inventaste. PROHIBIDO inventar plazos de entrega. Decile con honestidad que un asesor real le va a confirmar el tiempo exacto; si insiste o lo necesita con urgencia, llama a derivar_a_asesor.' });
            continue;
          }
          if (llamadaEnTexto) {
            logEtapa('llamada_de_tool_en_texto_rechazada', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): escribiste el nombre de una funcion como si fuera texto normal (ej. "mostrar_productos{...}") - eso el cliente lo ve tal cual, como codigo, nunca como una tarjeta. Las funciones se llaman de verdad con tool calling, JAMAS escribiendo su nombre en el mensaje. Volve a intentarlo llamando la funcion correspondiente de verdad, con los IDs reales del bloque de resultados (numeros, nunca nombres de producto).' });
            continue;
          }
          if (nombrado || listadoEnTexto) {
            logEtapa('producto_nombrado_sin_tarjeta', telefonoCliente, { producto: nombrado, listadoEnTexto, esUltimaVuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: `RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): ${nombrado ? `nombraste "${nombrado}"` : 'describiste productos'} en tu texto pero NO mandaste su tarjeta, asi que el cliente no ve ni foto, ni precio, ni tallas. Llama AHORA a mostrar_productos con los IDs del bloque de resultados. Si ya se lo mostraste antes en la conversacion, mandaselo igual: puede haber pasado un rato largo y no lo tiene a mano.` });
            continue;
          }
          if (modeloInventado) {
            logEtapa('modelo_inventado_rechazado', telefonoCliente, { vuelta, modeloInventado });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: `RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): dijiste "${modeloInventado}" como si fuera un modelo real, pero ESE NOMBRE NO EXISTE en el catalogo - te lo inventaste de memoria.
```

```
PROHIBIDO inventar nombres de productos. Llama AHORA a mostrar_productos con los IDs reales del bloque de resultados de arriba - esos son los unicos modelos que existen de verdad.` });
            continue;
          }
          if (interrogatorio) {
            logEtapa('interrogatorio_rechazado', telefonoCliente, { vuelta });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): mandaste mas de una pregunta en el mismo mensaje. Eso esta PROHIBIDO, parece un formulario. Volve a escribir el mensaje con UNA SOLA pregunta, la mas importante para poder avanzar; el resto se lo preguntas mas adelante si hace falta.' });
            continue;
          }
          if (anuncioDeBusqueda) {
            logEtapa('anuncio_de_busqueda_rechazado', telefonoCliente, { vuelta, yaEnCierre });
            mensajesTurno.push({ role: 'assistant', content: resp.content });
            mensajesTurno.push({ role: 'user', content: yaEnCierre
              ? 'RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): dijiste que ibas a revisar algo, pero el cliente ya esta en pleno cierre (tiene cosas en el carrito). NO le ofrezcas mas productos: segui el cierre pidiendole lo que falte (nombre, ubicacion o forma de pago), o llama a confirmar_pedido si ya tenes todo.'
              : `RECORDATORIO DEL SISTEMA (esto no lo dijo el cliente): dijiste que ibas a buscar algo, pero este mensaje es lo ultimo que el cliente recibe en este turno - se queda esperando de gusto. ${candidatosActuales.length ? 'Llama AHORA a mostrar_productos con los IDs del bloque de resultados y mostraselos de verdad.' : 'Todavia no podes mostrar productos: en vez de prometer una busqueda, hace la UNICA pregunta que te falta para poder buscar.'}` });
            continue;
          }
        } else if (carritoFalso) {
          // Ya no quedan vueltas: no se manda la afirmacion falsa tal cual.
          // Respuesta generica y siempre cierta - no puede nombrar el
          // producto sin arriesgarse a acertar mal cual era.
          logEtapa('carrito_falso_forzado', telefonoCliente, { vuelta });
          return {
            ok: true,
            demo: false,
            proveedor,
            modelo,
            respuesta: 'Para agregarlo a tu carrito me falta la talla y el color que preferís, ¿me confirmás eso?',
            fotos: fotosParaMostrar,
          };
        } else if (productoPendienteDeVariante) {
          // Ya no quedan vueltas: la pregunta la arma el codigo. Nunca se pasa
          // al cierre sin saber que combinacion se lleva el cliente.
          const tallas = [...new Set((productoPendienteDeVariante.variantes || [])
            .filter((v) => v.activa !== false && v.stock > 0)
            .map((v) => (v.atributos || {}).Talla)
            .filter(Boolean))];
          logEtapa('cierre_sin_variante_forzado', telefonoCliente, { vuelta, productoId: productoPendienteDeVariante.id });
          return {
            ok: true,
            demo: false,
            proveedor,
            modelo,
```

```
            respuesta: tallas.length
              ? `Para reservarte ${productoPendienteDeVariante.nombre}, ¿en qué talla lo querés? Tengo ${tallas.join(', ')}.`
              : `Para reservarte ${productoPendienteDeVariante.nombre}, ¿en qué talla y color lo querés?`,
            fotos: fotosParaMostrar,
          };
```

7.4 Tramo 4 — última vuelta: respuesta forzada por código

generarRespuesta (tramo 4: texto vago en última vuelta, forzado en código)

Archivo: `lib/services/agente.js` *· líneas 3809–4117*

```
        } else if (pidiendoPreferencia || nombrado || listadoEnTexto || interrogatorio || anuncioDeBusqueda || faltaStockFalsa
|| llamadaEnTexto || afirmoSinMostrar || debioLlamarTarjetaCategoria || falloInventado || plazoEntregaInventado ||
modeloInventado) {
          // Ya se quedo sin vueltas (insistio con texto vago 2 veces) y en
          // esta vuelta tools:[] - no puede corregirse solo. NO se le manda
          // ese texto al cliente tal cual: se fuerza una accion real en
          // codigo. Bug real en produccion: sin esto, la ultima vuelta
          // saltaba TODAS las validaciones y el texto de relleno se mandaba
          // igual ("dejame revisar de nuevo, un momento" sin mostrar nada).
          logEtapa('texto_vago_forzado_en_ultima_vuelta', telefonoCliente, { vuelta, pidiendoPreferencia, nombrado,
listadoEnTexto, interrogatorio, anuncioDeBusqueda, faltaStockFalsa, llamadaEnTexto, afirmoSinMostrar,
debioLlamarTarjetaCategoria, falloInventado, plazoEntregaInventado, modeloInventado, yaEnCierre });

          // El cliente ya eligio o tiene algo en el carrito: forzar
          // mostrar_productos aca reabre el catalogo a mitad del cierre -
          // bug real en produccion, el bot nunca terminaba de cobrar
          // (mandaba tarjetas de nuevo despues de que el cliente ya habia
          // dicho que no queria nada mas). En vez de eso, se fuerza una
          // pregunta de cierre real (nunca inventada: sale de los datos que
          // ya tiene o no tiene el cliente), el mismo espiritu que
          // productoPendienteDeVariante/carritoFalso de arriba.
          if (yaEnCierre) {
            // Antes de repetir la pregunta de cierre a ciegas: si el cliente
            // pidio EXPLICITAMENTE ver otra cosa (no una deteccion
            // incidental), se le muestra sin tocar el carrito ni el
            // favorito - mismo camino deterministico que ya usan los
            // bloques preemptivos de arriba (mostrar_tarjeta_categoria /
            // mostrar_categorias), solo que aca no se sale del cierre, se
            // pausa un turno y listo.
            if (pareceQuererVerOtraCosa(mensajeCliente) || detectados.categoriaId) {
              // detectados.categoriaId como señal alternativa: cubre un
              // nombre de categoria "pelado" (ej. "camisas", sin "quiero
              // ver" adelante) - bug real reportado por WhatsApp: el cliente
              // insistio con "camisas" a secas despues de que el primer
              // intento ("quiero ver tambien camisas") fallara, y como esa
              // frase sola no matchea pareceQuererVerOtraCosa, el bloque
              // caia directo a "¿me confirmas tu nombre?" ignorando por
              // completo lo que el cliente pedia. detectados.categoriaId ya
              // es un matching deterministico contra nombres reales
```

```
              // (extraerFiltros) - la misma señal que ya usan, con total
              // confianza, los bloques preemptivos de mas arriba.
              helpers.leadActual = leadActual;
              // Bug real reportado por WhatsApp real: el cliente ya habia
              // visto "Zapatillas" (correr/urbanas) y pidio explicitamente
              // "que mas tienes ya vi zapatillas" / "otras categorias?" -
              // frases que SI matchean pareceQuererCatalogoCompleto, la
              // señal de "quiere salir de la categoria actual, no verla nueva
              // vez". Sin este chequeo, categoriaDelLead seguia devolviendo
              // la categoria YA FIJADA en el lead (Zapatillas) y el bloque le
              // repetia sus subcategorias una y otra vez, sin dejarlo salir
              // - mismo criterio (quiereCatalogoCompleto) que ya usa, con
              // total confianza, el handler de la tool mostrar_categorias
              // (agente.js ~2633-2637) para el camino AI-driven.
              const quiereCatalogoCompletoAca = pareceQuererCatalogoCompleto(mensajeCliente) ||
pidioVerOtraCosaRespondiendoAlBot(ultimoAgente && ultimoAgente.contenido);
              const categoriaPedida = quiereCatalogoCompletoAca ? null : categoriaDelLead(productos, leadActual);
              // Un RUBRO con subcategorias (ej. "Camisas" -> Casuales/Manga
              // corta/De vestir) no tiene productos propios: mostrar_tarjeta_categoria
              // siempre le falla, y sin este chequeo el codigo caia al
              // respaldo de mas abajo (el menu COMPLETO de rubros) - el
              // mismo menu que el cliente ya habia visto, por lo que se
              // estaba quejando. Mismo patron que ya usa el handler de la
              // tool mostrar_categorias (agente.js ~2626-2657) para bajar de
              // nivel dentro de un rubro.
              const esRubroConSubcategorias = categoriaPedida && !categoriaPedida.padreId
                && subcategoriasDe(productos, categoriaPedida.id, leadActual).length;
              if (esRubroConSubcategorias) {
                const faltantesObligatoriosRubro = atributosFaltantes(categoriaPedida, leadActual, 'OBLIGATORIO');
                if (!faltantesObligatoriosRubro.length) {
                  const subcats = subcategoriasDe(productos, categoriaPedida.id, leadActual);
                  const listaSubcats = subcats.map((s, i) => `${i + 1}. ${s.nombre}`).join('\n');
                  logEtapa('cierre_pausado_para_navegar', telefonoCliente, { vuelta, rubro: categoriaPedida.nombre,
subcategorias: true });
                  return {
                    ok: true,
                    demo: false,
                    proveedor,
                    modelo,
                    respuesta: `Dentro de "${categoriaPedida.nombre}" tenemos:\n${listaSubcats}\n\n¿Cuál te interesa? Cuando
quieras seguimos con tu pedido.`,
                    fotos: fotosParaMostrar,
                  };
                }
                // Falta un atributo obligatorio propio del rubro (ej.
                // Genero): no es este el lugar de preguntarlo (estamos en
                // pleno cierre) - se deja caer al menu de rubros de
                // respaldo, mas abajo.
              }
              const yaMostradaPedida = categoriaPedida && ((leadActual.contexto || {}).tarjetasCategoriaMostradas ||
[]).includes(categoriaPedida.id);
```

```
              if (categoriaPedida && !esRubroConSubcategorias && !yaMostradaPedida) {
                const resultadoBrowse = await ejecutarFuncion({ name: 'mostrar_tarjeta_categoria', arguments: {} }, productos,
helpers);
                if (typeof resultadoBrowse === 'string' && resultadoBrowse.startsWith('TOOL_SUCCESS')) {
                  logEtapa('cierre_pausado_para_navegar', telefonoCliente, { vuelta, categoria: categoriaPedida.nombre });
                  return {
                    ok: true,
                    demo: false,
                    proveedor,
                    modelo,
                    respuesta: `Echale un vistazo a ${categoriaPedida.nombre.toLowerCase()} que tenemos 👆 Cuando quieras
seguimos con tu pedido.`,
                    fotos: fotosParaMostrar,
                  };
                }
                // TOOL_FAILED (sin stock, atributo obligatorio pendiente,
                // etc.) -> cae al menu de rubros de abajo como respaldo.
              }
              const arbolBrowse = arbolDeCategorias(productos, leadActual);
              if (arbolBrowse.length) {
                const lista = arbolBrowse.map((r, i) => `${i + 1}. ${r.nombre}`).join('\n');
                logEtapa('cierre_pausado_para_navegar', telefonoCliente, { vuelta, menu: true });
                return {
                  ok: true,
                  demo: false,
                  proveedor,
                  modelo,
                  respuesta: `Esto es lo que tenemos:\n${lista}\n\n¿Cuál te interesa? Cuando quieras seguimos con tu pedido.`,
                  fotos: fotosParaMostrar,
                };
              }
            }

            // Bug real reportado: el cliente ya habia visto el resumen y
            // contesto "si, todo" / "si" - pero la IA fallo las 3 vueltas sin
            // llamar a crear_pedido (a veces por texto de mas, a veces por
            // llamar de nuevo a confirmar_pedido en vez de crear_pedido), y
            // este mismo bloque le volvia a preguntar "¿confirmas que esta
            // todo bien?" IGUAL, como si el cliente no hubiera contestado
            // nada. Antes de repetir la pregunta: si no falta ningun dato
            // (primeraPreguntaDeCierre da null) se intenta CREAR el pedido
            // por codigo. Esto nunca depende de interpretar lo que dijo el
            // cliente en este mensaje - crear_pedido ya exige por su cuenta
            // que resumenConfirmado coincida EXACTO con el carrito actual
            // (ver el handler, unas lineas mas arriba), asi que si el
            // resumen nunca se confirmo de verdad, esto simplemente falla
            // silencioso y cae a la pregunta de siempre, sin ningun riesgo
            // de crear un pedido que el cliente no confirmo.
            const preguntaPendiente = primeraPreguntaDeCierre(leadActual, agente.config);
            if (!preguntaPendiente) {
              helpers.leadActual = leadActual;
```

```
              const resultadoCierre = await ejecutarFuncion({ name: 'crear_pedido', arguments: {} }, productos, helpers);
              if (typeof resultadoCierre === 'string' && resultadoCierre.startsWith('TOOL_SUCCESS')) {
                logEtapa('pedido_creado_forzado_preemptivo', telefonoCliente, { vuelta });
                return {
                  ok: true,
                  demo: false,
                  proveedor,
                  modelo,
                  respuesta: '¡Perfecto! Tu pedido quedó confirmado ✅ Ya lo estamos preparando. Cualquier duda, quedo
atento.',
                  fotos: fotosParaMostrar,
                };
              }
              // TOOL_FAILED (el resumen todavia no se confirmo de verdad, o
              // cambio algo desde entonces): antes de caer a la pregunta
              // generica de siempre, se intenta mostrar el resumen REAL
              // (items, precios, total) por codigo. Bug real reportado: sin
              // esto, un cliente podia llegar a "confirmado" sin haber visto
              // NUNCA el desglose - la IA fallaba las vueltas, caia a esta
              // pregunta vaga ("¿confirmas que esta todo bien?") sin decir
              // que era lo que habia que confirmar. El resumen mostrado a
              // partir de aca cuenta como "armado en este turno"
              // (resumenConfirmadoEnTurno queda en el turno actual, ver el
              // handler de confirmar_pedido) - recien el proximo mensaje del
              // cliente puede terminar de crear el pedido.
              const resultadoResumen = await ejecutarFuncion({ name: 'confirmar_pedido', arguments: {} }, productos, helpers);
              if (typeof resultadoResumen === 'string' && resultadoResumen.startsWith('TOOL_SUCCESS')) {
                const bloqueDatos = resultadoResumen
                  .replace(/^TOOL_SUCCESS\.[^\n]*\n/, '')
                  .split('\n\nTu texto ahora:')[0]
                  .trim();
                logEtapa('resumen_mostrado_forzado_preemptivo', telefonoCliente, { vuelta });
                return {
                  ok: true,
                  demo: false,
                  proveedor,
                  modelo,
                  respuesta: `Este es el resumen de tu pedido:\n${bloqueDatos}\n\n¿Confirmás que está todo bien para
procesarlo?`,
                  fotos: fotosParaMostrar,
                };
              }
              // TOOL_FAILED de confirmar_pedido tambien (ej. carrito vacio,
              // stock insuficiente detectado justo ahora) -> cae a la
              // pregunta de siempre, es un caso real que no se puede resolver
              // por codigo aca.
            }

            logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true, cierreForzado: true
});
            return {
```

```
              ok: true,
              demo: false,
              proveedor,
              modelo,
              respuesta: preguntaPendiente || 'Ya tengo todo listo para tu pedido — ¿confirmás que está todo bien para
procesarlo?',
              fotos: fotosParaMostrar,
            };
          }

          // Ademas de lo que ya marco buscar_producto (idsPedidosPorNombre),
          // se suma productoNombradoPorCliente directo: cubre el caso donde
          // el modelo ni siquiera intento mostrar_productos en ninguna vuelta
          // (ej. mando fotos sueltas con enviar_fotos_producto) pero el
          // cliente si nombro el producto en su mensaje.
          const idsNombrados = (helpers.idsPedidosPorNombre || []).length
            ? helpers.idsPedidosPorNombre
            : (productoNombradoPorCliente ? [productoNombradoPorCliente.id] : []);
          if (idsNombrados.length) {
            // El cliente nombro un producto puntual: el rescate muestra ESE,
            // sigue siendo mostrar_productos como antes.
            const idsForzados = idsNombrados.slice(0, MAX_PRODUCTOS_A_MOSTRAR);
            await ejecutarFuncion({ name: 'mostrar_productos', arguments: { idsProductos: idsForzados } }, productos, helpers);
            // El rescate puede no haber mostrado nada: ejecutarFuncion tiene
            // varios caminos que devuelven un aviso en vez de mandar tarjetas
            // (cupo de fotos del turno agotado, ids ya mostrados, envio
            // rechazado por WhatsApp). Antes ese resultado se descartaba y el
            // "Estas son las opciones 👆" salia igual, con el chat vacio -
            // bug real con capturas. Solo se afirma si hay tarjetas de verdad.
            if (helpers.tarjetasEnTurno) {
              logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true });
              return { ok: true, demo: false, proveedor, modelo, respuesta: conLinkPendienteSiFalta('Estas son las opciones que
tenemos 👆 ¿Alguna te interesa?', helpers), fotos: fotosParaMostrar };
            }
            logEtapa('forzado_sin_tarjetas', telefonoCliente, { vuelta, idsForzados });
            return { ok: true, demo: false, proveedor, modelo, respuesta: 'Perdón, no pude traerte las opciones en este
momento. ¿Te las busco de nuevo?', fotos: fotosParaMostrar };
          }
          if (candidatosActuales.length) {
            // Caso generico, sin nombre puntual: regla del negocio, se fuerza
            // la tarjeta de categoria, nunca productos sueltos (Fix C).
            const categoriaDelRescate = categoriaDelLead(productos, leadActual);
            // Bug real: sin categoria elegida todavia (candidatosActuales trae
            // TODO el catalogo sin filtrar en ese caso, no es señal de que haya
            // una categoria resuelta), categoriaDelRescate da null - la
            // tarjeta de categoria no aplica, hace falta el MENU de rubros
            // primero. Sin este chequeo, se intentaba mostrar_tarjeta_categoria
            // sin categoria, fallaba, y el cliente se quedaba con "no pude
            // traerte las opciones" sin haber visto ni el menu.
            if (!categoriaDelRescate) {
              const arbolRescate = arbolDeCategorias(productos, leadActual);
```

```
              if (arbolRescate.length) {
                const listaRescate = arbolRescate.map((r, i) => `${i + 1}. ${r.nombre}`).join('\n');
                logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true,
menuCategoriasForzado: true });
                return { ok: true, demo: false, proveedor, modelo, respuesta: `Esto es lo que
tenemos:\n${listaRescate}\n\n¿Cuál te interesa?`, fotos: fotosParaMostrar };
              }
            }
            // Rubro con subcategorias sin elegir todavia: la tarjeta de
            // categoria no aplica (el rubro no tiene productos propios). Se
            // calcula todo en codigo (mismo orden que mostrar_productos y
            // seccionProductos: atributo obligatorio del rubro primero,
            // subcategorias despues) en vez de parsear el texto de
            // mostrar_productos - asi se arma una respuesta real, nunca el
            // generico "tenemos varios tipos - cual te interesa" sin
            // opciones que se mandaba antes (bug real reportado).
            const esRubroConSubcategorias = Boolean(categoriaDelRescate && !categoriaDelRescate.padreId &&
subcategoriasDe(productos, categoriaDelRescate.id, leadActual).length);
            const yaMostroTarjeta = !esRubroConSubcategorias && ((leadActual.contexto || {}).tarjetasCategoriaMostradas ||
[]).includes(categoriaDelRescate?.id);
            if (esRubroConSubcategorias) {
              const faltantesRescate = atributosFaltantes(categoriaDelRescate, leadActual, 'OBLIGATORIO');
              if (faltantesRescate.length) {
                const deLaCategoriaRescate = productos.filter((p) => p.categoria?.id === categoriaDelRescate.id);
                const opcionesRescate = valoresRealesDeAtributo(deLaCategoriaRescate, faltantesRescate[0]);
                logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true,
rubroFaltaAtributo: true });
                return {
                  ok: true,
                  demo: false,
                  proveedor,
                  modelo,
                  respuesta: opcionesRescate.length
                    ? `Para mostrarte ${categoriaDelRescate.nombre.toLowerCase()}, ¿lo buscás de ${opcionesRescate.slice(0,
2).join(' o de ')}?`
                    : `Para mostrarte ${categoriaDelRescate.nombre.toLowerCase()}, ¿me confirmás
${faltantesRescate[0].toLowerCase()}?`,
                  fotos: fotosParaMostrar,
                };
              }
              const subsRescate = subcategoriasDe(productos, categoriaDelRescate.id, leadActual);
              const listaSubsRescate = subsRescate.map((s, i) => `${i + 1}. ${s.nombre}`).join('\n');
              logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true,
rubroConSubcategorias: true });
              return {
                ok: true,
                demo: false,
                proveedor,
                modelo,
                respuesta: `Dentro de "${categoriaDelRescate.nombre}" tenemos:\n${listaSubsRescate}\n\n¿Cuál te interesa?`,
                fotos: [],
```

```
              };
            }
            if (!yaMostroTarjeta) {
              const resultadoTarjeta = await ejecutarFuncion({ name: 'mostrar_tarjeta_categoria', arguments: {} }, productos,
helpers);
              const huboTarjeta = typeof resultadoTarjeta === 'string' && resultadoTarjeta.startsWith('TOOL_SUCCESS');
              logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true, tarjetaCategoria:
true, huboTarjeta });
              const nombreCat = categoriaDelRescate ? categoriaDelRescate.nombre.toLowerCase() : 'las opciones';
              return {
                ok: true,
                demo: false,
                proveedor,
                modelo,
                respuesta: huboTarjeta ? `Echale un vistazo a ${nombreCat} que tenemos 👆` : 'Perdón, no pude traerte las
opciones en este momento. ¿Te las busco de nuevo?',
                fotos: fotosParaMostrar,
              };
            }
            logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true, pideModeloPuntual:
true });
            const nombreCat = categoriaDelRescate ? categoriaDelRescate.nombre.toLowerCase() : 'los que tenemos';
            return { ok: true, demo: false, proveedor, modelo, respuesta: `¿Hay algún modelo en particular de ${nombreCat} que
te interese?`, fotos: fotosParaMostrar };
          }
          // No hay ningun candidato real todavia (falta un dato obligatorio):
          // en vez del texto vago del modelo, se le pide honestamente al
          // cliente la UNICA cosa que falta - nunca un "ya te aviso" vacio.
          const faltaInicial = preguntasInicialesFaltantes(agente.config, leadActual, productos)[0];
          const categoriaEnVuelta = categoriaDelLead(productos, leadActual);
          const faltaObligatorio = atributosFaltantes(categoriaEnVuelta, leadActual, 'OBLIGATORIO')[0];
          const pregunta = faltaInicial
            ? `Para poder mostrarte opciones, contame primero: ¿${faltaInicial.toLowerCase()}?`
            : !leadActual.categoriaInteres
              ? '¿Qué tipo de producto estás buscando?'
              : faltaObligatorio
                ? `Para mostrarte opciones, ¿me confirmás ${faltaObligatorio.toLowerCase()}?`
                : 'Contame un poco más qué estás buscando para poder ayudarte.';
          logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false, forzadoEnCodigo: true, sinCandidatos: true });
          return { ok: true, demo: false, proveedor, modelo, respuesta: pregunta, fotos: fotosParaMostrar };
        }
        logEtapa('respuesta_enviada', telefonoCliente, { vuelta, tools: false });
        return { ok: true, demo: false, proveedor, modelo, respuesta: conLinkPendienteSiFalta(resp.content || '...', helpers),
fotos: fotosParaMostrar };
      }
```

7.5 Tramo 5 — la IA llamó tools: ejecución, atajo sin segunda llamada y cierre de la función

generarRespuesta (tramo 5: ejecución de tools, respuesta_por_plantilla, agotó vueltas, catch)

Archivo: `lib/services/agente.js` *· líneas 4119–4207*

```
      mensajesTurno.push({ role: 'assistant', content: resp.content || null, tool_calls: resp.tool_calls });
      // Si esta vuelta tiene UNA sola tool call y esa tool devuelve un
      // resultado 100% determinista (ver "respuestaDirecta" en
      // ejecutarFuncion, ej. carrito), se salta la vuelta extra de la IA que
      // hoy solo serviria para redactar una frase de tramite sobre un
      // resultado que ya es texto fijo armado por el backend.
      let respuestaDirectaDelTurno = null;
      for (const toolCallOriginal of resp.tool_calls) {
        // FORZADO EN CODIGO: el cliente nombro un producto real puntual y el
        // modelo va a mostrar algo (mostrar_productos/ver_mas_productos) que
        // no lo incluye - se corrige el pedido en vez de confiar en que el
        // modelo eligio bien los IDs. No se toca ninguna otra tool.
        let toolCall = toolCallOriginal;
        if (productoNombradoPorCliente && (toolCall.name === 'mostrar_productos' || toolCall.name === 'ver_mas_productos')) {
          const idsPedidos = (toolCall.arguments?.idsProductos || []).map(Number);
          if (!idsPedidos.includes(productoNombradoPorCliente.id)) {
            logEtapa('producto_nombrado_por_cliente_forzado', telefonoCliente, { producto: productoNombradoPorCliente.nombre,
toolOriginal: toolCall.name });
            // Se marca como "pedido puntual" (mismo mecanismo que buscar_producto
            // ya usa) para que ademas se suprima el relleno de la pagina: el
            // cliente pidio UNO, no un pedido puntual mezclado con rellenos.
            helpers.idsPedidosPorNombre = [...new Set([...(helpers.idsPedidosPorNombre || []),
productoNombradoPorCliente.id])];
            toolCall = { ...toolCall, name: 'mostrar_productos', arguments: { idsProductos: [productoNombradoPorCliente.id] }
};
          }
        } else if (!productoNombradoPorCliente && !(helpers.idsPedidosPorNombre || []).length && (toolCall.name ===
'mostrar_productos' || toolCall.name === 'ver_mas_productos')) {
          // FIX B (regla del negocio): sin un producto puntual nombrado, el
          // modelo NO puede mandar tarjetas de productos individuales, aunque
          // el prompt ya se lo pida - esto es el backstop en codigo, mismo
          // espiritu que el bloque de arriba. Se redirige a la tarjeta de
          // categoria (o, si ya se mostro, se corta sin ejecutar nada y se le
          // pide al modelo que pregunte por el nombre puntual). El OR con
          // idsPedidosPorNombre es necesario: buscar_producto (linea ~2119)
          // marca ahi mismo un pedido puntual encontrado por nombre PARCIAL
          // ("las TEKKIRA CUP" para "ZAPATILLAS TEKKIRA CUP"), que
          // productoNombradoPorCliente no detecta porque exige el nombre
          // completo dentro del mensaje del cliente.
          const categoriaDelToolCall = categoriaDelLead(productos, leadActual);
          // Si es un rubro que todavia se divide en tipos (ej. "Calzado" con
          // "Botas"/"Sandalias"), la tarjeta de categoria no aplica (el rubro
          // en si no tiene productos propios) - se deja pasar el toolCall
          // original: el gate de rubro que YA existe dentro del handler de
          // mostrar_productos (agente.js linea ~1801) va a devolver la lista
```

```
          // de tipos, que es lo correcto en este punto.
          const esRubroConSubcategorias = Boolean(categoriaDelToolCall && !categoriaDelToolCall.padreId &&
subcategoriasDe(productos, categoriaDelToolCall.id, leadActual).length);
          const yaMostroTarjeta = ((leadActual.contexto || {}).tarjetasCategoriaMostradas ||
[]).includes(categoriaDelToolCall?.id);
          logEtapa('mostrar_productos_sin_puntual_redirigido', telefonoCliente, { categoria: categoriaDelToolCall?.nombre,
yaMostroTarjeta, esRubroConSubcategorias, toolOriginal: toolCall.name });
          if (!categoriaDelToolCall) {
            // Bug real: sin ninguna categoria elegida todavia, la tarjeta de
            // categoria no tiene de que categoria ser - hace falta el MENU de
            // rubros primero (mostrar_categorias ya tiene su propia logica
            // para armarlo bien, incluido el caso de un solo rubro).
            toolCall = { ...toolCall, name: 'mostrar_categorias', arguments: {} };
          } else if (esRubroConSubcategorias) {
            // dejar pasar sin tocar
          } else if (!yaMostroTarjeta) {
            toolCall = { ...toolCall, name: 'mostrar_tarjeta_categoria', arguments: {} };
          } else {
            mensajesTurno.push({ role: 'tool', tool_call_id: toolCallOriginal.id, content: `TOOL_FAILED: el cliente no nombro
un modelo puntual por su nombre - no le muestres tarjetas de productos individuales (ya vio la tarjeta de
"${categoriaDelToolCall?.nombre || 'esta categoria'}"). Preguntale, en texto, cual modelo puntual le interesa por su nombre.`
});
            continue;
          }
        }
        const resultado = await ejecutarFuncion(toolCall, productos, helpers);
        const esResultadoObjeto = resultado && typeof resultado === 'object';
        const paraModelo = esResultadoObjeto ? resultado.paraModelo : resultado;
        mensajesTurno.push({ role: 'tool', tool_call_id: toolCallOriginal.id, content: paraModelo || 'Hecho.' });
        if (resp.tool_calls.length === 1 && esResultadoObjeto && resultado.respuestaDirecta) {
          respuestaDirectaDelTurno = resultado.respuestaDirecta;
        }
      }

      if (respuestaDirectaDelTurno) {
        logEtapa('respuesta_por_plantilla', telefonoCliente, { vuelta, tool: resp.tool_calls[0].name });
        return {
          ok: true, demo: false, proveedor, modelo,
          respuesta: conLinkPendienteSiFalta(respuestaDirectaDelTurno, helpers),
          fotos: fotosParaMostrar,
        };
      }
    }
    logEtapa('respuesta_enviada', telefonoCliente, { vuelta: MAX_VUELTAS, agotoVueltas: true });
    return { ok: true, demo: false, proveedor, modelo, respuesta: 'Un momento, ya te ayudo con eso.', fotos: fotosParaMostrar
};
  } catch (err) {
    // Falla real del proveedor de IA (o de una tool durante el loop): NUNCA
    // se usa respuestaDemo() aca, porque esa funcion hace matching simple de
    // texto y podria "afirmar" disponibilidad sin haber consultado el
    // catalogo de verdad para esta conversacion puntual. Mejor ser honesto.
```

```
    logErrorEtapa(proveedor === 'openai' || proveedor === 'anthropic' ? 'proveedor_ia' : 'generacion', telefonoCliente, err);
    return { ok: true, demo: false, error: true, proveedor: 'error', respuesta: respuestaErrorTecnico() };
  }
}
```

7.6 Detectores (funciones puras, sin llamadas a IA)

pidePreferenciaSinMostrar

Archivo: `lib/services/agente.js` · líneas 919–937

```
/**
 * Detecta que el modelo esta pidiendo una preferencia (color, talla, marca,
 * presupuesto) en vez de mostrar lo que ya tiene listo para mostrar.
 *
 * El dueño fue explicito: primero se muestra, despues se afina. Un cliente que
 * todavia no vio una foto y ya tiene que contestar de que color la quiere,
 * abandona.
 */
function pidePreferenciaSinMostrar(texto) {
  if (!texto || !texto.includes('?')) return false;
  const t = normalizarSimple(texto);
  const atributo = '(color|talla|tamano|numero|marca|presupuesto)';
  const pedir = '(prefier|prefer|busca|quer|gusta|usas|necesit|interesa|estas)';
  // "¿que color?" / "en que color las buscas" / "que talla usas" y tambien el
  // orden invertido: "necesito saber el color que preferis".
  return new RegExp(`(que|cual|en que)\\s+${atributo}`).test(t)
    || new RegExp(`${atributo}[^.?!]{0,30}${pedir}`).test(t)
    || new RegExp(`${pedir}[^.?!]{0,30}${atributo}`).test(t);
}
```

pareceInterrogatorio

Archivo: `lib/services/agente.js` · líneas 939–950

```
/**
 * Detecta que el modelo mando VARIAS preguntas en un mismo mensaje (el
 * "interrogatorio" que el negocio no quiere). Paso en produccion: el bot
 * pidio marca, ocasion y talla juntas, en una lista con viñetas.
 *
 * Se cuentan solo los cierres de pregunta: dos o mas en un mismo mensaje ya
 * es un formulario. Un mensaje normal de venta tiene una sola.
 */
function pareceInterrogatorio(texto) {
  if (!texto) return false;
  return (String(texto).match(/\?/g) || []).length >= 2;
}
```

pareceAnuncioDeBusqueda

Archivo: `lib/services/agente.js` *· líneas 952–961*

```
/**
 * Detecta que el modelo prometio buscar en vez de mostrar ("dame un momento",
 * "voy a buscar"). Es un mensaje que deja al cliente esperando algo que nunca
 * llega, porque el turno termina ahi.
 */
function pareceAnuncioDeBusqueda(texto) {
  if (!texto) return false;
  const t = normalizarSimple(texto);
  return /(dame un momento|un momentito|ya te (muestro|paso|busco|env[ií]o)|voy a
(buscar|revisar|ver|mostrarte|consultar)|estoy buscando|permiteme (un|revisar|corregirlo)|enseguida te|te mostrare|te (voy a )?
mostrar[ée])/.test(t);
}
```

afirmaFaltaDeStockSinRespaldo

Archivo: `lib/services/agente.js` *· líneas 999–1019*

```
/**
 * Detecta que el modelo afirma, en texto libre, que no hay stock/disponibilidad
 * de algo. No es lo mismo que un "no lo tenemos" dicho DESPUES de una consulta
 * real (buscar_producto, mostrar_productos, mostrar_categorias devolviendo
 * vacio) - ese caso es legitimo y no pasa por este detector.
 *
 * Bug real con capturas: el bot dijo "no tenemos zapatillas en stock" y, en el
 * mismo tramo de la conversacion, mostro tarjetas reales de zapatillas. Nada
 * en codigo validaba esa frase antes de mandarla - se uso el mismo criterio
 * que "antes de decir que no lo tenemos, buscalo" (ya en el prompt), pero
 * como backstop de codigo, igual que los demas detectores de este archivo.
 */
function afirmaFaltaDeStockSinRespaldo(texto) {
  if (!texto) return false;
  const t = normalizarSimple(texto);
  // La ventana NO cruza comas a proposito: "no hay problema, tenemos
  // disponible ese producto" es una afirmacion de stock, no una negacion -
  // sin el corte por coma, el detector la marcaba como falsa igual (bug real
  // encontrado en revision de codigo, verificado corriendo la funcion).
  return /(no (tenemos|hay|contamos con|disponemos de)\b[^,.?!]{0,25}(stock|disponible|disponibilidad|en existencia)|no tengo
(la capacidad|como|forma) (de|para)[^,.?!]{0,40}(mostrar|ver|consultar|acceder)[^,.?!]{0,30}
(inventario|stock|productos|catalogo))/.test(t);
}
```

afirmaQueMostroAlgoSinTarjeta

Archivo: `lib/services/agente.js` *· líneas 1021–1036*

```
/**
 * Detecta que el bot le dice al cliente que le mando/mostro algo (tarjeta,
 * catalogo, opciones) cuando en este turno NO se mando ninguna tarjeta real
 * (ni de producto ni de categoria). Reusa el mismo criterio que ya pide el
```

```
 * prompt ("AQUI TIENES SOLO SI DE VERDAD MANDASTE ALGO", agente.js linea
 * ~1505) pero como backstop en codigo: si mostrar_tarjeta_categoria fallo
 * (TOOL_FAILED, ej. el envio real por WhatsApp no salio), el modelo puede
 * igual escribir un texto confiado como si hubiera mandado algo - bug real
 * reportado: "Genial, te muestro las zapatillas... Aqui puedes revisar las
 * opciones." sin ninguna imagen ni link adjunto.
 */
function afirmaQueMostroAlgoSinTarjeta(texto) {
  if (!texto) return false;
  const t = normalizarSimple(texto);
  return /(te muestro|(aqui|aca) (tienes|tenes|puedes revisar|te dejo|las tienes|van( los)?)|te paso|te envie|te
comparto|echale un vistazo|hechale un vistazo|dale un vistazo|revisa las opciones|mira las opciones|dejame mostrarte|estas son
las opciones)/.test(t);
}
```

afirmaFalloTecnicoInventado

Archivo: `lib/services/agente.js` *· líneas 1038–1061*

```
/**
 * El espejo de afirmaQueMostroAlgoSinTarjeta: en vez de inventar un exito
 * que no paso, el modelo inventa un FALLO tecnico que tampoco paso, como
 * excusa para no seguir. Bug real reportado con capturas: cliente agrego un
 * jogger al carrito, dijo "no nada mas" (queria pasar al cierre) y el bot
 * respondio "Parece que hubo un inconveniente al enviar las opciones de los
 * joggers. ¿Te parece si intento mandartelas de nuevo?" - un TOOL_FAILED que
 * nunca ocurrio (no se llamo a ninguna tool en absoluto ese turno). Se
 * detecta el texto por regex y se cruza, en el call site, contra si de
 * verdad hubo un TOOL_FAILED real en este turno (ver mensajesTurno) - la
 * misma disciplina anti-invencion que el resto del archivo, solo que para
 * fallas en vez de para exitos.
 */
function afirmaFalloTecnicoInventado(texto) {
  if (!texto) return false;
  const t = normalizarSimple(texto);
  // Bug real probando en vivo (segunda vez): "¡Perdon por el inconveniente,
  // parece que no se envio la informacion de las camisas!" - una variante
  // que la version anterior de este regex no cubria (no dice "hubo un
  // inconveniente" ni "parece que hubo", dice "perdon por el inconveniente"
  // y "parece que no se envio"). Mismo fallo inventado, otra forma de
  // decirlo.
  return /(hubo un (inconveniente|problema|error)|parece que hubo|perdon por el inconveniente|parece que no se
(envio|mando|llego)|no se (envio|mando)\b|intento (mandartelas?|enviartelas?|mostrartelas?|mandarlas?|enviarlas?|mostrarlas?
|pasartelas?) de nuevo|no se pudo (enviar|mandar|mostrar))/.test(t);
}
```

afirmaPlazoDeEntregaInventado

Archivo: `lib/services/agente.js` *· líneas 1063–1078*

```
/**
 * Afirma un plazo de entrega concreto ("2 a 3 dias habiles", "en 24 horas",
```

```
 * "la proxima semana") que el sistema no tiene forma de saber: no existe
 * ningun campo de tiempo de entrega en toda la base (ver
 * prisma/schema.prisma) - cualquier plazo que el modelo diga es 100%
 * inventado. Bug real probando en vivo: el cliente pregunto "¿en cuanto
 * tiempo me llegara?" ya con el pedido creado, y el bot contesto "entre 2 a
 * 3 dias habiles" de la nada. Lo correcto es decir que un asesor real lo va
 * a confirmar (mismo patron que crear_pedido ya le dice: "un asesor
 * coordinara la entrega").
 */
function afirmaPlazoDeEntregaInventado(texto) {
  if (!texto) return false;
  const t = normalizarSimple(texto);
  return /(entrega[^.?!]{0,25}(suele|tarda|demora|toma)|tardar[^.?!]{0,20}\d|lleg(a|ara|aria)\s+en\s+\d|\d+\s*a\s*\d+\s*(dias?
|horas?|semanas?)|dentro de\s+\d+\s*(dias?|horas?|semanas?))/.test(t);
}
```

pareceListadoDeProductosEnTexto

Archivo: `lib/services/agente.js` *· líneas 841–857*

```
// La ficha que ve el CLIENTE por WhatsApp: solo lo que le sirve para decidir
// (precio, marca, material si los hay, tallas/colores reales agrupados por
// talla en vez de una linea por cada combinacion). Categoria, tipo, corte,
// estilo, genero, ocasion, temporada son datos internos para que la IA
// filtre/recomiende (ver textoCompletoProducto/scoreProducto), no le
// interesan al cliente en la tarjeta y solo suman ruido.
// Detecta cuando el modelo escribio una lista de productos con precio como
// texto plano en vez de llamar mostrar_productos (regla ya prohibida en el
// prompt, esto es el backstop en codigo). Exige VARIOS precios Y varias
// marcas de lista/vinieta en el mismo texto para evitar falsos positivos
// con un mensaje normal que solo menciona un precio de pasada.
function pareceListadoDeProductosEnTexto(texto) {
  if (!texto) return false;
  const precios = (texto.match(/\d+[.,]\d{2}\b/g) || []).length;
  const itemsDeLista = (texto.match(/^\s*(\d+[.)]|-|·)\s/gm) || []).length;
  return precios >= 2 && itemsDeLista >= 2;
}
```

nombraUnProductoReal

Archivo: `lib/services/agente.js` *· líneas 883–894*

```
/**
 * ¿El texto nombra un producto del catalogo? Se compara contra los nombres
 * reales de los candidatos, normalizados, para no depender de que el modelo
 * escriba exactamente igual.
 */
function nombraUnProductoReal(texto, candidatos = []) {
  if (!texto) return null;
  for (const p of candidatos) {
    if (nombreDeProductoEnTexto(texto, p.nombre)) return p.nombre;
  }
```

```
  return null;
}
```

mencionaModeloInventado

Archivo: `lib/services/agente.js` *· líneas 896–917*

```
/**
 * Lo opuesto de nombraUnProductoReal: detecta cuando el modelo INVENTA un
 * nombre de modelo que no existe en el catalogo, tipicamente al dar
 * "ejemplos" de memoria en vez de mostrar el bloque de resultados real.
 * Bug real probando en vivo: el cliente eligio "Zapatillas urbanas" y el
 * bot respondio "tenemos modelos como *Zapatillas Urbanas Flex* o
 * *Zapatillas CityWalk*" - ninguno de los dos existe en el catalogo (los
 * reales eran Park St 2.0, Superstar 80 ADV, Drop Step low 2.0, TEKKIRA
 * CUP). Se busca el patron tipico de "improvisar ejemplos" (modelos
 * como/por ejemplo + nombres resaltados con * o comillas) y se verifica
 * cada nombre contra los candidatos reales - si ALGUNO no matchea ningun
 * producto real, es invencion.
 */
function mencionaModeloInventado(texto, candidatos = []) {
  if (!texto) return null;
  const m = texto.match(/(?:modelos? como|por ejemplo,?)\s+([^.!?\n]{4,200})/i);
  if (!m) return null;
  const nombres = [...m[1].matchAll(/[*"“]([^*"”]{4,60})[*"”]/g)].map((x) => x[1]);
  if (!nombres.length) return null;
  const inventado = nombres.find((n) => !candidatos.some((p) => nombreDeProductoEnTexto(n, p.nombre) ||
nombreDeProductoEnTexto(p.nombre, n)));
  return inventado || null;
}
```

pideDatosDeCierre

Archivo: `lib/services/agente.js` *· líneas 963–978*

```
/**
 * Detecta que el bot esta pidiendo datos de CIERRE (entrega, pago, nombre,
 * direccion). Sirve para frenarlo cuando todavia no sabe QUE combinacion
 * exacta se lleva el cliente.
 *
 * Bug real con capturas: el cliente contesto "si" a "¿alguna te interesa?" y
 * el bot salto directo a "¿te lo enviamos a domicilio?" sin haberle
 * preguntado talla ni color. El pedido se armaba a ciegas: agregar_al_carrito
 * SI exige la variante, pero el modelo puede llegar al cierre conversando sin
 * pasar nunca por el carrito.
 */
function pideDatosDeCierre(texto) {
  if (!texto) return false;
  const t = normalizarSimple(texto);
  return /(a domicilio|lo (enviamos|mandamos|llevamos)|retiro en (la )?tienda|pasas a (buscarlo|recogerlo)|forma de pago|metodo
de pago|como (lo )?(vas a |queres |prefieres )?pagar|a nombre de quien|cual es tu nombre|tu direccion|cual es la
```

```
direccion|compartirme tu ubicacion)/.test(t);
}
```

8. Modelos de datos involucrados (Prisma)

model ClienteFinal

Archivo: `prisma/schema.prisma` *· líneas 638–718*

```
model ClienteFinal {
  id        Int      @id @default(autoincrement())
  empresaId Int
  empresa   Empresa  @relation(fields: [empresaId], references: [id], onDelete: Cascade)
  nombre    String?
  // Nombre de perfil de WhatsApp (contacts[0].profile.name del webhook de
  // Meta) - siempre disponible desde el primer mensaje, a diferencia de
  // "nombre" (que recien se llena cuando el cliente lo confirma para el
  // pedido). Se usa solo para mostrar en el panel, nunca para datos de envio.
  nombrePerfil String?
  telefono  String
  createdAt DateTime @default(now())

  // Estado de la conversacion de ventas (equivalente al "lead" del bot):
  // se actualiza por codigo (extraccion determinista) y via la tool
  // actualizar_datos_lead del agente. Se reinicia solo si el cliente cambia
  // de opinion, nunca automaticamente.
  categoriaInteres String?
  // Categoria real (de esta empresa) a la que se resolvio categoriaInteres,
  // para saber sus atributos obligatorios (ver Categoria/CategoriaAtributo).
  // categoriaInteres se mantiene como el texto literal que dijo el cliente.
  categoriaId      Int?
  // Texto literal que dijo el cliente ("hasta 500 bs", "unos 300").
  presupuesto      String?
  // El mismo presupuesto ya interpretado a numero por el backend. El texto
  // solo sirve para repetirselo al cliente; el filtrado real usa estos dos,
  // asi la busqueda no depende de reparsear un string en cada consulta.
  presupuestoMin   Decimal? @db.Decimal(10, 2)
  presupuestoMax   Decimal? @db.Decimal(10, 2)
  cantidad         String?
  direccionEntrega String?
  // Domicilio o retiro en tienda. Sin esto no se puede crear el pedido: son
  // dos flujos distintos (uno necesita direccion del cliente, el otro manda
  // la ubicacion real del local).
  tipoEntrega      TipoEntrega?
  // Coordenadas reales cuando el cliente comparte su ubicacion de WhatsApp
  // (mucho mas confiable que una direccion escrita a mano para coordinar
  // la entrega). direccionEntrega igual queda con un texto legible/link.
  ubicacionLat     Float?
  ubicacionLng     Float?
  observaciones    String?
  // Preferencias puntuales de producto que el cliente fue mencionando.
  marca            String?
  talla            String?
  color            String?
```

```
  // Como va a pagar, elegida entre las formas de pago que esa tienda tiene
  // habilitadas (ver AgenteConfig.aceptaQr/aceptaEfectivo/aceptaTarjeta).
  // crear_pedido no deja crear el pedido sin esto (mismo criterio que
  // direccionEntrega).
  formaPago        FormaPago?
  nivelInteres     NivelInteres @default(FRIO)
  estadoLead       EstadoLead   @default(NUEVO)
  // Estado de la conversacion de venta (donde esta el cliente en el proceso
  // de decision). Lo actualiza la IA via actualizar_datos_lead; el codigo lo
  // usa para cambiar el comportamiento del prompt (ej: si ya eligio un
  // producto favorito, dejar de ofrecerle mas opciones y enfocarse en cerrar).
  estadoConversacion EstadoConversacion @default(EXPLORANDO)
  // IDs de productos ya mostrados (para no repetirlos) y descartados
  // explicitamente por el cliente (para no volver a sugerirlos), y el
  // producto que el cliente eligio como favorito (el motor deja de buscar
  // alternativas y se enfoca en cerrar la venta de ese producto).
  productosMostrados    Int[] @default([])
  productosDescartados  Int[] @default([])
  productoFavoritoId    Int?
  // La combinacion exacta (talla+color) que eligio, no solo el producto: el
  // pedido siempre termina apuntando a una variante real.
  varianteFavoritaId    Int?
  // Datos efimeros del bot (ej. que productos ya se les envio foto, para
  // no repetir el envio). Libre para que cualquier bot vertical lo use.
  contexto         Json         @default("{}")
  // Valores de atributos definidos por la categoria (ver CategoriaAtributo)
  // que no son ya un campo propio de este modelo (ej: {"Genero":"Hombre",
  // "Uso":"Running"}). Talla/color/marca siguen usando sus campos de siempre.
  atributosLead    Json         @default("{}")
  updatedAt        DateTime     @updatedAt

  pedidos Pedido[]

  @@unique([empresaId, telefono])
  @@map("clientes_finales")
}
```

model Conversacion

Archivo: `prisma/schema.prisma` *· líneas 448–477*

```
// Una conversacion agrupa los mensajes de un mismo cliente dentro de una
// ventana de tiempo. Se cobra 1 conversacion, no 1 por mensaje.
model Conversacion {
  id              Int      @id @default(autoincrement())
  agenteId        Int
  agente          Agente   @relation(fields: [agenteId], references: [id], onDelete: Cascade)
  telefonoCliente String
  origen          OrigenUso @default(INCLUIDA) // de donde se cobro
  // Mientras un humano tiene el control, el agente de IA NO responde este
  // chat (se libera solo manualmente, ver /panel/conversaciones/:id/control).
  modo            ModoConversacion @default(IA)
```

```
  tomadaPorId     Int?
  tomadaPor       Usuario?  @relation(fields: [tomadaPorId], references: [id], onDelete: SetNull)
  ultimoMensajeAt DateTime  @default(now())
  createdAt       DateTime  @default(now())

  // Atribucion a anuncios de Meta (Click to WhatsApp): WhatsApp manda estos
  // datos solos, sin conectar nada, en el primer mensaje de una conversacion
  // que arranco desde un anuncio (objeto "referral" del webhook). Quedan en
  // null si el cliente escribio por su cuenta.
  anuncioId        String?
  anuncioTitulo    String?
  anuncioImagenUrl String?

  mensajes Mensaje[]
  pedidos  Pedido[]

  @@index([agenteId, telefonoCliente, ultimoMensajeAt])
  @@map("conversaciones")
}
```

model Mensaje

Archivo: `prisma/schema.prisma` · líneas 484–502

```
model Mensaje {
  id             Int          @id @default(autoincrement())
  conversacionId Int
  conversacion   Conversacion @relation(fields: [conversacionId], references: [id], onDelete: Cascade)
  rol            RolMensaje
  contenido      String
  // Cuando un miembro del equipo escribe manualmente (modo HUMANO), para
  // saber quien mando que en el historial.
  usuarioId      Int?
  usuario        Usuario?     @relation(fields: [usuarioId], references: [id], onDelete: SetNull)
  // Adjunto opcional (foto o audio) mandado por el cliente o por un humano
  // del equipo. mediaTipo: "imagen" | "audio".
  mediaUrl       String?
  mediaTipo      String?
  createdAt      DateTime     @default(now())

  @@index([conversacionId, createdAt])
  @@map("mensajes")
}
```

model Pedido

Archivo: `prisma/schema.prisma` · líneas 764–795

```
model Pedido {
  id               Int           @id @default(autoincrement())
  empresaId        Int
  empresa          Empresa       @relation(fields: [empresaId], references: [id], onDelete: Cascade)
```

```
  clienteId        Int?
  cliente          ClienteFinal? @relation(fields: [clienteId], references: [id])
  // De que conversacion salio este pedido (para saber si vino de un anuncio
  // de Meta, ver Conversacion.anuncioId). Null en pedidos viejos o creados
  // sin conversacion asociada.
  conversacionId   Int?
  conversacion     Conversacion? @relation(fields: [conversacionId], references: [id], onDelete: SetNull)
  total            Decimal       @default(0) @db.Decimal(10, 2)
  // Copiada de ClienteFinal al crear el pedido, igual que formaPago. En un
  // pedido con RECOJO la direccion queda vacia a proposito (retira el
  // cliente en el local).
  tipoEntrega      TipoEntrega?
  direccionEntrega String?
  entregaLat       Float?
  entregaLng       Float?
  notas            String?
  // Copiado de ClienteFinal.formaPago al momento de crear el pedido. Solo
  // registra que eligio el cliente - el cobro en si lo verifica un humano
  // por fuera del bot (ver /panel/pedidos/:id/estado).
  formaPago        FormaPago?
  estado           EstadoPedido  @default(NUEVO)
  createdAt        DateTime      @default(now())

  items PedidoItem[]

  @@index([empresaId, estado])
  @@map("pedidos")
}
```

model PedidoItem

Archivo: `prisma/schema.prisma` · líneas 797–815

```
// Linea de un pedido. Guarda nombre y precio al momento de la compra
// (denormalizado) para que el historial no cambie si el producto se edita
// o se borra despues.
model PedidoItem {
  id         Int       @id @default(autoincrement())
  pedidoId   Int
  pedido     Pedido    @relation(fields: [pedidoId], references: [id], onDelete: Cascade)
  productoId Int?
  producto   Producto? @relation(fields: [productoId], references: [id], onDelete: SetNull)
  // Si el producto tenia variantes, cual combinacion exacta se vendio.
  varianteId Int?
  variante   Variante? @relation(fields: [varianteId], references: [id], onDelete: SetNull)
  nombre     String
  precio     Decimal   @db.Decimal(10, 2)
  cantidad   Int       @default(1)

  @@index([pedidoId])
```

```
  @@map("pedido_items")
}
```

model Pago

Archivo: `prisma/schema.prisma` · líneas 826–846

```
model Pago {
  id         Int        @id @default(autoincrement())
  empresaId  Int
  empresa    Empresa    @relation(fields: [empresaId], references: [id], onDelete: Cascade)
  tipo       TipoPago
  monto      Decimal    @db.Decimal(10, 2)
  moneda     String     @default("BOB") // BOB (bolivianos) | USD
  estado     EstadoPago @default(PENDIENTE)
  referencia String? // id de la pasarela / comprobante
  paqueteId  Int?
  paquete    Paquete?       @relation(fields: [paqueteId], references: [id])
  // Vinculo exacto con la compra que este pago paga (solo tipo PAQUETE): al
  // confirmar/rechazar este pago, se activa/rechaza ESA compra puntual, sin
  // adivinar cual por fecha o cantidad.
  compraPaqueteId Int?
  compraPaquete   CompraPaquete? @relation(fields: [compraPaqueteId], references: [id])
  fecha      DateTime   @default(now())

  @@index([empresaId, estado])
  @@map("pagos")
}
```

model UsoIA

Archivo: `prisma/schema.prisma` · líneas 221–242

```
// Costo real de IA que paga Proshop (no el cliente) por cada llamada a
// OpenAI/Anthropic. Ver docs del dashboard interno /admin/costo-ia.
model UsoIA {
  id        Int     @id @default(autoincrement())
  empresaId Int
  empresa   Empresa @relation(fields: [empresaId], references: [id], onDelete: Cascade)
  // referencia suelta, sin FK, solo para poder ubicar el turno si hace falta
  conversacionId Int?
  proveedor      String // "openai" | "anthropic"
  modelo         String
  // "respuesta" (vuelta 0, turno normal) | "reintento_forzado" (vuelta > 0,
  // el sistema corrigio la vuelta anterior via RECORDATORIO DEL SISTEMA)
  proposito        String   @default("respuesta")
  vuelta           Int      @default(0)
  promptTokens     Int
  cachedTokens     Int      @default(0)
  completionTokens Int
  createdAt        DateTime @default(now())
```

```
  @@index([empresaId, createdAt])
  @@map("usos_ia")
}
```